

TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN



KHÓA LUẬN TỐT NGHIỆP

**XÂY DỰNG HỆ THỐNG GỢI Ý PHIM SỬ DỤNG HỆ
THỐNG ĐA TÁC NHÂN**

Chuyên ngành: Khoa học dữ liệu

NGUYỄN KHẮC LUẬT – 21099741

NGUYỄN THỊ TY TY – 21096511

Giảng viên hướng dẫn: TS NGUYỄN CHÍ KIÊN

TP. Hồ Chí Minh, tháng 12 năm 2025

**INDUSTRIAL UNIVERSITY OF HO CHI MINH CITY
FACULTY OF INFORMATION TECHNOLOGY**



GRADUATION THESIS

**A MULTI-AGENT SYSTEM FOR MOVIE
RECOMMENDATION**

Major: Data Science

NGUYEN KHAC LUAT – 21099741

NGUYEN THI TY TY – 21096511

Instructor: PhD. NGUYEN CHI KIEN

Ho Chi Minh City, December 2025

A MULTI-AGENT SYSTEM FOR MOVIE RECOMMENDATION

Abstract

Large Language Model (LLM)-based agents have gained considerable attention due to their decision-making skills and proven ability to handle complex tasks. These agents utilize semantic understanding, planning, and decision-making capabilities, which unlock new potential for more nuanced recommendations. While researchers have begun utilizing agents for recommendation, existing work often focuses primarily on simulating user or item behaviors, or attempts to use single agents, which can struggle with complex decision-making tasks within recommendation scenarios. Recognizing this gap in leveraging agent capabilities for effective multi-agent collaboration in recommendation systems, we propose a novel framework designed to enhance recommendation systems through collaborative efforts.

This system differs significantly from previous studies focused on user/item simulation by aiming to deploy specialized multi-agents to tackle recommendation tasks directly. The framework is built on the belief that multi-agent collaboration, similar to human workflows, can accomplish complex tasks more effectively through collective intelligence. Recommendation tasks are addressed through the collaborative efforts of several specialized agents. These agents, built on LLMs, include a Manager to plan and manage the task execution, a Reflector to critique and reflect on previous errors, User/Item Analysts to understand user preferences and item attributes, a Searcher to retrieve additional information using external tools, and a Task Interpreter to translate dialogues into executable recommendation tasks.

Furthermore, we demonstrate the broad applicability of this flexible architecture by providing application examples on diverse recommendation tasks. These include rating prediction, sequential recommendation, conversational recommendation, and the explanation generation of recommendation results. The system allows developers to easily select and customize agents for collaboration based on the varying requirements of different scenarios. The complete framework and a demonstration video are publicly available.

XÂY DỰNG HỆ THỐNG GỢI Ý PHIM SỬ DỤNG HỆ THỐNG ĐA TÁC NHÂN

Tóm Tắt

Các tác nhân (agent) dựa trên Mô hình ngôn ngữ lớn (Large Language Model – LLM) đang thu hút sự quan tâm đáng kể nhờ năng lực ra quyết định và khả năng đã được chứng minh trong việc xử lý các tác vụ phức tạp. Những tác nhân này khai thác khả năng hiểu ngữ nghĩa, lập kế hoạch và ra quyết định, từ đó mở ra tiềm năng mới cho các hệ thống gợi ý có mức độ tinh vi và cá nhân hóa cao hơn. Mặc dù các nghiên cứu gần đây đã bắt đầu ứng dụng Agent trong bài toán gợi ý, phần lớn các công trình hiện tại chủ yếu tập trung vào việc mô phỏng hành vi người dùng hoặc vật phẩm, hoặc chỉ sử dụng một agent đơn lẻ, vốn gặp nhiều hạn chế khi phải giải quyết các bài toán ra quyết định phức tạp trong bối cảnh hệ thống gợi ý. Nhận thấy khoảng trống này trong việc khai thác năng lực của agent cho sự phối hợp đa tác nhân (multi-agent) một cách hiệu quả, chúng em đề xuất một khuôn khổ (framework) mới nhằm nâng cao hiệu suất của hệ thống gợi ý thông qua cơ chế cộng tác.

Hệ thống được đề xuất khác biệt rõ rệt so với các nghiên cứu trước đây vốn tập trung vào mô phỏng người dùng/vật phẩm, khi hướng tới việc triển khai các đa tác nhân chuyên biệt để trực tiếp giải quyết các nhiệm vụ gợi ý. Khuôn khổ này được xây dựng trên giả định rằng sự phối hợp của nhiều agent, tương tự như quy trình làm việc của con người, có thể xử lý các tác vụ phức tạp một cách hiệu quả hơn nhờ trí tuệ tập thể. Các nhiệm vụ gợi ý được giải quyết thông qua sự hợp tác của nhiều agent chuyên biệt dựa trên LLM, bao gồm: Manager chịu trách nhiệm lập kế hoạch và điều phối quá trình thực thi; Reflector đảm nhiệm việc đánh giá, phản biện và rút kinh nghiệm từ các sai sót trước đó; User/Item Analysts phân tích sở thích người dùng và đặc trưng của vật phẩm; Searcher truy xuất thông tin bổ sung thông qua các công cụ bên ngoài; và Task Interpreter chuyển đổi các hội thoại thành các tác vụ gợi ý có thể thực thi.

Bên cạnh đó, chúng em chứng minh tính ứng dụng rộng rãi của kiến trúc linh hoạt này thông qua các ví dụ trên nhiều bài toán gợi ý khác nhau, bao gồm dự đoán điểm đánh giá (rating prediction), gợi ý theo chuỗi (sequential recommendation), gợi ý hội thoại (conversational recommendation) và sinh diễn giải cho kết quả gợi ý (explanation generation). Hệ thống cho phép các nhà phát triển dễ dàng lựa chọn và tùy chỉnh các agent tham gia cộng tác, tùy theo yêu cầu cụ thể của từng kịch bản ứng dụng. Toàn bộ khuôn khổ và video minh họa đã được công bố công khai.

LỜI CẢM ƠN

Đầu tiên, chúng em xin gửi lời cảm ơn chân thành đến quý thầy cô trong Khoa Công nghệ thông tin – Trường Đại học Công nghiệp TP. Hồ Chí Minh, những người đã luôn tạo điều kiện thuận lợi cho chúng em trong suốt quá trình học tập, nghiên cứu và hoàn thành khóa luận tốt nghiệp. Sự tận tâm giảng dạy và nhiệt huyết của các thầy cô là hành trang quý báu giúp chúng em trưởng thành hơn từng ngày trên con đường học vấn.

Chúng em đặc biệt gửi lời cảm ơn đến TS. Nguyễn Chí Kiên, giảng viên hướng dẫn của đề tài, người đã luôn theo sát, tận tình chỉ bảo, góp ý từng chi tiết và hỗ trợ chúng em hết mình trong suốt quá trình thực hiện khóa luận. Chính nhờ những định hướng đúng đắn và những chia sẻ quý báu của Thầy mà chúng em có thể hoàn thành đồ án một cách hoàn chỉnh và thành công nhất trong khả năng của mình.

Chúng em cũng xin cảm ơn Ban Giám hiệu nhà trường đã luôn quan tâm, hỗ trợ và tạo điều kiện để sinh viên có môi trường học tập, nghiên cứu hiệu quả và tích cực. Những hỗ trợ này chính là động lực để chúng em học tập và theo đuổi đam mê công nghệ của mình.

Một lần nữa, chúng em xin chân thành cảm ơn tất cả Quý Thầy cô đã đồng hành, chỉ dẫn và truyền cảm hứng cho chúng tôi trong suốt hành trình học tập vừa qua. Mọi sự giúp đỡ đều là một phần quan trọng góp nên thành quả hôm nay.

Trân trọng!

[illegible]

(Ký ghi rõ họ tên)

This image shows a full page of white paper with horizontal ruling lines. The lines are evenly spaced and extend across the width of the page. There are no margins, text, or other markings on the paper.

(Ký ghi rõ họ tên)

NHẬN XÉT VÀ ĐÁNH GIÁ CỦA GIÁO VIÊN PHẢN BIỆN 2

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

TP. Hồ Chí Minh, ngày ... tháng 12 năm 2025

Giáo viên phản biện 2

(Ký ghi rõ họ tên)

MỤC LỤC

A MULTI-AGENT SYSTEM FOR MOVIE RECOMMENDATION	i
XÂY DỰNG HỆ THỐNG GỢI Ý PHIM SỬ DỤNG HỆ THỐNG ĐA TÁC	
NHÂN	ii
LỜI CẢM ƠN	iii
NHẬN XÉT VÀ ĐÁNH GIÁ CỦA GIÁO VIÊN HƯỚNG DẪN	iv
NHẬN XÉT VÀ ĐÁNH GIÁ CỦA GIÁO VIÊN PHẢN BIỆN 1	v
MỤC LỤC	vii
DANH MỤC HÌNH ẢNH	xi
DANH MỤC BẢNG BIỂU	xii
DANH MỤC TỪ VIẾT TẮT	xiii
CHƯƠNG 1. GIỚI THIỆU.....	1
1.1. Tổng quan.....	1
1.1.1. Bối cảnh	1
1.1.2. Lý do chọn đề tài.....	2
1.2. Mục tiêu nghiên cứu	3
1.3. Phạm vi nghiên cứu	4
1.4. Ý nghĩa khoa học và thực tiễn	5
1.4.1. Ý nghĩa khoa học.....	5
1.4.2. Ý nghĩa thực tiễn.....	6
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	7
2.1. Hệ thống gợi ý (Recommendation Systems)	7
2.1.1. Collaborative Filtering (CF)	7
2.1.2. Matrix Factorization (MF).....	9
2.1.3. Sequential Recommendation	11
2.2. Multi-Agent System	12
2.2.1. Khái niệm.....	12
2.2.2. Kiến trúc	12
2.2.3. Cơ chế communication trong Multi-Agent System	14
2.3. Retrieval-Augmented Generation (RAG)	15
2.3.1. Khái niệm.....	15

2.3.2. Thành phần của RAG	15
2.3.3. RAG trong Text-to-SQL	16
2.3.4. RAG trong Recommendation Systems	17
2.4. Thuật toán và kỹ thuật áp dụng trong Multi-Agent System	18
2.4.1. Thuật toán áp dụng trong Multi-Agent System.....	18
2.4.2. Kỹ thuật áp dụng trong Multi-Agent System	24
2.5. API trong Multi-Agent Systems.....	27
2.5.1. Khái niệm.....	27
2.5.2. Vai trò của API trong MAS	27
2.5.3. Các thiết kế API trong MAS hiện nay	28
2.5.4. Xu hướng thiết kế API trong MAS	28
2.6. Mối liên hệ giữa Multi-Agent System và hệ thống khuyến nghị	28
2.6.1. Khả năng giải thích.....	28
2.6.2. Xử lý tác vụ phức tạp (Complex Reasoning).....	29
2.6.3. Giải quyết vấn đề Cold-start và dữ liệu tĩnh	29
2.6.4. Cơ chế phản hồi và tự sửa lỗi (Reflexion/Self-correction)	30
CHƯƠNG 3: DỮ LIỆU VÀ MÔ HÌNH	31
3.1. Dữ liệu	31
3.1.1. Tổng quan về nguồn dữ liệu	31
3.1.2. Cấu trúc dữ liệu hệ thống	31
3.1.2. Thống kê và Đặc điểm dữ liệu	33
3.1.3. Tiền xử lý dữ liệu	34
3.2. Kiến trúc hệ thống.....	34
3.2.1. Tổng quan kiến trúc	34
3.2.2. Kiến trúc luồng xử lý của Multi-Agent.....	36
3.2.3. Sơ đồ kiến trúc xử lý truy vấn đa tác tử cho Recommendation System	40
3.2.2. Các tác nhân hệ thống	42
3.2.3. Các tác vụ (Tools)	45
3.2.6. Luồng xử lý.....	47
CHƯƠNG 4: THỰC NGHIỆM VÀ KẾT QUẢ	53

4.1. Thiết lập thực nghiệm	53
4.1.1. Môi trường thực nghiệm	53
4.1.2. Các hệ thống đánh giá	53
4.1.3. Metrics đánh giá	54
4.2. Huấn luyện mô hình	54
4.2.1. Matrix Factorization	54
4.2.2. Collaborative Filtering	56
4.2.3. SASRec	58
4.3. Cấu hình thực nghiệm	61
4.3.1. API	61
4.3.2. Prompt	61
4.4. Kết quả	63
4.4.1. Rating Prediction	63
4.4.2. Sequential Recommendation	64
4.4.3. Conversational Recommendation	64
4.4.4. Giao diện hoạt động	66
4.5. Phân tích	68
4.5.1. Ưu điểm của Multi-Agent system	68
4.5.2. Nhược điểm và thách thức	70
4.4.3. Trade-offs	71
4.5. Case Studies	72
4.5.1. Case Study 1: Người dùng khởi đầu (Cold-start User)	72
4.5.2. Case Study 2: Yêu cầu gợi ý với ràng buộc phức tạp	74
4.5.3. Case Study 3: Yêu cầu giải thích gợi ý (explanation request)	76
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	78
5.1. Kết luận	78
5.1.1. Tổng kết công việc đã thực hiện	78
5.1.2. Đóng góp	79
5.1.3. Kết quả đạt được	79
5.2. Hạn chế	81
5.2.1. Hạn chế về hiệu suất	81

5.2.2. Hạn chế về dữ liệu.....	81
5.2.3. Hạn chế về kỹ thuật	81
5.2.2. Hướng phát triển	82
5.5. Lời kết	84
TÀI LIỆU THAM KHẢO.....	84
KẾ HOẠCH THỰC HIỆN	87
NHẬT KÝ LÀM VIỆC	88

DANH MỤC HÌNH ẢNH

Hình 1: Kiến trúc của một Multi-Agent System	12
Hình 2: Kiến trúc hệ thống của đề tài	34
Hình 3: Kiến trúc luồng xử lý đa tác tử trong hệ thống Recommendation	39
Hình 4: Sơ đồ kiến trúc xử lý truy vấn đa tác tử cho Recommendation System sử dụng mô hình tuần tự SASRec	42
Hình 5: Rating Prediction Multi-Agent Workflow	49
Hình 6: Conversational Search Flow của hệ thống	50
Hình 7: Trang chủ	66
Hình 8: Giao diện khi người dùng nhập vào yêu cầu đã xem phim A và gợi ý phim B	67
Hình 9: Giao diện khi hệ thống gợi ý 5 phim khác nhau	67
Hình 10: Giao diện hệ thống gợi ý	68
Hình 11: Case Study người dùng khởi đầu	72
Hình 12: Case Study yêu cầu gợi ý với ràng buộc phức tạp	74
Hình 13: Case Study yêu cầu giải thích gợi ý	76

DANH MỤC BẢNG BIỂU

Bảng 1: Các thiết kế API trong MAS hiện nay	28
Bảng 2: Bảng cấu trúc dữ liệu tương tác	31
Bảng 3: Bảng cấu trúc dữ liệu người dùng	32
Bảng 4: Bảng cấu trúc dữ liệu phim	32
Bảng 5: Data Retrieval Tools của hệ thống	37
Bảng 6: Tham số huấn luyện mô hình Matrix Factorization	55
Bảng 7: Các tham số của mô hình Collaborative Filtering	57
Bảng 8: Tham số huấn luyện mô hình SASRec	59
Bảng 9: Cấu hình API	61
Bảng 10: Kết quả Rating Prediction	63
Bảng 11: Bảng kết quả Sequential Recommendation	64
Bảng 12: Bảng phân tích hệ thống cho case study người dùng khởi đầu	73
Bảng 13: Bảng phân tích hệ thống cộng tác đa tác tử cho yêu cầu gợi ý với ràng buộc phức tạp	75
Bảng 14: Phân tích hiệu suất hệ thống cộng tác đa tác tử yêu cầu giải thích gợi ý ...	76

DANH MỤC TỪ VIẾT TẮT

TỪ VIẾT TẮT	TỪ ĐẦY ĐỦ	Ý NGHĨA
LLM	Large Language Model	Mô hình ngôn ngữ lớn.
API	Application Programming Interface	Giao tiếp giữa các ứng dụng.
OS	Operating System	Hệ điều hành.
MAE	Mean Absolute Error	Sai số tuyệt đối TB.
RMSE	Root Mean Squared Error	Sai số bình phương căn bậc hai TB.
MF	Matrix Factorization	Phân rã ma trận (gợi ý).
CF	Collaborative Filtering	Lọc cộng tác (gợi ý).
MAS	Multi-Agent System	Hệ thống đa tác tử.

CHƯƠNG 1. GIỚI THIỆU

1.1. Tổng quan

1.1.1. Bối cảnh

Cùng với sự bùng nổ của Internet và các nền tảng số, lượng nội dung được tạo ra và truyền tải đến người dùng ngày càng tăng nhanh. Tuy nhiên, sự phong phú này đi kèm với vấn đề quá tải thông tin (information overload), khiến người dùng khó xác định nội dung phù hợp với nhu cầu cá nhân.

Hệ thống gợi ý (Recommender Systems – RS) trở thành giải pháp hiệu quả nhằm cá nhân hóa nội dung và tối ưu trải nghiệm người dùng. Nhiều nghiên cứu và hệ thống thực tế đã chứng minh vai trò trung tâm của RS trong các nền tảng lớn như: Netflix (gợi ý phim), Spotify (gợi ý âm nhạc), Amazon (gợi ý sản phẩm thương mại điện tử), YouTube/TikTok (gợi ý video dựa trên hành vi thời gian thực).

Các mô hình gợi ý truyền thống được sử dụng rộng rãi gồm:

- Collaborative Filtering (CF)
- Content-Based Filtering (CBF)
- Matrix Factorization (MF)
- Factorization Machines (FM, xDeepFM)
- Sequential Models như GRU4Rec, SASRec

Tuy nhiên, những phương pháp cổ điển này vẫn còn hạn chế trong bối cảnh yêu cầu mới:

- Không hỗ trợ đối thoại và truy vấn ngôn ngữ tự nhiên.
- Khó mở rộng cho các biểu diễn ngữ nghĩa phức tạp.
- Giải thích cho gợi ý còn hạn chế, thiếu minh bạch.
- Không linh hoạt khi yêu cầu kết hợp nhiều nguồn thông tin.

Sự xuất hiện của Large Language Models (LLMs) như GPT-4, Gemini và Llama-3 mở ra hướng tiếp cận mới cho hệ thống gợi ý. Nhiều nghiên cứu gần đây cho thấy LLMs có khả năng:

- Hiểu truy vấn ngôn ngữ tự nhiên phức tạp.
- Tương tác hội thoại nhiều lượt.
- Kết hợp các nguồn tri thức khác nhau.
- Giải thích rõ ràng quyết định gợi ý.
- Suy luận đa bước như con người (chain-of-thought reasoning).

Bên cạnh đó, phong trào nghiên cứu Multi-Agent Systems (MAS) – nơi nhiều tác nhân (agents) phối hợp để giải quyết vấn đề – đang trở thành xu hướng trong năm 2023–2025. Hệ thống đa tác nhân cho phép LLMs kết hợp khả năng của nhau, chia nhiệm vụ thành từng vai trò chuyên biệt, và tối ưu hiệu quả gợi ý.

Nhiều nghiên cứu như CAMEL, AutoGen, ReAct, Reflexion, LLM Agents for Reasoning đã chứng minh rằng: *“Hệ thống nhiều tác nhân có thể vượt trội so với một mô hình LLM đơn lẻ, đặc biệt trong các bài toán yêu cầu suy luận phân tích, hợp tác hoặc xử lý nhiều thông tin dị thể.”*

Do đó, việc kết hợp LLM + Multi-Agent Architecture + Recommender Models trở thành một hướng nghiên cứu rất mới, nhiều tiềm năng và phù hợp với bối cảnh hiện tại.

1.1.2. Lý do chọn đề tài

Việc lựa chọn đề tài xuất phát từ ba nhóm cơ sở quan trọng, bao gồm xu hướng nghiên cứu hiện đại, hạn chế của các hệ thống gợi ý truyền thống, và tiềm năng nổi bật của kiến trúc đa tác nhân trong việc nâng cao chất lượng gợi ý.

Thứ nhất, về xu hướng khoa học và công nghệ, các hội nghị hàng đầu trong lĩnh vực trí tuệ nhân tạo như NeurIPS, ICLR, KDD và RecSys đều ghi nhận sự phát triển mạnh mẽ của các hướng tiếp cận mới, đặc biệt là *LLM-enhanced Recommendation*, *Conversational Recommendation*, *Multi-Agent Reasoning* và *Explainable Recommendation*. Các nghiên cứu gần đây khẳng định rằng các mô hình ngôn ngữ lớn (LLMs) có khả năng hiểu và mô hình hóa hành vi người dùng theo cách tự nhiên, linh hoạt và giàu ngữ nghĩa hơn so với các phương pháp truyền thống, mở ra tiềm năng cải thiện sâu sắc trải nghiệm tương tác trong hệ thống gợi ý.

Thứ hai, hệ thống gợi ý truyền thống vẫn còn tồn tại nhiều nhược điểm, nhất là trong bối cảnh nhu cầu người dùng ngày càng phức tạp. Các mô hình cổ điển thường chỉ dựa vào dữ liệu cấu trúc, thiếu khả năng tương tác trực tiếp, không thể đặt câu hỏi làm rõ khi thông tin chưa đầy đủ, và gần như không cung cấp lời giải thích cho gợi ý. Bên cạnh đó, việc kết hợp nhiều chiến lược gợi ý trong cùng một hệ thống (hybrid reasoning) còn gặp nhiều hạn chế, khiến kết quả gợi ý đôi khi kém linh hoạt và thiếu cá nhân hóa.

Thứ ba, kiến trúc đa tác nhân (Multi-Agent) mang lại nhiều lợi ích thiết thực, đáp ứng tốt các yêu cầu mới của hệ thống gợi ý hiện đại. Cách tiếp cận này cho phép phân tách nhiệm vụ rõ ràng, giúp từng tác nhân chuyên môn hóa vào một vai trò cụ thể, đồng thời phối hợp với nhau để suy luận và ra quyết định hiệu quả hơn. Multi-agent còn hỗ trợ cơ chế tự giám sát và điều chỉnh sai sót (self-reflection), giúp hệ thống thích ứng tốt hơn với bối cảnh và nhu cầu thực tế. Trong lĩnh vực Recommendation, kiến trúc này có thể mô phỏng nhiều vai trò chuyên biệt như tác nhân phân tích hành vi, tác nhân tìm kiếm phim, tác nhân sinh gợi ý từ mô hình truyền thống, tác nhân đánh giá chất lượng, hoặc tác nhân giải thích lý do gợi ý cho người dùng.

Từ những cơ sở trên, đề tài lựa chọn hướng tiếp cận hệ thống gợi ý dựa trên kiến trúc đa tác nhân kết hợp mô hình ngôn ngữ lớn, nhằm khai thác tối đa ưu điểm của cả hai phương pháp, đồng thời khắc phục những hạn chế tồn tại của các hệ thống gợi ý truyền thống.

1.2. Mục tiêu nghiên cứu

Mục tiêu chính của đề tài là xây dựng và đánh giá một hệ thống gợi ý phim dựa trên kiến trúc đa tác nhân (Multi-Agent Architecture) kết hợp với mô hình ngôn ngữ lớn (LLM). Để đạt được mục tiêu tổng thể này, nghiên cứu tập trung vào bốn nhóm mục tiêu cụ thể như sau:

Thứ nhất, đề tài xây dựng một kiến trúc Multi-Agent System gồm các tác nhân có chức năng chuyên biệt, đảm nhiệm những vai trò khác nhau trong quá trình xử lý và tạo gợi ý gồm:

- *Manager Agent* điều phối luồng công việc và quản lý quá trình tương tác;
- *Analyst Agent* phân tích dữ liệu người dùng và thông tin về sản phẩm;
- *Searcher Agent* tra cứu và thu thập thông tin từ các nguồn dữ liệu khác nhau;
- *Reflector Agent* đánh giá, kiểm tra và điều chỉnh kết quả gợi ý;
- *Interpreter Agent* diễn giải và trình bày kết quả tự nhiên đến người dùng.

Việc phân rã nhiệm vụ theo từng tác nhân giúp tối ưu hóa quá trình xử lý và tăng khả năng mở rộng của hệ thống.

Thứ hai, nghiên cứu kết hợp các kỹ thuật gợi ý truyền thống như Collaborative Filtering, Matrix Factorization và mô hình tuần tự SASRec với khả năng suy luận ngữ nghĩa của LLMs, hình thành một hệ thống gợi ý lai (Hybrid Recommendation System) linh hoạt và hiệu quả. Các công cụ hỗ trợ (tools) được xây dựng và gắn vào từng tác nhân để tăng cường khả năng xử lý, tìm kiếm và suy luận.

Thứ ba, hệ thống gợi ý hướng đến việc xử lý đồng thời nhiều loại nhiệm vụ quan trọng trong lĩnh vực Recommendation, bao gồm dự đoán điểm đánh giá (Rating Prediction), gợi ý tuần tự sản phẩm tiếp theo (Sequential Recommendation), gợi ý thông qua hội thoại (Conversational Recommendation), và sinh giải thích cho gợi ý (Explanation Generation). Điều này giúp hệ thống đáp ứng đa dạng nhu cầu của người dùng trong thực tế.

Thứ tư, đánh giá và so sánh hiệu quả hệ thống. đánh giá toàn diện các mô hình gợi ý được xây dựng, so sánh với các phương pháp baseline truyền thống nhằm xác định mức độ cải thiện về độ chính xác, sự linh hoạt, khả năng giải thích, cũng như khả năng tương tác. Nghiên cứu, phân tích ưu điểm và hạn chế của từng phương pháp để rút ra những nhận định cho hướng phát triển trong tương lai.

1.3. Phạm vi nghiên cứu

Phạm vi nghiên cứu của đề tài được xác định dựa trên đặc trưng dữ liệu, công nghệ sử dụng và các chức năng trọng tâm cần triển khai. Đồng thời, đề tài cũng chỉ rõ các giới hạn nhằm đảm bảo tính khả thi trong khuôn khổ thời gian và nguồn lực.

Về phạm vi dữ liệu, sử dụng bộ dữ liệu MovieLens 100K (ml-100k) làm nguồn dữ liệu chính. Bộ dữ liệu này bao gồm 100.000 đánh giá từ 943 người dùng đối với 1.682 bộ phim, kèm theo thông tin hỗ trợ như tuổi, giới tính, nghề nghiệp của người dùng; tiêu đề, thể loại và năm phát hành của phim.

Về phạm vi công nghệ, đề tài sử dụng Python 3.10 trở lên làm ngôn ngữ lập trình chính. Các thư viện và framework trọng tâm gồm:

- PyTorch xây dựng và huấn luyện các mô hình học máy truyền thống;
- LangChain tích hợp và điều phối các mô hình ngôn ngữ lớn hệ thống;

- Streamlit để xây dựng giao diện web phục vụ việc thử nghiệm và minh họa.

Các mô hình ngôn ngữ lớn được sử dụng bao gồm OpenAI GPT (GPT-3.5 và GPT-4), Google Gemini, cùng mô hình mã nguồn mở.

Về phạm vi chức năng, hệ thống tập trung vào ba nhóm nhiệm vụ chính trong lĩnh vực gợi ý, bao gồm dự đoán điểm đánh giá (Rating Prediction), gợi ý tuần tự (Sequential Recommendation), và gợi ý dựa trên hội thoại (Chat-based Recommendation). Bên cạnh đó, nghiên cứu triển khai và đánh giá nhiều cơ chế điều phối tác nhân như ReAct, Reflection, Collaboration và Hybrid. Giao diện minh họa được phát triển dưới dạng ứng dụng web thông qua Streamlit.

Về giới hạn nghiên cứu, đề tài chỉ tập trung trong phạm vi miền dữ liệu phim ảnh và chưa hướng đến việc triển khai một hệ thống ở quy mô sản xuất thực tế (production-scale). Các yếu tố như tối ưu chi phí sử dụng API LLMs, xử lý dữ liệu streaming thời gian thực, hay tối ưu hóa hạ tầng hệ thống chưa nằm trong phạm vi thực hiện của nghiên cứu này. Những nội dung này được đề xuất cho các hướng phát triển trong tương lai.

1.4. Ý nghĩa khoa học và thực tiễn

1.4.1. Ý nghĩa khoa học

Đề tài đóng góp vào lĩnh vực hệ thống gợi ý và trí tuệ nhân tạo thông qua kết hợp kiến trúc đa tác nhân (Multi-Agent Architecture) với mô hình ngôn ngữ lớn (LLM) — một hướng tiếp cận hiện đang được quan tâm mạnh trong các công trình nghiên cứu quốc tế. Các ý nghĩa khoa học nổi bật:

- Đề xuất mô hình kiến trúc đa tác nhân cho hệ thống gợi ý: Việc thiết kế các tác nhân chuyên biệt (phân tích, tìm kiếm, phản tư, giải thích...) làm rõ vai trò và cơ chế phối hợp của tác nhân trong bối cảnh Recommendation, một chủ đề còn mới và chưa được nghiên cứu sâu.
- Kết hợp phương pháp truyền thống và LLMs: khả năng tích hợp giữa các thuật toán Recommendation cổ điển (CF, MF, SASRec) với sức mạnh suy luận ngữ nghĩa của LLMs. Đây là xu hướng phát triển Hybrid-Learning và LLM-enhanced Recommendation, hướng đến những hệ gợi ý thông minh và linh hoạt hơn.
- Xây dựng cơ chế suy luận ReAct – Reflection – Collaboration trong Recommendation: Việc áp dụng các chiến lược điều phối tác nhân vào bài

toán gợi ý góp phần mở rộng phương pháp luận của Multi-Agent Reasoning sang lĩnh vực hệ thống gợi ý.

- Đóng góp bộ công cụ thực nghiệm và quy trình đánh giá: đánh giá rõ ràng cho các mô hình Recommendation dựa trên LLMs, hỗ trợ các nghiên cứu tiếp theo tham khảo và mở rộng.

Đề tài góp phần củng cố nền tảng lý thuyết về Multi-Agent Systems trong Recommendation và mở ra các hướng phát triển mới trong nghiên cứu ứng dụng LLM vào hệ gợi ý.

1.4.2. Ý nghĩa thực tiễn

Bên cạnh các đóng góp học thuật, đề tài mang lại nhiều giá trị thực tiễn trong việc xây dựng các hệ thống gợi ý hiện đại:

- Nâng cao trải nghiệm người dùng: Hệ thống hỗ trợ tương tác tự nhiên, có khả năng đặt câu hỏi làm rõ sở thích và đưa ra gợi ý phù hợp hơn. Điều này đặc biệt hữu ích trong các nền tảng xem phim, thương mại điện tử hoặc dịch vụ nội dung số.
- Tăng tính minh bạch trong gợi ý: Nhờ tác nhân Interpreter, người dùng có thể nhận được lời giải thích rõ ràng về lý do một bộ phim được đề xuất, giúp tăng niềm tin và mức độ hài lòng với hệ thống.
- Khả năng mở rộng và tùy chỉnh cao: Kiến trúc đa tác nhân cho phép doanh nghiệp dễ dàng bổ sung, thay thế hoặc nâng cấp các thành phần mà không ảnh hưởng đến toàn hệ thống. Điều này phù hợp với các môi trường cần thay đổi nhanh như sản phẩm công nghệ.
- Ứng dụng vào nhiều lĩnh vực: Mặc dù đề tài tập trung vào miền dữ liệu phim, kiến trúc có thể mở rộng sang các lĩnh vực khác như thương mại điện tử, âm nhạc, giáo dục, du lịch hoặc gợi ý sản phẩm trong siêu ứng dụng.
- Làm cơ sở cho triển khai hệ thống hội thoại thông minh: Hệ thống gợi ý dạng chat (Conversational Recommendation) có thể được tích hợp trực tiếp vào chatbot chăm sóc khách hàng, trợ lý ảo hoặc nền tảng OTT.

Đề tài không chỉ có giá trị về mặt nghiên cứu mà còn mang tính ứng dụng cao, góp phần định hướng cho việc phát triển các hệ thống gợi ý thông minh thế hệ mới trong bối cảnh LLMs và Multi-Agent Systems đang trở thành nền tảng của nhiều ứng dụng AI hiện đại.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Hệ thống gợi ý (Recommendation Systems)

Hệ thống gợi ý (Recommendation System) là thành phần quan trọng trong nhiều nền tảng thương mại điện tử, mạng xã hội và dịch vụ số, giúp cá nhân hóa trải nghiệm người dùng và tăng khả năng tương tác. Mục tiêu là dự đoán sở thích hoặc nhu cầu tương lai của người dùng dựa trên dữ liệu lịch sử, từ đó đề xuất các mục nội dung (item) phù hợp như phim, bài hát, sản phẩm, bài viết,...

Hệ thống gợi ý hiện đại thường được xây dựng dựa trên ba hướng tiếp cận phổ biến: Collaborative Filtering, Matrix Factorization và Sequential Recommendation.

2.1.1. Collaborative Filtering (CF)

Collaborative Filtering (CF) là một trong những phương pháp nền tảng và được sử dụng rộng rãi nhất trong các hệ thống gợi ý hiện đại. Phương pháp này dựa trên giả định quan trọng: những người dùng có hành vi tương tự trong quá khứ sẽ có xu hướng đưa ra lựa chọn giống nhau trong tương lai. Thay vì yêu cầu thông tin về nội dung sản phẩm hoặc hồ sơ người dùng, CF khai thác trực tiếp ma trận tương tác (interaction matrix) giữa người dùng và các đối tượng (item) như đánh giá (rating), lượt xem (view), lượt mua (purchase), hoặc lượt nhấp (click).

Nguyên lý hoạt động

CF vận hành dựa trên hai ý tưởng chính:

- Người dùng tương tự có sở thích tương tự (User Similarity). Nếu hai người dùng có lịch sử tương tác gần giống nhau, hệ thống có thể gợi ý các sản phẩm mà một người thích cho người còn lại.
- Các sản phẩm tương tự thường được lựa chọn bởi cùng nhóm người dùng (Item Similarity). Nếu hai sản phẩm thường được tiêu thụ cùng nhau, hệ thống có thể gợi ý sản phẩm còn lại khi người dùng đã tương tác với sản phẩm đầu tiên.

2.1.1.1. User-based Collaborative Filtering

Phương pháp này tập trung phân tích mối quan hệ giữa người dùng với nhau. Hệ thống sẽ:

- Tính độ tương đồng giữa người dùng dựa trên lịch sử tương tác (ví dụ: Cosine Similarity, Pearson Correlation).

- Xác định nhóm “người dùng lân cận” (nearest neighbors) của người dùng mục tiêu.
- Gợi ý các sản phẩm mà những người dùng tương tự đã tương tác nhưng người dùng mục tiêu chưa xem.

Ưu điểm: Dễ triển khai, trực quan trong cách diễn giải, tận dụng tốt hành vi cộng đồng.

Hạn chế: Không phù hợp với tập dữ liệu lớn do chi phí tính tương đồng giữa các cặp người dùng tăng theo quy mô. Dễ bị nhiễu khi hành vi của người dùng thay đổi liên tục.

2.1.1.2. Item-based Collaborative Filtering

Khác với User-based, phương pháp này tập trung tính toán mức độ tương đồng giữa các sản phẩm:

- Phân tích lịch sử tương tác để đo lường sự liên quan giữa các item.
- Xây dựng ma trận tương đồng item-item.
- Khi người dùng tương tác với một item, hệ thống sẽ gợi ý các item tương tự trong ma trận.

Ưu điểm: Ổn định hơn so với User-based vì đặc trưng item ít thay đổi theo thời gian. Có khả năng mở rộng tốt hơn và thích hợp với các hệ thống lớn (ví dụ: Amazon).

Hạn chế: Chỉ dựa trên đồng xuất hiện (co-occurrence), khó nắm bắt được mối quan hệ ngữ nghĩa hoặc nội dung sâu.

2.1.1.3. Ưu điểm và hạn chế

Ưu điểm

- Không yêu cầu dữ liệu nội dung (content-free).
- Có khả năng phát hiện các quan hệ phức tạp giữa người dùng và item.
- Dễ ứng dụng cho nhiều loại dữ liệu tương tác khác nhau.

Hạn chế

- Vấn đề cold-start: khó gợi ý khi có người dùng mới hoặc item mới chưa có lịch sử tương tác.
- Ma trận tương tác thưa (sparsity): lượng tương tác thực thường rất nhỏ so với tổng số khả năng.
- Khả năng mở rộng: khi dữ liệu lớn, việc tính toán tương đồng trở nên tốn kém.

2.1.1.4. Vai trò

Collaborative Filtering là nền tảng cho nhiều mô hình gợi ý nâng cao, bao gồm Matrix Factorization và các mô hình Deep Learning về sau. Dù gặp hạn chế trong các bài toán quy mô lớn và dữ liệu thưa, CF vẫn là phương pháp quan trọng giúp hình thành ảnh hưởng hành vi người dùng và cách thức mô hình hóa sự tương đồng trong hệ thống gợi ý.

2.1.2. Matrix Factorization (MF)

Matrix Factorization (MF) là một trong những phương pháp quan trọng và hiệu quả nhất trong hệ thống gợi ý hiện đại, đặc biệt sau khi được giới thiệu trong nghiên cứu của Simon Funk (2006) và sau đó được mở rộng mạnh mẽ trong công trình của Koren, Bell và Volinsky (2009). Các nghiên cứu này minh chứng rằng MF có khả năng mô hình hóa hiệu quả các mối quan hệ tiềm ẩn giữa người dùng và sản phẩm trong ma trận tương tác thưa, giúp cải thiện đáng kể chất lượng dự đoán so với Collaborative Filtering truyền thống.

2.1.2.1. Cơ sở lý thuyết

Theo Funk (2006), MF hoạt động dựa trên nguyên lý phân rã ma trận tương tác lớn R thành hai ma trận hạng thấp hơn mô tả đặc trưng tiềm ẩn (latent factors) của người dùng và sản phẩm. Ý tưởng này được khẳng định và hoàn thiện trong nghiên cứu kinh điển của Koren et al. (2009), cho rằng sở thích người dùng và thuộc tính của item có thể được biểu diễn trong không gian latent có kích thước thấp.

Cho ma trận tương tác: $R \in \mathbb{R}^{m \times n}$

MF phân rã thành: $R \approx PQ^T$

- $P_u \in \mathbb{R}^k$: vector latent của người dùng.
- $Q_i \in \mathbb{R}^k$: vector latent của sản phẩm.
- k : số chiều không gian ẩn.

Dự đoán tương tác: $\widehat{r_{u,i}} = P_u \cdot Q_i$

Nghiên cứu của Koren (2009) chỉ ra rằng cách biểu diễn này giúp mô hình nắm bắt được cấu trúc tiềm ẩn của dữ liệu, kể cả khi ma trận rất thưa.

2.1.2.2. Bài toán tối ưu

MF được huấn luyện bằng cách tối thiểu hóa sai số bình phương có

regularization để tránh overfitting:

$$\mathcal{L} = \sum_{(u,i) \in \mathcal{K}} (R_{u,i} - P_u Q_i^T)^2 + \lambda (||P_u||^2 + ||Q_i||^2)$$

Trong đó:

- κ : tập dữ liệu có tương tác
- λ : hệ số regularization

Hai phương pháp tối ưu chính phổ biến trong nghiên cứu:

- Stochastic Gradient Descent (SGD): Funk (2006) áp dụng SGD để cập nhật từng cặp (u,i) , cho phép mô hình hội tụ nhanh và dễ triển khai.
- Alternating Least Squares (ALS): Theo Zhou et al. (2008), ALS hoạt động hiệu quả trong môi trường dữ liệu lớn và tính toán phân tán (ví dụ: Spark MLlib), bằng cách lần lượt tối ưu P và Q .

2.1.2.3. Ưu điểm theo các nghiên cứu

Xử lý tốt dữ liệu thưa: Koren (2009) cho rằng MF đặc biệt phù hợp với tình huống rating matrix sparse đến 99%.

Khả năng tổng quát hóa tốt nhờ regularization.

Mở rộng linh hoạt: có thể kết hợp với temporal dynamics, biases, implicit feedback (theo nghiên cứu SVD++, Koren 2008).

Hiệu năng dự đoán vượt trội so với CF truyền thống, đặc biệt trên bộ dữ liệu Netflix Prize.

2.1.2.4. Hạn chế

Cold-start: MF không hoạt động tốt khi người dùng hoặc sản phẩm mới không có tương tác (Koren 2009).

Không mô hình hóa yếu tố thời gian hoặc thứ tự hành vi (He et al., 2018).

Diễn giải hạn chế: latent factors mang tính trừu tượng, khó hiểu rõ ý nghĩa (Su & Khoshgoftaar, 2009).

Những hạn chế này đã thúc đẩy sự phát triển của các phương pháp tiếp theo như Factorization Machines, Neural Collaborative Filtering và Sequential Recommendation (ví dụ: SASRec).

Matrix Factorization đóng vai trò nền tảng trong hệ thống gợi ý hiện đại. Các nghiên cứu kinh điển đã chứng minh MF không chỉ hiệu quả trong dự đoán mà còn

có khả năng mở rộng và tích hợp vào nhiều mô hình tiên tiến. Đây là bước tiến quan trọng từ Collaborative Filtering truyền thống sang các mô hình dựa trên không gian latent.

2.1.3. Sequential Recommendation

2.1.3.1. Khái niệm

Hệ thống gợi ý tuần tự (Sequential Recommendation) tập trung mô hình hóa chuỗi hành vi theo thời gian của người dùng để dự đoán món tiếp theo mà người dùng sẽ tương tác. Khác với Collaborative Filtering truyền thống – xem dữ liệu dưới dạng ma trận người dùng – sản phẩm tĩnh – các mô hình tuần tự nhấn mạnh rằng thứ tự tương tác mang ý nghĩa quan trọng trong dự đoán hành vi tương lai.

Trong nhiều bối cảnh như gợi ý phim, thương mại điện tử, âm nhạc, việc nắm bắt “context theo thời gian” giúp mô hình hiểu rõ hơn về thói quen ngắn hạn (short-term preference) và động lực lựa chọn của người dùng. Các nghiên cứu cho thấy mô hình tuần tự thường vượt trội hơn so với CF truyền thống, đặc biệt trong các tình huống như “xu hướng mới”, “hành vi thời điểm”, và “ngữ cảnh vừa thay đổi” (Hidasi et al., 2016; Kang & McAuley, 2018).

SASRec (Self-Attentive Sequential Recommendation) là mô hình do Kang và McAuley (2018) đề xuất, đánh dấu bước tiến quan trọng khi lần đầu tiên áp dụng cơ chế Self-Attention trong Transformer vào bài toán gợi ý tuần tự.

2.1.3.2. Cơ chế hoạt động chính

Embedding chuỗi tương tác: Mỗi sản phẩm (item) được ánh xạ thành vector embedding. Chuỗi lịch sử tương tác của người dùng được biểu diễn dạng:

$$S_u = [i_1, i_2, \dots, i_l]$$

Self-Attention Encoder: Mô hình sử dụng *Scaled Dot-Product Attention* để học mối quan hệ giữa mọi cặp item trong chuỗi:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d}}\right)V$$

Nhờ cơ chế này, SASRec tự động học được:

- Những item nào quan trọng trong lịch sử của người dùng
- Mức độ ảnh hưởng của item trước đối với item sau

- Các pattern dài hạn và ngắn hạn trong hành vi người dùng

SASRec sử dụng masking nhằm đảm bảo mô hình chỉ sử dụng thông tin quá khứ để dự đoán item kế tiếp.

Final Prediction: Vector biểu diễn vị trí cuối cùng của chuỗi được sử dụng để tính xác suất người dùng sẽ tương tác với từng item trong tập ứng viên.

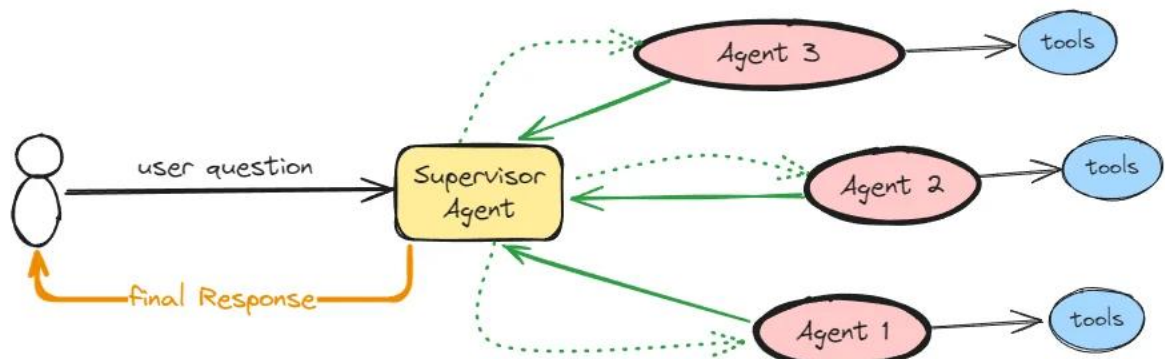
2.2. Multi-Agent System

2.2.1. Khái niệm

Multi Agent System (MAS) là một mô hình trong đó nhiều AI Agent cùng hoạt động, tương tác và phối hợp với nhau để thực hiện nhiệm vụ hoặc cùng giải quyết một vấn đề nào đó. Mỗi Agent là một thực thể có thể tự đưa ra quyết định và hành động mà không cần sự điều khiển trực tiếp của con người.

Ví dụ, trong một hệ thống xử lý ngôn ngữ tự nhiên, Multi Agent sẽ đảm nhiệm các nhiệm vụ khác nhau như tóm tắt, dịch thuật, hay tạo nội dung. Các agent này sẽ phối hợp với nhau để tạo ra một quy trình làm việc linh hoạt và tối ưu.

2.2.2. Kiến trúc



Hình 1: Kiến trúc của một Multi-Agent System

Một Multi-Agent gồm bốn thành phần chính: User, Supervisor Agent, một tập các Worker Agents (Agent 1..N), và các Tools (dịch vụ bên ngoài, APIs, cơ sở dữ liệu, v.v.). Người dùng gửi một yêu cầu (user query) đến Supervisor; Supervisor phân tích yêu cầu, phân phối công việc đến các Worker Agents, mỗi Worker có thể gọi một hoặc nhiều Tools để thực hiện nhiệm vụ con, rồi trả về kết quả cho Supervisor. Supervisor hợp nhất các kết quả, đánh giá độ tin cậy, và phản hồi cho người dùng.

2.2.2.1. Vai trò và trách nhiệm

User

- Phát sinh yêu cầu (natural language question, task description, constraint).
- Nhận phản hồi cuối cùng (final response).

Supervisor Agent

- Tiền xử lý yêu cầu: phân tách intent, xác định các subtask, chuẩn hóa/kiểm tra đầu vào.
- Phân phối công việc: chọn agent phù hợp, quyết định số lượng agents tham gia, và chỉ ra tool (nếu cần).
- Giám sát & hợp nhất: thu thập phản hồi từ workers, thực hiện aggregation (ví dụ: majority voting, confidence-weighted fusion, hoặc ensemble reasoning).
- Kiểm tra chất lượng: chạy các kiểm tra plausibility, consistency, hoặc thêm bước verification (self-critique).
- Quản lý luồng: retry, fallback, timeout, và logging.

Worker Agents

- Mỗi worker là một chuyên gia (expert) hoặc một mô-đun chức năng (retriever, generator, verifier, calculator, planner).
- Tương tác với Tools để thực thi subtask (ví dụ: truy vấn cơ sở dữ liệu, chạy mô hình chuyên biệt, gọi API bên ngoài).
- Báo cáo: trả về nội dung kết quả, điểm tin cậy (confidence), và metadata (latency, traces) đến Supervisor Agent.

Tools

- Dịch vụ bên ngoài hoặc nội bộ (search engine, knowledge base, calculator, code execution environment, LLMs chuyên dụng).
- Tính bất đồng bộ/ngoại vi: Supervisor quản lý truy vấn tới các Tools qua Workers.

2.2.2.2. Luồng dữ liệu và kiểu tương tác

User → Supervisor: một câu hỏi tự nhiên (text), có thể kèm ngữ cảnh.

Supervisor (analysis): parse intent → tách subtask → lập kế hoạch (plan).

Supervisor → Workers: gửi subtask + constraints:

- Luồng chính: chỉ huy/hợp lệ hoá các lệnh gửi cho các sub-agent.
- Luồng phản hồi hoặc gợi ý (feedback, critique, or monitoring channel).

Worker → Tools: worker gọi tool để xử lý subtask.

Worker → Supervisor: trả kết quả, kèm confidence/metadata.

Supervisor (aggregation & verification): tổng hợp, đối chiếu kết quả, có thể gọi thêm workers/verifier hoặc tools để kiểm chứng.

Supervisor → User: trả về final response; có thể kèm giải thích (rationale) hoặc trace của các nguồn.

2.2.3. Cơ chế communication trong Multi-Agent System

Communication (giao tiếp) đóng vai trò trung tâm, đảm bảo các agent có thể phối hợp, chia sẻ tri thức và tổ chức hành động nhằm đạt được mục tiêu chung hoặc mục tiêu riêng. Không có cơ chế giao tiếp hiệu quả, một hệ MAS sẽ bị phân mảnh, khó đạt được sự thống nhất hành vi.

Communication trong MAS không chỉ dừng lại ở việc trao đổi thông điệp đơn thuần, mà còn phản ánh mức độ hợp tác, chiến lược truy tìm thông tin, cơ chế đàm phán, phân chia nhiệm vụ, lập kế hoạch và ra quyết định phân tán.

Các cơ chế giao tiếp (communication mechanisms) là những phương thức cho phép thiết lập và duy trì sự trao đổi thông tin hiệu quả giữa các tác tử trong hệ thống đa tác tử. Các cơ chế này bao gồm các kênh truyền tải vật lý và kênh truyền tải ảo dùng để truyền thông điệp giữa các tác tử, đồng thời tích hợp các thuật toán và kỹ thuật phục vụ cho việc mã hóa và giải mã thông tin. Nhìn chung, cơ chế giao tiếp trong MAS có thể được chia thành hai loại chính: cơ chế tập trung và cơ chế phi tập trung.

Cơ chế tập trung (Centralized Mechanisms): dựa trên một hoặc một số tác tử trung tâm đóng vai trò điều phối hoặc môi giới cho quá trình giao tiếp. Mọi thông điệp đều được gửi đến các tác tử điều phối trước; sau đó chúng mới tiếp tục được chuyển đến các tác tử đích. Các hệ thống dạng này thường áp dụng mô hình publish–subscribe, trong đó một kho dữ liệu trung tâm lưu trữ thông tin và đóng vai trò hỗ trợ trao đổi thông điệp giữa các tác tử.

Cơ chế phi tập trung (Decentralized Mechanisms): cho phép các tác tử giao tiếp trực tiếp với nhau mà không cần bất kỳ thực thể trung tâm nào. Phân bổ nhiệm vụ đồng đều giữa các tác tử và tuân theo mô hình giao tiếp ngang hàng (peer-to-peer).

So với cơ chế tập trung, cơ chế phi tập trung có khả năng mở rộng tốt hơn, đồng thời thể hiện mức độ bền vững và khả năng chịu lỗi cao hơn nhờ không tồn tại điểm lỗi đơn (single point of failure).

2.3. Retrieval-Augmented Generation (RAG)

2.3.1. Khái niệm

RAG (Retrieval-Augmented Generation) là một phương pháp kết hợp giữa khả năng truy xuất thông tin từ các nguồn dữ liệu bên ngoài (Retrieval) với sức mạnh tạo nội dung của mô hình ngôn ngữ lớn (LLM - Large Language Model)

RAG đang trở thành một bước tiến quan trọng trong việc phát triển các ứng dụng AI thông minh và đáng tin cậy. Thay vì dựa hoàn toàn vào trí nhớ của các mô hình ngôn ngữ lớn (LLM), RAG kết hợp khả năng truy xuất thông tin từ các nguồn dữ liệu bên ngoài với sức mạnh suy luận của LLM.

2.3.2. Thành phần của RAG

Nguồn tri thức bên ngoài (External Knowledge Source): kho lưu trữ thông tin nền tảng, bao gồm các tài liệu chuyên ngành, API, hoặc cơ sở dữ liệu có chứa kiến thức đặc thù lĩnh vực (domain-specific) hoặc kiến thức tổng quát mà mô hình cần tham chiếu.

Phân đoạn và tiền xử lý văn bản (Text Chunking and Preprocessing): chuẩn bị dữ liệu, các tài liệu lớn được chia nhỏ thành các đoạn (chunks) có kích thước phù hợp và trải qua bước làm sạch để đảm bảo tính nhất quán (ví dụ: loại bỏ ký tự không cần thiết, chuẩn hóa định dạng).

Mô hình nhúng (Embedding Model): chuyển đổi các đoạn văn bản đã được tiền xử lý thành các vector số học. Các vector này mã hóa ý nghĩa ngữ nghĩa (semantic meaning) của văn bản, hiểu được mức độ tương đồng giữa các khái niệm.

Cơ sở dữ liệu vector (Vector Database): lưu trữ tập hợp các vector nhúng đã tạo ra. Được tối ưu hóa để thực hiện tìm kiếm độ tương đồng (similarity search) nhanh chóng, giúp truy xuất thông tin liên quan một cách hiệu quả.

Bộ mã hóa truy vấn (Query Encoder): Khi người dùng đưa ra một truy vấn (query), bộ mã hóa sẽ biến đổi truy vấn đó thành một vector tương đương. Vector truy vấn này sau đó được sử dụng để so sánh với các vector đã lưu trữ trong cơ sở dữ

liệu vector.

Bộ truy xuất (Retriever): tìm kiếm và trả về các đoạn văn bản (chunks) phù hợp nhất từ cơ sở dữ liệu vector.

Lớp tăng cường ngữ cảnh (Prompt Augmentation Layer): kết hợp các đoạn văn bản đã truy xuất với truy vấn ban đầu của người dùng. Tạo ra một lời nhắc (prompt) giàu ngữ cảnh, cung cấp cho LLM (Generator) tạo ra phản hồi.

LLM (Generator): tạo ra câu trả lời cuối cùng. Sử dụng prompt đã được tăng cường ngữ cảnh, LLM sẽ tạo ra một phản hồi có căn cứ (grounded response), đảm bảo tính chính xác và liên quan dữ liệu đã cung cấp.

Bộ cập nhật (Updater): hệ thống luôn được trang bị tri thức mới nhất. Nó định kỳ thực hiện làm mới và nhúng lại (re-embed) dữ liệu mới hoặc đã sửa đổi vào Cơ sở Dữ liệu Vector, giúp cơ sở tri thức luôn cập nhật.

2.3.3. RAG trong Text-to-SQL

RAG cải thiện đáng kể độ chính xác và tính thực thi của các hệ thống Text-to-SQL. Bằng cách trang bị cho các Mô hình Ngôn ngữ Lớn (LLM) ngữ cảnh liên quan, RAG cho phép mô hình tạo ra các truy vấn SQL chính xác và có thể thực thi được từ các câu hỏi bằng ngôn ngữ tự nhiên của người dùng.

Quy trình ứng dụng RAG trong hệ thống Text-to-SQL:

User Query: Người dùng nhập một câu hỏi bằng ngôn ngữ tự nhiên (ví dụ: “Hiển thị 5 khách hàng hàng đầu đã chi tiêu nhiều nhất trong tháng trước”). Đây là đầu vào khởi tạo toàn bộ quy trình.

Retrieval: Thành phần truy xuất của hệ thống RAG tìm kiếm trong một cơ sở tri thức (knowledge base) để thu thập thông tin liên quan đến truy vấn của người dùng và lược đồ cơ sở dữ liệu (database schema) mục tiêu. Cơ sở tri thức này bao gồm:

- Lược đồ cơ sở dữ liệu (Database Schema): Tên bảng, tên cột, kiểu dữ liệu, các mối quan hệ và mô tả chi tiết của chúng.
- Tài liệu hướng dẫn (Documentation): Giải thích chuyên sâu về các bảng, cột cụ thể hoặc các quy tắc nghiệp vụ (business rules).
- Truy vấn ví dụ (Example Queries): Các truy vấn SQL đã thành công trong quá khứ và các câu hỏi ngôn ngữ tự nhiên tương ứng, giúp LLM học hỏi mẫu hình.

- Từ đồng nghĩa (Synonyms): Các biệt danh hoặc thuật ngữ thay thế cho các bảng và cột, đảm bảo hệ thống hiểu được ý định của người dùng ngay cả khi họ sử dụng ngôn ngữ không chuẩn.

Tăng cường ngữ cảnh (Augmentation): Thông tin đã truy xuất được sử dụng để tăng cường (augment) lời nhắc (prompt) cung cấp cho LLM. Lời nhắc được làm giàu này cung cấp cho LLM ngữ cảnh cần thiết để hiểu ý định (intent) của người dùng, nắm bắt cấu trúc (structure) của cơ sở dữ liệu.

Tạo sinh Truy vấn (Generation): prompt đã tăng cường ngữ cảnh, tạo ra một truy vấn SQL tương ứng với câu hỏi bằng ngôn ngữ tự nhiên và hoàn toàn tương thích với cơ sở dữ liệu mục tiêu.

Thực thi và tinh chỉnh: truy vấn SQL được tạo ra có thể được thực thi trên cơ sở dữ liệu. Nếu có lỗi xảy ra hoặc kết quả trả về không như mong đợi, hệ thống sử dụng các cơ chế phản hồi (ví dụ: thông báo lỗi SQL, phản hồi của người dùng) để thực hiện quy trình tinh chỉnh lặp đi lặp lại (iterative refinement), bằng cách điều chỉnh prompt hoặc truy xuất thông tin ngữ cảnh khác để cải thiện chất lượng truy vấn.

2.3.4. RAG trong Recommendation Systems

Các mô hình ngôn ngữ lớn (LLM) như ChatGPT (OpenAI), LLaMA (Meta) và Claude (Anthropic) đã được huấn luyện trên các tập dữ liệu khổng lồ để tạo ra văn bản mạch lạc và phù hợp với ngữ cảnh dựa trên prompt của người dùng. Các LLM này phân tích hồ sơ người dùng, tương tác trong quá khứ và thuộc tính mặt hàng để đưa ra các đề xuất được cá nhân hóa.

Tuy nhiên, các LLM này gặp phải một hạn chế cơ bản: hiện tượng ảo giác (Hallucinations). Do được huấn luyện trên các bộ dữ liệu đồ sộ, LLM có thể tạo ra các đề xuất không chính xác hoặc không tồn tại, bởi vì chúng không thể xác minh thông tin hoặc thiếu hiểu biết về ngữ cảnh thế giới thực, đặc biệt khi cần tích hợp các yếu tố thời gian thực như: Cập nhật sản phẩm mới, Xu hướng thị trường hiện tại, Đánh giá và phản hồi của khách hàng.

Để giải quyết vấn đề này, kiến trúc (RAG) như một cơ chế hiệu quả nhằm tăng cường khả năng truy xuất tri thức ngoài cho LLM. RAG giúp mô hình trích xuất thông tin thích hợp từ các nguồn dữ liệu bên ngoài, đảm bảo tính xác thực để đề xuất.

Các nguồn tri thức bên ngoài mà RAG khai thác:

- Cơ sở dữ liệu sản phẩm như danh mục sản phẩm trên các nền tảng thương mại điện tử.
- Nội dung do người dùng tạo như các đánh giá (reviews), diễn đàn, blog và xếp hạng của người dùng.
- Đồ thị tri thức như google's knowledge graph, wikidata, cung cấp cấu trúc ngữ nghĩa về các thực thể.
- Xu hướng thị trường và cập nhật giá theo thời gian thực (real-time market trends and pricing updates).

Việc tích hợp RAG cho phép LLM tập trung vào dữ liệu liên quan nhất thay vì chỉ dựa vào dữ liệu đã được huấn luyện trước. Điều này không chỉ tăng cường hiệu suất và độ chính xác của mô hình mà còn giúp khắc phục vấn đề thưa thớt dữ liệu (data sparsity) và hạn chế tối đa hiện tượng ảo giác trong quá trình tạo đề xuất.

2.4. Thuật toán và kỹ thuật áp dụng trong Multi-Agent System

2.4.1. Thuật toán áp dụng trong Multi-Agent System

Multi-Agent System (MAS) vận hành dựa trên các thuật toán cho phép các tác tử tự trị hợp tác, cạnh tranh hoặc cùng tồn tại nhằm đạt được những mục tiêu riêng lẻ hoặc mục tiêu chung. Các phương pháp phổ biến bao gồm các kỹ thuật dựa trên lý thuyết trò chơi, thuật toán đồng thuận và các phương pháp học tăng cường. Những thuật toán này được thiết kế để giải quyết các thách thức như phối hợp tác vụ, phân bổ tài nguyên và ra quyết định phân tán trong các môi trường mà mỗi tác tử chỉ có thông tin không đầy đủ hoặc có thể theo đuổi những mục tiêu xung đột.

2.4.1.1. Thuật toán phối hợp & lập kế hoạch

Distributed Constraint Optimization

Giải quyết các bài toán tối ưu hóa phân tán, nơi một hàm mục tiêu chung cần được tối ưu hóa dưới các ràng buộc phân tán giữa các agent. Tối ưu hóa toàn cục một hàm mục tiêu bằng cách điều chỉnh các biến của từng agent, với chi phí/lợi ích của một agent phụ thuộc vào giá trị biến của các agent lân cận. Phân bổ tài nguyên trong mạng lưới, lập lịch phân tán, hệ thống gợi ý, và mạng cảm biến.

Các thuật toán:

- Asynchronous Distributed OPTimization: tối ưu hóa phân tán không đồng

bộ, sử dụng cây ưu tiên để tìm lời giải, cho phép các agent chạy song song.

- Dynamic Programming Optimisation Protocol: quy hoạch động trên một cấu trúc cây được tạo ra từ đồ thị ràng buộc, đảm bảo tìm được lời giải tối ưu.
- Max-Sum Algorithm / Max-Plus Algorithm: thuật toán dựa trên truyền thông tin trên đồ thị nhân tử, được dùng để tìm lời giải gần tối ưu nhanh chóng.
- Complete/Incomplete DCOP solvers: Thuật toán tìm được lời giải tối ưu và không hoàn chỉnh tìm lời giải tốt nhưng không đảm bảo tối ưu.

Multi-Agent Planning (MAP)

Xây dựng các kế hoạch hành động cho một nhóm agent, cho phép mỗi agent tự lập kế hoạch và hợp tác để thực thi các kế hoạch đó. Bao gồm các kỹ thuật để chia sẻ kiến thức, tài nguyên và phối hợp các hành động.

Các thuật toán:

- MA-STRIPS, MAPR, FMAP (Forward Multi-Agent Planning): mở rộng của các thuật toán lập kế hoạch cổ điển sang môi trường đa tác tử, tập trung vào việc quản lý các hiệu ứng song song và phụ thuộc giữa các kế hoạch.
- Planning dựa trên Partial-Order Planning: Cho phép các agent lập kế hoạch theo từng phần, không cần sắp xếp thứ tự hành động một cách nghiêm ngặt, chỉ cần đảm bảo các ràng buộc thứ tự cần thiết.

2.4.1.2. Thuật toán giao tiếp & đồng thuận (Communication & Consensus)

Consensus Algorithms (Thuật toán đồng thuận)

Mục tiêu là để một nhóm agent đạt đến một giá trị thống nhất (ví dụ: vị trí, tốc độ, giá trị dữ liệu) mặc dù mỗi agent chỉ có thông tin cục bộ hoặc có lỗi. Được ứng dụng trong Robot swarm, mạng cảm biến, điều khiển phân tán, và blockchain.

Các thuật toán:

- Average Consensus: Các agent lặp đi lặp lại việc cập nhật giá trị của mình bằng cách lấy trung bình (hoặc trung bình có trọng số) của chính nó và các agent gần nhau, hội tụ về giá trị trung bình toàn cục.
- Weighted Consensus: Tương tự Average Consensus nhưng gán trọng số khác nhau cho thông tin từ các agent gần nhau.
- Leader-Follower Consensus: Một agent được chỉ định làm người dẫn đầu (Leader), và các agent còn lại điều chỉnh trạng thái của mình để theo kịp Leader.
- Byzantine Fault Tolerant Consensus: đạt đồng thuận khi có một số agent hoạt động sai, được áp dụng trong các hệ thống phi tập trung như blockchain.

Gossip Algorithms

Thông tin được lan truyền thông qua các tương tác ngẫu nhiên hoặc bán ngẫu nhiên giữa từng cặp agent trong mạng. Khả năng mở rộng cao, độ tin cậy tốt, và chịu lỗi trong các hệ thống lớn, phân tán, nơi việc giao tiếp đồng bộ là không khả thi.

2.4.1.3. Thuật toán ra quyết định và học tăng cường (Reinforcement Learning)

Multi-Agent Reinforcement Learning (MARL)

Agent học dựa trên tương tác trong môi trường và với các agent khác, giải quyết các vấn đề về tính ổn định của môi trường do sự thay đổi của các agent khác.

Các dạng MARL:

- Independent Q-Learning: Mỗi agent coi các agent khác là một phần của môi trường và học độc lập, đơn giản nhưng không tối ưu vì không ổn định.
- Centralized Training, Decentralized Execution (CTDE): Huấn luyện tập trung sử dụng thông tin toàn cục để ổn định quá trình học và thực thi phân tán, mỗi agent đưa ra quyết định dựa trên quan sát cục bộ.
- QMIX (Q-Mix): học Q-toàn cục bằng cách kết hợp tuyến tính các hàm Q-cục bộ của các agent, đảm bảo tính nhất quán giữa tối ưu cục bộ và toàn cục.
- Value Decomposition Networks: Tương tự QMIX, phân rã hàm Q-toàn cục thành tổng các hàm Q-cục bộ.
- Multi-Agent Deep Deterministic Policy Gradient: Mở rộng cho MARL, sử dụng CTDE; mỗi agent có một critic tập trung để huấn luyện agent phân tán.
- Multi-Agent PPO: Mở rộng của Proximal Policy Optimization cho MARL, thường dùng kiến trúc CTDE, sự ổn định và hiệu suất cao.

Game-theoretic RL

Áp dụng các khái niệm và giải pháp từ lý thuyết trò chơi vào RL đa agent.

Nash Q-learning: cân bằng nash, không agent nào có thể đơn phương thay đổi hành động để cải thiện kết quả.

Friend-or-Foe Q-learning: Một agent phân loại các agent khác là cộng tác hoặc đối kháng và tối ưu hóa hàm Q của mình.

2.4.1.4. Thuật toán đàm phán và ra quyết định (Negotiation & decision-making)

Giải quyết xung đột hoặc đạt được thỏa thuận tối ưu khi các agent có mục tiêu khác nhau hoặc cần trao đổi tài nguyên/nhiệm vụ.

Auction-based Algorithms

Các dạng Auction-based Algorithms:

- First-price: agent trả kết quả cao nhất.
- Second-price: agent trả kết quả cao thứ hai, khuyến khích agent trả kết quả trung thực.
- Combinatorial Auctions: hữu ích cho các nhiệm vụ phức tạp, phụ thuộc nhau.

Game Theory Algorithms

Equilibrium (Nash Equilibrium): Trạng thái ổn định, không agent nào muốn thay đổi hành động.

Stackelberg Games: một agent ra quyết định và các agent khác phản ứng lại.

Cooperative game solutions:

- Shapley value: phân bổ lợi ích của một nhóm agent cho từng agent khác dựa trên đóng góp cận biên.
- Core, Stable Matching: ghép cặp ổn định giữa hai tập hợp (ví dụ: người dùng và sản phẩm).

2.4.1.5. Thuật toán tổ chức và điều phối trong MAS

Task Allocation Algorithms (Phân công nhiệm vụ)

Xác định agent nào sẽ thực hiện nhiệm vụ nào để tối ưu hóa chi phí, hiệu suất hoặc lợi ích chung. Agent chia việc dựa trên:

- Chi phí: Agent có chi phí thấp nhất được ưu tiên.
- Độ phù hợp: Agent có kỹ năng hoặc tài nguyên phù hợp nhất.
- Tối ưu toàn cục: Sử dụng các thuật toán tối ưu hóa để đảm bảo tổng thể hệ thống đạt hiệu quả cao nhất.

Thuật toán:

- GREEDY (Cost-based Assignment): Phân công nhiệm vụ một cách tuần tự cho agent có chi phí thấp nhất tại thời điểm đó.
- Hungarian Algorithm (Optimal assignment): Giải quyết bài toán được phân công, tìm ra sự phân công tối ưu giữa agent và nhiệm vụ hoặc giữa hai tập hợp.
- Market-based allocation: Sử dụng cơ chế đấu giá như Contract Net Protocol để phân bổ nhiệm vụ theo cơ chế thị trường.

Coalition Formation Algorithms

Dynamic coalition: Các nhóm thay đổi thành viên theo thời gian và nhu cầu

nhiệm vụ.

Merge-and-split algorithms: đề xuất việc sáp nhập các nhóm nhỏ thành nhóm lớn hơn hoặc chia tách nhóm lớn thành nhóm nhỏ hơn để tăng hiệu suất.

Coalition formation game theory: Áp dụng Lý thuyết Trò chơi để xác định nhóm nào là ổn định và lợi ích được chia sẻ như thế nào.

2.4.1.6. Thuật toán trong MAS ứng dụng cho hệ thống gợi ý (Recommender Multi-Agent System)

Sử dụng kiến trúc đa agent để cải thiện chất lượng, tính đa dạng và khả năng giải thích của hệ thống gợi ý.

Multi-Agent Recommender Architecture

Phân chia các chức năng chính của hệ thống gợi ý thành các agent chuyên biệt:

- User Modeling Agent: theo dõi và mô hình hóa sở thích, hành vi, và bối cảnh của người dùng.
- Item Analysis Agent: phân tích các thuộc tính và đặc trưng của sản phẩm/mục (item).
- Recommendation Agent: Agent core thực hiện các thuật toán gợi ý chính (ví dụ: Collaborative Filtering, Matrix Factorization).
- Coordinator Agent: điều phối thông tin và kết quả từ các agent khác để đưa ra gợi ý cuối cùng.
- Explanation Agent: tạo ra lời giải thích cho kết quả gợi ý, tăng tính minh bạch và độ tin cậy.

Thuật toán áp dụng trong Recommendation Agent (MAS)

Collaborative Filtering (user-based, item-based): Gợi ý dựa trên sự tương đồng giữa người dùng hoặc giữa các mục.

Matrix Factorization (SVD, ALS, BPR): Phân tích ma trận tương tác User-Item thành các yếu tố tiềm ẩn (latent factors).

Graph-based Recommendation: mô hình hóa người dùng, mục và tương tác như một đồ thị để tìm đường đi và mối quan hệ phức tạp.

Reinforcement Learning Recommender (RL-based): Coi quá trình gợi ý là một quy trình ra quyết định tuần tự, tối ưu hóa kết quả (ví dụ: sự tương tác hoặc giữ chân người dùng).

Bandit Algorithms (Multi-Armed Bandit): các tình huống khám phá

(exploration) và khai thác (exploitation), đặc biệt là khi dữ liệu tương tác còn thưa thớt (cold-start).

Upper Confidence Bound: chọn mục có tiềm năng cao nhất dựa trên trung bình và độ bất định.

Thompson Sampling: chọn mục dựa trên xác suất (Bayesian probability).

Thuật toán điều phối giữa các agent gợi ý

Sử dụng để tổng hợp kết quả (ví dụ: danh sách gợi ý) từ nhiều agent/kỹ thuật gợi ý khác nhau (ensemble/hybrid recommender).

Weighted voting: Kết quả được tổng hợp bằng cách gán trọng số cho gợi ý của từng agent (ví dụ: agent có độ tin cậy cao hơn có trọng số lớn hơn).

Rank aggregation algorithms (thuật toán tổng hợp thứ hạng):

- Borda Count: Gán điểm cho mục dựa trên thứ hạng của chúng trong danh sách của từng agent.
- Kemeny-Young: Tìm ra thứ hạng tổng hợp tối ưu để giảm thiểu tổng khoảng cách (dissimilarity) với tất cả các thứ hạng thành phần.
- Reciprocal Rank Fusion: tổng hợp danh sách xếp hạng bằng cách sử dụng nghịch đảo của thứ hạng.

2.4.1.7. Thuật toán trong Multi-Agent LLM System

Lĩnh vực mới nổi sử dụng các Mô hình Ngôn ngữ Lớn (LLM) làm bộ não của các agent, tập trung vào khả năng lập luận và cộng tác.

Reasoning & Planning Agents (Lập luận và Lập kế hoạch)

Tree of Thought (ToT): LLM tạo ra và đánh giá nhiều khả năng (thoughts) trong một cấu trúc cây trước khi đưa ra câu trả lời, giúp cải thiện khả năng lập luận phức tạp.

Graph of Thoughts (GoT): Mở rộng của ToT, cho phép các ý nghĩ được sắp xếp thành một đồ thị linh hoạt hơn, hỗ trợ lặp lại, tổng hợp, và chỉnh sửa ý nghĩ.

Multi-Agent Debate: Các LLM Agent đóng vai trò khác nhau (ví dụ: công tố viên, luật sư bào chữa) và tranh luận với nhau để đạt được giải pháp chất lượng cao.

Reflexion / Self-Reflection: Agent có khả năng tự đánh giá kết quả của mình và sửa đổi hành vi/kế hoạch, sử dụng các lần thử trước làm kinh nghiệm.

Socratic Agent Collaboration: Các agent hỏi và trả lời lẫn nhau, sử dụng phương

pháp đàm thoại Socratic để đào sâu và làm rõ vấn đề.

Task Decomposition (Phân rã nhiệm vụ)

RQDecompose (LLM tự phân rã nhiệm vụ): LLM được hướng dẫn để chia một nhiệm vụ lớn thành một chuỗi các bước nhỏ (subtasks) có thể quản lý được.

Task Graph Planning: Biểu diễn các nhiệm vụ nhỏ và phụ thuộc của chúng dưới dạng một đồ thị, cho phép các agent thực hiện song song hoặc tuần tự theo một kế hoạch cấu trúc.

Arbitration Algorithms (Phân xử)

Nhiều LLM Agent đưa ra các câu trả lời/giải pháp khác nhau, cần một cơ chế để chọn ra kết quả tốt nhất.

- Majority voting: chọn câu trả lời được đa số các agent có chung kết quả.
- Confidence weighted voting: gán trọng số cho câu trả lời dựa trên độ tin cậy mà agent đưa ra (ví dụ: entropy thấp).
- LLM-based judge Agent: một LLM Agent chuyên biệt được huấn luyện để đóng vai trò trọng tài, đánh giá chất lượng và sự chính xác của các câu trả lời từ các agent khác để chọn ra kết quả cuối cùng.

2.4.2. Kỹ thuật áp dụng trong Multi-Agent System

Multi-Agent Systems (MAS) sử dụng các kỹ thuật phục vụ giao tiếp, phối hợp và ra quyết định, đồng thời khai thác các mô hình AI như LLM để tăng cường năng lực suy luận. Bên cạnh đó, MAS vận hành dựa trên các thuật toán chuyên biệt (như Lý thuyết trò chơi và Học tăng cường) cùng các mô hình kiến trúc điển hình (tập trung, phi tập trung, lai và phân cấp) nhằm quản lý các tác tử tự trị trong quá trình hợp tác, chia sẻ dữ liệu và giải quyết các bài toán phức tạp theo cả hướng cộng tác hoặc cạnh tranh.

Các thành phần trọng yếu bao gồm cơ chế truyền thông điệp phục vụ giao tiếp, kỹ thuật phân chia nhiệm vụ nhằm điều phối hoạt động và cơ chế điều phối tác vụ (orchestration) cho quản lý luồng công việc, từ đó hình thành các hệ thống AI có khả năng mở rộng và thích ứng cao.

2.4.2.1. Kỹ thuật giao tiếp

Trao đổi thông tin: Agent gửi thông điệp trực tiếp đến agent đích (point-to-point). Giao tiếp đơn giản, trực tiếp, cho các tác vụ cần bảo mật hoặc riêng tư.

Bảng chia sẻ thông tin chung: Các agent đọc và ghi thông tin vào một khu vực bộ nhớ/cấu trúc dữ liệu chung. Giải quyết các vấn đề phức tạp, nơi kiến thức cần được tích hợp từ nhiều nguồn khác nhau (ví dụ: nhận dạng tín hiệu).

Publish–Subscribe Model: Agent phát đi thông điệp về một chủ đề; các agent liên quan sẽ nhận được thông điệp đó. Giao tiếp phân tán, linh hoạt, agent không cần biết danh tính của nhau, giảm thiểu sự phụ thuộc.

Ontology & Semantic Communication: Sử dụng ngữ nghĩa hóa thông điệp dựa trên các thuật ngữ và mối quan hệ được định nghĩa trước để đảm bảo các agent hiểu thông điệp một cách rõ ràng. Tăng cường tính mạnh mẽ và khả năng tương tác giữa các agent được phát triển độc lập.

RDF, OWL, FIPA ACL: Các chuẩn mực để mã hóa ngữ nghĩa (RDF, OWL) và ngôn ngữ giao tiếp (FIPA ACL) được tiêu chuẩn hóa. Đảm bảo khả năng tương thích và giao tiếp có cấu trúc giữa các MAS.

2.4.2.2. Kỹ thuật nhận thức và cảm nhận môi trường

Sensor fusion techniques (Hợp nhất cảm biến)

Các kỹ thuật kết hợp dữ liệu từ nhiều cảm biến để tạo ra một ước tính môi trường chính xác và đáng tin cậy hơn.

Kalman Filter & Particle Filter: Các thuật toán lọc thống kê để ước tính trạng thái của môi trường theo thời gian, đặc biệt hữu ích cho việc theo dõi và định vị.

Multi-source data fusion: Kết hợp dữ liệu từ các loại cảm biến khác nhau (ví dụ: thị giác, âm thanh, radar).

Cảm biến phân tán

Sensor-sharing protocols: quy tắc để các agent chia sẻ dữ liệu cảm biến cục bộ.

Cooperative perception: Nhiều agent cùng nhau xây dựng một bản đồ hoạt động hoặc hiểu biết chung về môi trường.

2.4.2.3. Kỹ thuật bảo mật và tin cậy

Fault tolerance (Chịu lỗi)

Redundant agents: Sử dụng các agent dự phòng để tiếp quản nhiệm vụ khi agent chính gặp lỗi.

Byzantine-tolerant protocols: Các giao thức được thiết kế để chịu lỗi Byzantine

(hành vi độc hại hoặc sai lệch).

Security (Bảo mật)

Secure MAS communication: Đảm bảo mã hóa và tính toàn vẹn của thông điệp giữa các agent.

Authentication & authorization: Xác minh danh tính của agent và cấp quyền truy cập tài nguyên.

Privacy-preserving MAS: Sử dụng các kỹ thuật như Học Liên Bang (Federated Learning) hoặc Mã hóa Đồng hình (Homomorphic Encryption) để bảo vệ dữ liệu riêng tư.

2.4.2.4. Kỹ thuật sử dụng LLM trong Multi-Agent System

Reasoning Techniques

Tree of Thoughts và Graph of Thoughts: LLM tạo ra và khám phá các con đường lập luận khác nhau trước khi đưa ra kết quả cuối cùng.

Multi-Agent Debate: Agent tranh luận để thẩm định và cải thiện chất lượng của các giải pháp.

Arguments & Refutation Techniques: Agent tạo ra các lập luận (Arguments) để bảo vệ giải pháp và các phản bác (Refutations) để chỉ ra lỗi sai trong giải pháp của agent khác.

Phân rã nhiệm vụ

Recursive Task Planning: LLM tự phân rã một nhiệm vụ phức tạp thành các nhiệm vụ con một cách đệ quy cho đến khi đạt được các bước thực thi cơ bản.

LLM-based Task Graph Construction: LLM xây dựng một đồ thị các nhiệm vụ, xác định thứ tự và sự phụ thuộc giữa chúng để phối hợp hành động.

Phân xử và Tổng hợp

Majority vote & Confidence-weighted vote: Các phương pháp thống kê để tổng hợp kết quả từ nhiều LLM Agent (xem chi tiết ở phần trước).

LLM Judge / Moderator: Một LLM chuyên biệt được giao vai trò phân xử, đánh giá câu trả lời của các agent khác dựa trên tiêu chí chất lượng và tính chính xác, sau đó chọn hoặc tổng hợp kết quả tốt nhất.

2.5. API trong Multi-Agent Systems

2.5.1. Khái niệm

API (Application Programming Interface) là giao diện lập trình ứng dụng, cầu nối cốt lõi đảm bảo khả năng giao tiếp (communication), phối hợp (coordination) và tương tác với môi trường bên ngoài của các tác tử. API trong MAS được chia thành ba tầng chính:

- Intra-agent API: các hàm/tool mà một tác tử đơn lẻ sử dụng (tool calling, memory access, reasoning module).
- Inter-agent API: giao thức trao đổi thông tin giữa các tác tử (message passing, blackboard, contract-net protocol, v.v.).
- System-to-environment API: giao diện kết nối hệ thống đa tác tử với thế giới thực hoặc các hệ thống bên ngoài (web service, database, IoT devices, LLM API,...).

2.5.2. Vai trò của API trong MAS

Các Multi-Agent System dựa trên Large Language Model (LLM-based MAS) như AutoGen (Microsoft), LangGraph (LangChain), CrewAI, MetaGPT, CAMEL, và Letta đều coi API là thành phần trung tâm quyết định hiệu năng và khả năng mở rộng của hệ thống:

- Tool-calling API: Cho phép mỗi agent gọi các hàm bên ngoài (tính toán, tra cứu dữ liệu, thực thi hành động). OpenAI Function Calling (2023) và JSON Mode đã trở thành chuẩn thực tế cho tầng này.
- Structured communication API: Thay vì để các agent giao tiếp tự do bằng văn bản tự nhiên (dễ gây nhiễu và tốn token), các hệ thống mới sử dụng schema-based message (Pydantic, JSON Schema, Protocol Buffers) để định nghĩa rõ ràng loại tin nhắn (Request, Proposal, Vote, Commit, Inform, v.v.).
- Shared memory / vector store API: Pinecone, Weaviate, Chroma, Qdrant cung cấp API thống nhất để nhiều agent đồng thời đọc/ghi ký ức dài hạn, tránh hiện tượng “catastrophic forgetting” và giảm chi phí gọi LLM lặp lại.
- Orchestration API: LangGraph và Autogen Studio cung cấp StateGraph API cho phép lập trình viên định nghĩa luồng điều khiển phức tạp (conditional branching, loop, human-in-the-loop) một cách khai báo (declarative) thay vì viết code thủ công.

2.5.3. Các thiết kế API trong MAS hiện nay

Framework	Đặc trưng API nổi bật	Ưu điểm	Nhược điểm
AutoGen (2023)	ConversableAgent + register_function	Linh hoạt, dễ mở rộng tool	Message format chưa chặt chẽ
LangGraph (2024-2025)	StateGraph + ToolNode + Pregel architecture	Kiểm soát luồng trạng thái cực mạnh	Learning curve cao
CrewAI (2024)	Task → Tool → Agent schema rất rõ ràng	Dễ dùng cho người mới	Linh hoạt thấp hơn LangGraph
MetaGPT (2023)	SOP (Standard Operating Procedure) + Role-specific API	Tự động sinh code & tài liệu	Khó tùy biến sâu
Letta (2024)	Memory Block API + hierarchical memory	Quản lý bộ nhớ dài hạn tốt nhất hiện tại	Còn mới, cộng đồng nhỏ

Bảng 1: Các thiết kế API trong MAS hiện nay

2.5.4. Xu hướng thiết kế API trong MAS

Từ “text-only” → “structured-first”: 90% các dự án mới đều yêu cầu message tuân theo JSON Schema hoặc Pydantic models trước khi gửi.

Async & streaming API: Hỗ trợ real-time collaboration (giống Google Docs) giữa nhiều agent và con người.

Zero-trust security API: Mỗi tool call phải đi kèm signature và capability-based access control để tránh prompt injection lan truyền trong hệ thống.

Multi-modal API: Kết hợp text + image + audio tool calling (GPT-4o, Claude-3.5, Llama-3.2-vision).

Observability API: OpenTelemetry + LangSmith/LangFuse trở thành chuẩn để trace toàn bộ chuỗi tin nhắn và tool call trong hệ thống hàng trăm agent.

2.6. Mối liên hệ giữa Multi-Agent System và hệ thống khuyến nghị

Hệ thống gợi ý truyền thống (Collaborative Filtering, Matrix Factorization) thường hoạt động như một “hộp đen”, thiếu khả năng giải thích và khó xử lý các truy vấn mang tính ngữ nghĩa hoặc logic phức tạp. Việc tích hợp kiến trúc Đa tác tử (Multi-Agent System) vào hệ thống gợi ý mang lại nhiều lợi ích cốt lõi:

2.6.1. Khả năng giải thích

Trong RecSys truyền thống, mô hình chỉ trả về một danh sách item mà không cung cấp lý do đằng sau gợi ý.

Trong kiến trúc MAS, các agent có thể trao đổi và tranh luận với nhau, từ đó tạo ra lời giải thích tự nhiên.

Ví dụ: Analyst Agent gửi thông báo cho Manager Agent và người dùng: *“Tôi gợi ý phim này vì người dùng thường xem phim của Christopher Nolan và vừa xem Inception.”*

Điều này giúp tăng tính minh bạch và mức độ tin cậy của hệ thống.

2.6.2. Xử lý tác vụ phức tạp (Complex Reasoning)

Nhiều yêu cầu của người dùng mang tính ngữ nghĩa và không thể biểu diễn bằng vector tương tác: *“Tìm cho tôi bộ phim giống Titanic nhưng kết thúc có hậu và thời lượng không quá dài.”*

Các mô hình CF/MF không thể hiểu được các ràng buộc như “kết thúc có hậu” hay “không quá dài”.

MAS có thể phân rã yêu cầu qua mô hình LLM kết hợp ReAct. Ví dụ:

- Searcher Agent tìm các phim cùng chủ đề lãng mạn.
- Filter Agent lọc theo thời lượng.
- Review Reader Agent kiểm tra nội dung, đánh giá để xác định câu trả lời cuối cùng đến người dùng.

Qua đó MAS có khả năng suy luận và đáp ứng các yêu cầu phức tạp theo cách mà RecSys truyền thống không thể.

2.6.3. Giải quyết vấn đề Cold-start và dữ liệu tĩnh

RecSys truyền thống dựa trên mô hình học máy tĩnh: muốn cập nhật dữ liệu mới phải huấn luyện lại mô hình. Trong khi đó, MAS có thể tích hợp RAG (Retrieval-Augmented Generation) để:

- Lấy dữ liệu thời gian thực (lịch chiếu phim, tin tức, rating mới...).
 - Phản ứng ngay với các sự kiện mới (diễn viên scandal, phim mới ra mắt).
- Gợi ý được cập nhật mà không cần retrain mô hình nền.

Điều này đặc biệt hữu ích trong trường hợp cold-start hoặc lĩnh vực có dữ liệu thay đổi liên tục.

2.6.4. Cơ chế phản hồi và tự sửa lỗi (Reflexion/Self-correction)

Nếu người dùng từ chối gợi ý, MAS không chỉ ghi nhận tín hiệu âm như RecSys truyền thống mà có thể hỏi lại: *“Bạn không thích vì lý do gì?”*

Các agent sau đó thảo luận để điều chỉnh chiến lược tìm kiếm trong cùng phiên hội thoại—ví dụ:

- Giảm độ ưu tiên các phim buồn.
- Tăng trọng số cho phim kết thúc có hậu.
- Loại bỏ phim dài hơn 120 phút.

Khả năng tự sửa lỗi (reflexion) cho phép hệ thống học theo thời gian thực, thay vì đợi quá trình huấn luyện lại.

CHƯƠNG 3: DỮ LIỆU VÀ MÔ HÌNH

3.1. Dữ liệu

3.1.1. Tổng quan về nguồn dữ liệu

Chúng em sử dụng bộ dữ liệu MovieLens 100K (ml-100k), một trong những bộ dữ liệu chuẩn (benchmark dataset) phổ biến nhất trong lĩnh vực Hệ thống gợi ý (Recommender Systems). Bộ dữ liệu này được thu thập và phát hành bởi nhóm nghiên cứu GroupLens tại Đại học Minnesota.

Nguồn gốc: GroupLens Research Project.

Thời gian thu thập: từ 19/09/1997 đến 22/04/1998.

Thông tin bộ dữ liệu:

- 100.000 lượt đánh giá (ratings) từ 943 người dùng (users) cho 1.682 bộ phim (items).
- Mỗi người dùng đã đánh giá ít nhất 20 bộ phim.
- Thang điểm đánh giá: 1 đến 5 sao (số nguyên).

3.1.2. Cấu trúc dữ liệu hệ thống

3.1.2.1. Các tệp dữ liệu chính

Dữ liệu tương tác (Interactions)

Dữ liệu đánh giá được chia thành các tập huấn luyện (train), kiểm thử (test) và phát triển (dev) để phục vụ quy trình Machine Learning.

Tệp tin:

- `train.csv`: Tập huấn luyện chính (chiếm khoảng 80% dữ liệu).
- `test.csv`: Tập kiểm thử (chiếm khoảng 10% dữ liệu).
- `dev.csv`: Tập phát triển/validation (chiếm khoảng 10% dữ liệu).
- `all.csv`: Toàn bộ dữ liệu tương tác gộp lại.

Cấu trúc

Tên trường	Kiểu dữ liệu	Mô tả
<code>user_id</code>	Integer	Mã định danh duy nhất của người dùng (1-943).
<code>item_id</code>	Integer	Mã định danh duy nhất của bộ phim (1-1682).
<code>rating</code>	Integer	Điểm đánh giá của người dùng cho phim (1-5).
<code>timestamp</code>	Integer	Thời điểm đánh giá (Unix timestamp).

Bảng 2: Bảng cấu trúc dữ liệu tương tác

Dữ liệu người dùng (Users)

Tập tin user.csv chứa thông tin nhân khẩu học của người dùng, giúp hệ thống hiểu rõ hơn về đối tượng sử dụng để cá nhân hóa gợi ý.

Cấu trúc

Tên trường	Kiểu dữ liệu	Mô tả
user_id	Integer	Mã định danh người dùng (Primary Key).
age	Integer	Tuổi của người dùng.
gender	String	Giới tính ('M': Nam, 'F': Nữ).
occupation	String	Nghề nghiệp (ví dụ: student, engineer, doctor...).
zip_code	String	Mã bưu chính (vùng miền).

Bảng 3: Bảng cấu trúc dữ liệu người dùng.

Dữ liệu phim (Items)

Tập tin: item.csv chứa thông tin chi tiết về các bộ phim, bao gồm tiêu đề và thể loại. Đây là nguồn dữ liệu quan trọng cho Content-based Filtering và hiển thị thông tin cho người dùng.

Cấu trúc

Tên trường	Kiểu dữ liệu	Mô tả
item_id	integer	Mã định danh bộ phim (Primary Key).
title	string	Tên phim (kèm năm phát hành). Ví dụ: “Toy Story (1995)”.
release_date	string	Ngày phát hành phim (định dạng dd-MM-yyyy).
video_release_date	string	Ngày phát hành video
genre	string	Danh sách thể loại phân cách bởi dấu gạch đứng (‘
item_attributes	string	Chuỗi thuộc tính tổng hợp dùng cho text embedding hoặc hiển thị. Ví dụ: “Title: Toy Story (1995), Genres: Animation Childrens Comedy”.

Bảng 4: Bảng cấu trúc dữ liệu phim

3.1.2.2. Cơ sở dữ liệu SQLite

Hệ thống sử dụng cơ sở dữ liệu SQLite (database.db) để lưu trữ dữ liệu có cấu trúc. Việc sử dụng SQLite giúp các Agent có thể thực hiện các truy vấn SQL phức tạp một cách hiệu quả ngay trong môi trường phát triển, hỗ trợ các Agent như Searcher (cho truy vấn lọc ứng viên) và Analyst. Lưu trữ dưới file `database.db`

Các bảng (Tables): Dữ liệu CSV ban đầu được chuyển đổi và nhập vào ba bảng chính, thiết lập quan hệ khóa ngoại cần thiết

- `users`: Tương ứng với dữ liệu trong `user.csv`.
- `items`: Tương ứng với dữ liệu trong `item.csv`.
- `ratings`: Tương ứng với dữ liệu trong `all.csv` (hoặc `train.csv` tùy cấu hình), liên kết `users` và `items`.

Sơ đồ quan hệ

- `users.user_id` ↔ `ratings.user_id` (1-N)
- `users.item_id` ↔ `ratings.item_id` (1-N)

3.1.2. Thông kê và Đặc điểm dữ liệu

Phân bố đánh giá (Rating Distribution): Dữ liệu có xu hướng lệch về các đánh giá tích cực (3, 4, 5 sao), phản ánh tâm lý người dùng thường đánh giá những phim họ thích hoặc xem hết.

- 5 sao: ~21%
- 4 sao: ~34% (Phổ biến nhất)
- 3 sao: ~27%
- 2 sao: ~11%
- 1 sao: ~6%

Độ thưa (Sparsity) cho thấy ma trận tương tác User-Item rất thưa, đây là thách thức chính cho các thuật toán gợi ý.

- Tổng số ô trong ma trận: $943 \times 1682 = 1,586,126$
- Số lượng đánh giá thực tế: 100,000
- Độ thưa (Sparsity Level):

$$Sparsity = 1 - \frac{100,000}{1,586,126} \rightarrow Sparsity \approx 1 - 0.063 \approx 93.7\%$$

Đặc điểm người dùng và phim:

- Người dùng: Đa dạng về độ tuổi và nghề nghiệp, tập trung nhiều ở nhóm sinh viên (student), kỹ sư (engineer), và giáo dục (educator).
- Phim: Bao gồm nhiều thể loại, trong đó Drama và Comedy là hai thể loại phổ biến nhất. Một bộ phim có thể thuộc nhiều thể loại (Multi-label).

3.1.3. Tiền xử lý dữ liệu

Trước khi đưa vào hệ thống, dữ liệu được tiền xử lý:

Làm sạch, kiểm tra và loại bỏ các dòng dữ liệu lỗi.

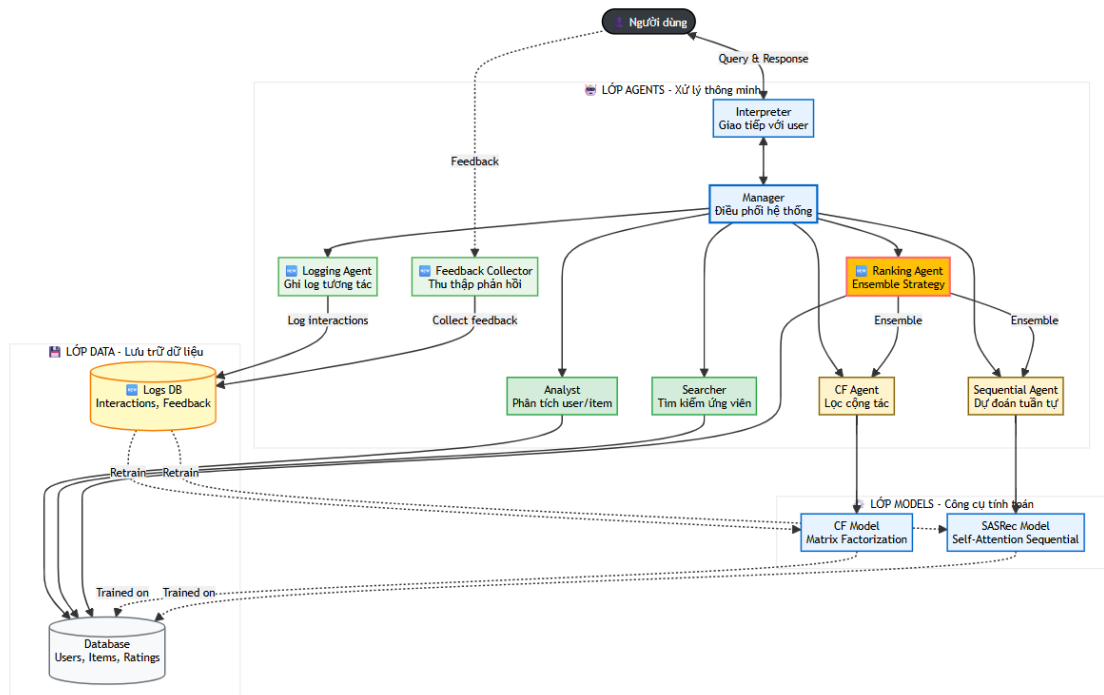
Mã hóa (encoding):

- Chuyển đổi các cột thể loại (genres) sang dạng one-hot encoding (như trong bảng `item.csv`).
- Đảm bảo ID người dùng và phim là liên tục và bắt đầu từ 1 (hoặc 0 tùy mô hình).

Chia tập dữ liệu (Splitting): Sử dụng phương pháp Leave-One-Out hoặc chia theo tỷ lệ thời gian (Time-based splitting) để tạo tập train/test/dev, đảm bảo tính khách quan khi đánh giá mô hình dự đoán tuần tự (Sequential Recommendation).

3.2. Kiến trúc hệ thống

3.2.1. Tổng quan kiến trúc



Hình 2: Kiến trúc hệ thống của đề tài

3.2.1.1. Lớp Người dùng (User Layer)

Giao diện của hệ thống.

- Tiếp nhận yêu cầu đầu vào từ người dùng (ví dụ: “*Gợi ý cho tôi các phim hài lãng mạn mới nhất*” hoặc “*Tôi không thích gợi ý phim hành động nữa*”) và hiển thị kết quả gợi ý cuối cùng sau khi được xử lý.
- Lớp này đảm bảo tính tương tác *trực tiếp* và *thân thiện* với người dùng, thu thập cả truy vấn lẫn phản hồi một cách rõ ràng (*explicit*) hoặc ngầm định (*implicit*).

3.2.1.2. Lớp Agents – Xử lý Thông minh (Intelligent Agents Layer)

Trung tâm điều khiển và xử lý thông tin của hệ thống, các Agent chuyên biệt mô phỏng quá trình tư duy và phân tích nhiều bước của con người.

Manager Agent (Điều phối hệ thống): bộ não điều hành của hệ thống.

- Nhận yêu cầu đã được Interpreter dịch, sau đó thiết lập một lộ trình xử lý logic. Nó quyết định *thứ tự* và *tác tử* nào cần được kích hoạt.
- Cải tiến vòng lặp: Manager có khả năng tiếp thu phản hồi từ Feedback Collector để tinh chỉnh các quyết định điều phối trong các lần gợi ý tiếp theo, đảm bảo tính linh hoạt của hệ thống.

Interpreter Agent (Giao tiếp với người dùng): chuyên đổi ngôn ngữ tự nhiên của người dùng thành một định dạng dữ liệu có cấu trúc mà hệ thống có thể xử lý. Nó thực hiện Phân tích Mục đích (Intent Analysis) (ví dụ: mục đích là *gợi ý phim*) và Trích xuất Thực thể (Entity Extraction) (ví dụ: thể loại *hài lãng mạn*, thời gian *mới nhất*).

Analyst Agent (Phân tích user/item): phân tích dữ liệu chuyên sâu, truy cập vào Main Database để xây dựng hồ sơ thời gian thực của người dùng (lịch sử mua sắm, tương tác gần đây) hoặc phân tích sâu đặc trưng của item. Kết quả là các Insight quan trọng được truyền đến các Agent gợi ý để làm cơ sở cho việc dự đoán.

Searcher Agent (tìm kiếm): Sàng lọc sơ bộ dựa trên các tiêu chí ban đầu (thể loại, độ phổ biến, mức độ mới), nó nhanh chóng tìm kiếm và tạo ra một Danh sách Ứng viên (Candidate List) từ hàng triệu item có sẵn. Mục đích là giảm thiểu số lượng item cần tính toán chi tiết ở các bước sau.

Hệ thống sử dụng các Agent chuyên về các mô hình ML khác nhau để đảm bảo tính đa dạng và chính xác:

- CF Agent (Collaborative Filtering): mô hình CF (Matrix Factorization) để dự đoán sở thích dựa trên sự tương đồng với những người dùng khác.
- Sequential Agent: mô hình SASRec (Self-Attention) để phân tích chuỗi hành vi của người dùng theo thời gian. Rất mạnh mẽ trong việc dự đoán *bước tiếp theo* của người dùng.

Ranking Agent: ra quyết định, nhận kết quả từ nhiều nguồn (CF Agent, Sequential Agent) và kết hợp chúng bằng các chiến lược như *Weighted Sum* (tổng có trọng số) hoặc *Rank Fusion*. Mục tiêu là tạo ra một Danh sách Gợi ý Cuối cùng có độ chính xác và độ ổn định cao hơn bất kỳ mô hình đơn lẻ nào.

Logging Agent: ghi chép mọi hành động, quyết định của Agent và kết quả. Đây là nguồn dữ liệu quan trọng cho việc kiểm tra và gỡ lỗi.

Feedback Collector: thu thập dữ liệu phản hồi (like, dislike, thời gian xem, v.v.) và gửi chúng về lớp DATA để cải thiện chất lượng mô hình theo thời gian.

3.2.1.3. Lớp DATA

Logs Database: dữ liệu động về tương tác mới nhất và phản hồi, được dùng để tinh chỉnh mô hình.

Main Database: dữ liệu cốt lõi (Hồ sơ User/Item, lịch sử rating) dùng để huấn luyện các mô hình chính.

3.2.1.4. Lớp MODELS

CF Model và SASRec Model là các công cụ thuật toán. Các Agent tương ứng sẽ gọi các mô hình này để tính toán và dự đoán, đảm bảo sự tách biệt giữa logic nghiệp vụ (agent) và logic thuật toán (model).

3.2.2. Kiến trúc luồng xử lý của Multi-Agent

User Query: Người dùng nhập yêu cầu qua giao diện “*Hãy gợi ý bộ phim giống Inception cho tôi*”. Truy vấn được chuyển ngay đến Phase 1.

Phase 1 – Input & Routing (Tiếp nhận và định tuyến truy vấn): giai đoạn phân tích cú pháp và định danh.

- Interpreter Agent: phân tích ngữ nghĩa truy vấn. Đầu ra xác định rõ ràng các tham số:

- + user-id: 123
- + target-item-id: 456 (Inception)
- + intent: recommend_similar_item (gợi ý mục tương tự)
- Extractor Agent: Trích xuất các thực thể và từ khóa liên quan (ví dụ: phim, Sci-Fi, giống).
- Manager / Router Agent: Nhận kết quả phân tích và quyết định rằng yêu cầu này cần một kế hoạch phân tích dữ liệu chuyên sâu gửi tới Phase 2.

Phase 2 – Analyst Planning (Lập kế hoạch xử lý) Analyst Agent (LLM) thực hiện, đóng vai trò lập kế hoạch.

- Analyst (LLM Agent): Dựa trên intent (recommend_similar_item), Agent lập ra một chuỗi hành động Multi-Step để đạt được mục tiêu.
- Item selection:
 - Gọi UserInfo Tool (lấy thông tin cơ bản người dùng).
 - Gọi ItemInfo Tool (lấy chi tiết mục tiêu gợi ý - Inception).
 - Gọi UserHistory Tool (lịch sử đánh giá của người dùng).
 - Gọi RatingPredictor Tool (mô hình AI dự đoán điểm).
 - Đầu ra: Kế hoạch chi tiết được gửi lại cho Manager để thực thi.

Phase 3 – Data Retrieval Tools: hệ thống tải dữ liệu cần thiết từ các file hoặc database:

Công cụ (Tool)	Dữ liệu	Đầu vào (Input)	Đầu ra (Output)
UserInfo Tool	users.csv	user-id=123	Hồ sơ: John Doe, 25 tuổi, kỹ sư.
ItemInfo Tool	items.csv	item-id=456	Chi tiết: Inception (Sci-Fi, 2010), Đạo diễn Nolan.
UserHistory Tool	ratings.csv	user=123	Lịch sử đánh giá: Interstellar 5*, Dark Knight 4.5*.
Phản hồi đến			Dữ liệu được tập hợp và gửi tới Phase 4.

Bảng 5: Data Retrieval Tools của hệ thống

Phase 4 – Rating Predictor Tool: dự đoán điểm gợi ý

- Mô hình Gợi ý (CF / Matrix Factorization): Tính toán mức độ phù hợp của item được hỏi (Inception) đối với người dùng (John Doe) dựa trên các yếu tố đã học. Sử dụng các vector ẩn (latent factors):

$$\text{Predicted Rating} = \text{User Factors} \cdot \text{Item Factors}$$

- Hệ thống dự đoán rằng người dùng sẽ chấm 4.5* cho phim này. Kết quả dự đoán được gửi trở lại Analyst Agent.

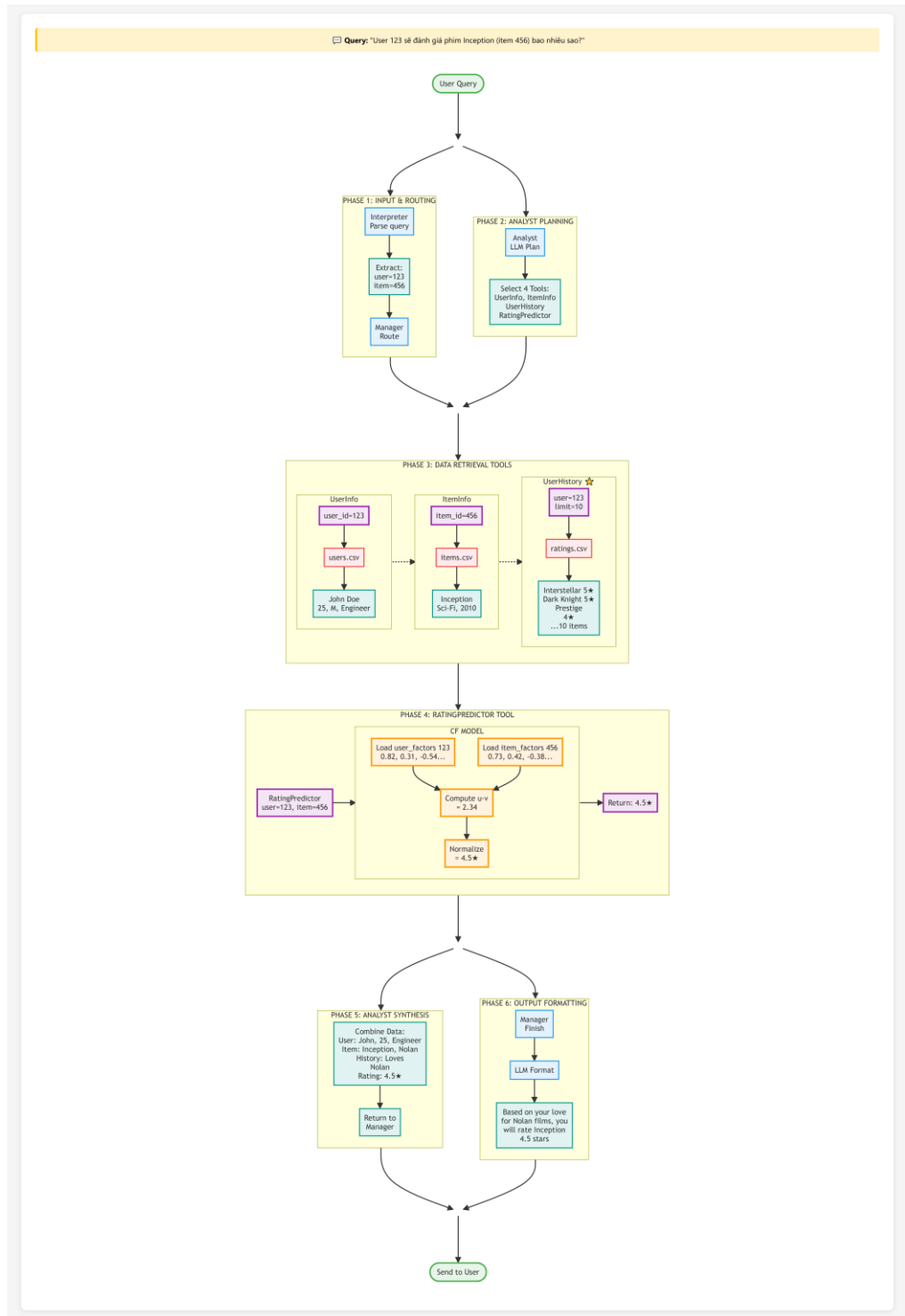
Phase 5 – Analyst Synthesis (phân tích tổng hợp kết quả) Analyst Agent sử dụng khả năng lý luận của LLM để tổng hợp và diễn giải các mảnh dữ liệu rời rạc. Kết hợp các dữ liệu thô từ Phase 3 và kết quả dự đoán từ Phase 4.

- User: John Doe, kỹ sư
- Target item: Inception (Sci-Fi, Nolan)
- Lịch sử xem: Phim của Nolan (Interstellar, Dark Knight)
- Dự đoán: Rất phù hợp (4.5*)
- Output: Gợi ý nên tập trung vào thể loại Sci-Fi và phong cách Nolan.

Phase 6 – Output Formatting (sinh câu trả lời) Manager chuyển kết quả tổng hợp sang cho một Agent chuyên trách về ngôn ngữ.

- LLM Formatter Agent: Chuyển đổi dữ liệu có cấu trúc từ Analyst thành một câu trả lời tự nhiên, trôi chảy, và có tính giải thích.
- Câu trả lời Sinh ra: “Dựa trên sở thích của bạn với các phim khoa học viễn tưởng (như Interstellar và Dark Knight) cùng phong cách đạo diễn Christopher Nolan, chúng tôi dự đoán bạn sẽ rất thích Inception với mức điểm tiềm năng là 4.5/5. Chúng tôi sẽ gợi ý thêm các phim khác cùng thể loại.”

Send to user (gửi kết quả): Kết quả được hiển thị trên Lốp người dùng.



Hình 3: Kiến trúc luồng xử lý đa tác tử trong hệ thống Recommendation

3.2.3. Sơ đồ kiến trúc xử lý truy vấn đa tác tử cho Recommendation System

3.2.3.1. User Query (Người dùng gửi Truy vấn)

Đầu vào: Yêu cầu gợi ý dựa trên hành vi gần đây của người dùng.

Ví dụ: “*Người dùng mở ứng dụng và hệ thống cần gợi ý phim tiếp theo.*”

Truy vấn được đưa vào Lớp Agent để xử lý.

3.2.3.2. Phase 1 – Input & Routing (tiếp nhận và định tuyến)

Giai đoạn xác định ý định và chuẩn bị dữ liệu đầu vào.

Interpreter Agent: phân tích ngữ cảnh hoặc truy vấn để xác định mục tiêu.

Đầu ra:

- user-id: 123.
- intent: sequential_recommendation (gợi ý tuần tự).

Manager Agent (Router): nhận intent và quyết định cần gọi Analyst Agent để kích hoạt pipeline xử lý tuần tự (Phase 2).

Output: truy vấn được chuẩn hóa và hệ thống sẵn sàng truy xuất dữ liệu lịch sử.

3.2.3.3. Phase 2 – Get User History (lấy lịch sử người dùng)

Analyst Agent: Gọi UserHistory Tool để truy xuất chuỗi tương tác (phiên bản tuần tự của lịch sử).

Database Query: Truy vấn ratings.csv hoặc hệ thống cơ sở dữ liệu để lấy danh sách item đã xem theo thứ tự thời gian.

Output (chuỗi Tuần tự): Dữ liệu được sắp xếp:

$$Sequence = \{Item_1, Item_2, \dots, Item_{10}\}$$

3.2.3.4. Phase 3 – SASRec Prediction (Dự đoán phim tiếp theo)

Analyst gọi SequentialPredictor Tool: Công cụ này bao bọc mô hình SASRec (Self-Attentive Sequential Recommendation), một mô hình dựa trên cơ chế Self-Attention (tương tự Transformer).

Quá trình SASRec Model:

- Embedding: biến đổi các Item trong chuỗi thành các vector số học.
- Self-Attention Encoder: mô hình học cách trọng số hóa các item trước đó (ví dụ: prestige quan trọng hơn Inception nếu gần đây hơn) để hiểu sở thích hiện tại của người dùng.
- Prediction: Dự đoán xác suất cho item tiếp theo.

Kết quả mô hình (Top 5): Danh sách item được dự đoán có khả năng cao nhất: Tenet, Dunkirk, Memento, Batman, Following. Các phim được gợi ý phản ánh xu hướng thời gian (phim mới hơn) và phong cách/đạo diễn (Nolan).

3.2.3.5. Phase 4 – Enrichment & Output (làm giàu dữ liệu và sinh câu trả lời)

Kết quả dự đoán được làm giàu và chuyển thành câu trả lời tự nhiên.

Truy vấn Enrichment: Analyst Agent gọi ItemInfo Tool để truy vấn items.csv và lấy Metadata (thể loại, đạo diễn, năm) cho các phim được gợi ý (Tenet, Dunkirk...).

Analyst Agent (Tổng hợp): Kết hợp Danh sách Top 5 và Metadata, chuẩn hóa thành một đối tượng dữ liệu duy nhất.

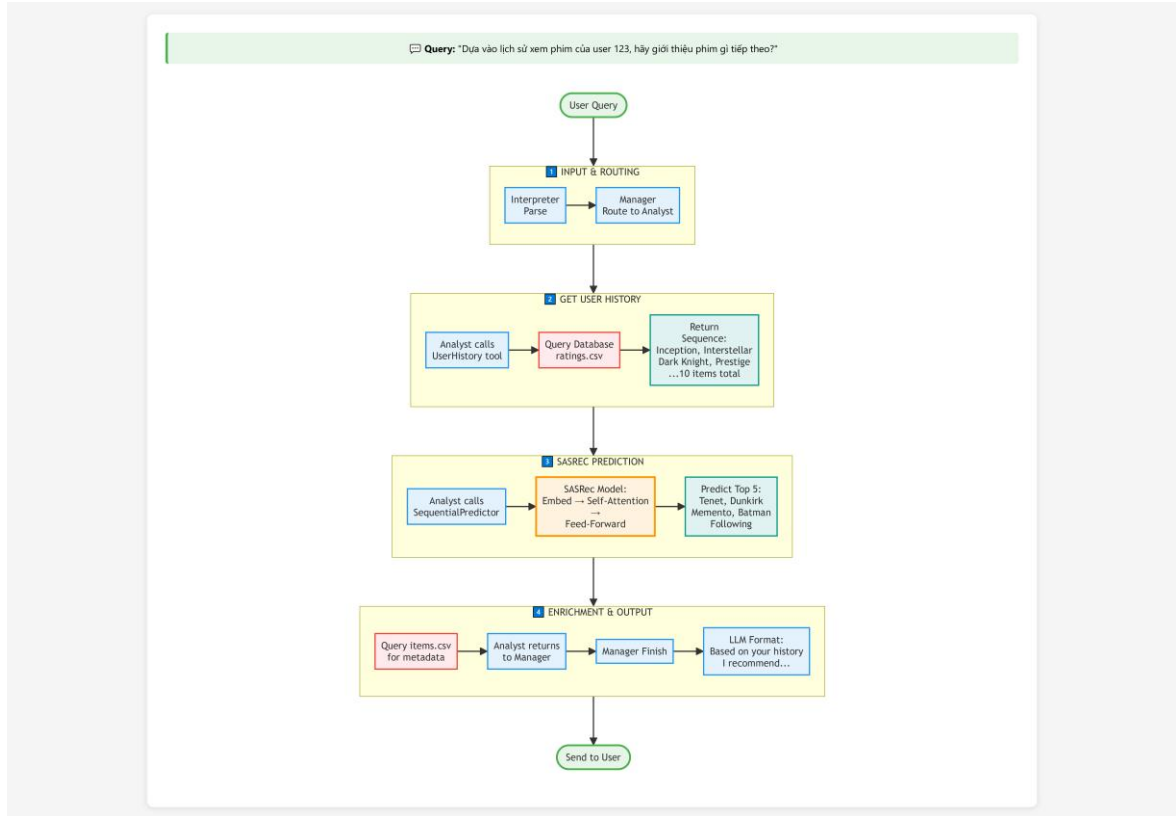
Ví dụ: [{ 'title': 'Tenet', 'genre': 'Sci-Fi/Action', 'director': 'Nolan'}, ...]

Manager Finish: nhận luồng xử lý đã hoàn tất và chuyển kết quả tổng hợp tới Agent tạo output.

LLM Formatter Agent: khả năng ngôn ngữ để biến dữ liệu có cấu trúc thành văn bản dễ hiểu. Output: “*Dựa trên lịch sử xem phim gần đây của bạn (Inception, Interstellar, Dark Knight...), các bộ phim được đề xuất tiếp theo là Tenet (hành động khoa học viễn tưởng), Dunkirk và Memento. Các phim này đều rất phù hợp với phong cách xem tuần tự của bạn.*”

3.2.3.6. Send to user (trả kết quả cho người dùng)

Kết quả được hiển thị trên giao diện người dùng (UI) hoặc qua API.



Hình 4: Sơ đồ kiến trúc xử lý truy vấn đa tác tử cho Recommendation System sử dụng mô hình tuần tự SASRec

3.2.2. Các tác nhân hệ thống

3.2.2.1. ReActSystem

ReActSystem (Reason + Act) là một cơ chế cho phép mô hình vừa lập luận (reasoning) vừa thực hiện hành động (acting) song song. Cơ chế này được giới thiệu nhằm nâng cao khả năng tư duy nhiều bước và sử dụng công cụ của LLM.

Chức năng

- Phân tích nội dung yêu cầu từ người dùng
- Tạo chuỗi suy luận nội bộ
- Lựa chọn hành động cụ thể (ví dụ: gọi API, truy vấn dữ liệu, đọc file)
- Nhận kết quả và suy luận tiếp
- Tạo phản hồi cuối cùng đã qua suy luận

Vai trò

- ReActSystem đóng vai trò như “bộ não vận hành từng bước”, giúp từng agent hiểu cách tương tác với dữ liệu và môi trường.
- ReAct giúp agent biết khi nào nên truy vấn dữ liệu sản phẩm, kiểm tra hành vi người dùng và gọi mô hình gợi ý.

3.2.2.2. ReflectionSystem

ReflectionSystem là hệ thống cho phép agent tự đánh giá, phản tư và cải thiện câu trả lời trước khi gửi cho người dùng. Đây là mô-đun nhằm tăng chất lượng và độ tin cậy của kết quả.

Chức năng

- Kiểm tra lỗi logic trong câu trả lời.
- Bổ sung thông tin còn thiếu.
- Chuẩn hóa câu văn.
- Loại bỏ sai sót sai ngữ cảnh.
- Tối ưu hóa độ mạch lạc và chính xác.

Vai trò

- ReflectionSystem đảm nhận vai trò như bộ phận kiểm duyệt chất lượng (Quality Assurance) cho toàn hệ thống.
- Trong hệ thống gợi ý: Nếu agent trả lời chưa đủ chính xác, hoặc còn mơ hồ, thiếu căn cứ, reflectionsystem sẽ tự động chỉnh sửa trước khi phản hồi cho người dùng.
- Giúp hệ thống trở nên an toàn, chính xác và chuyên nghiệp hơn.

3.2.2.3. ChatSystem

ChatSystem là hệ thống chịu trách nhiệm xử lý giao tiếp hội thoại với người dùng, đảm bảo phản hồi tự nhiên, mạch lạc và phù hợp với ngữ cảnh.

Chức năng

- Hiểu câu hỏi người dùng thông qua NLP
- Giữ ngữ cảnh xuyên suốt cuộc hội thoại
- Trả lời thân thiện và đúng trọng tâm
- Chuyển nội dung từ các agent phân tích thành dạng giao tiếp tự nhiên

Vai trò

- ChatSystem đóng vai trò tiếp nhận yêu cầu và agent trình bày kết quả sau khi các agent khác đã xử lý
- Người dùng hỏi câu phức tạp → ChatSystem diễn giải → gửi đến AnalyseSystem
- Sau khi hệ thống xử lý → ChatSystem chuyển đổi thành câu trả lời sinh động, dễ hiểu
- Đảm bảo trải nghiệm người dùng tốt nhất.

3.2.2.4. AnalyseSystem

AnalyseSystem là hệ thống chuyên xử lý các tác vụ phân tích logic, phân tích dữ liệu và trích xuất thông tin cần thiết nhằm phục vụ các tác vụ chuyên sâu.

Chức năng

- Phân tích yêu cầu phức tạp của người dùng.
- Tách nhiệm vụ thành các phần nhỏ.
- Truy vấn và xử lý dữ liệu.
- Thực hiện các phép lọc, thống kê, so sánh.
- Chuẩn bị dữ liệu cho các hệ thống khác (ví dụ: hệ thống gợi ý).

Vai trò

- AnalyseSystem đóng vai trò như bộ não phân tích dữ liệu của toàn hệ thống.
- Ví dụ người dùng hỏi: *“Tìm các phim có thể loại kinh dị nhưng rating cao trong 6 tháng gần đây.”*
- AnalyseSystem sẽ hiểu yêu cầu → truy vấn dataset → lọc sản phẩm theo tiêu chí → gửi danh sách cho Recommendation Agent
- AnalyseSystem là phần không thể thiếu trong hệ thống gợi ý.

3.2.2.5. CollaborationSystem

CollaborationSystem là hệ thống điều phối và phối hợp giữa các agent trong kiến trúc Multi-Agent. Nó giúp nhiều agent có thể làm việc song song, trao đổi thông tin và hoàn thành nhiệm vụ phức tạp.

Chức năng

- Chia nhiệm vụ cho các agent phù hợp.
- Điều phối luồng thông tin giữa các hệ thống.
- Xử lý xung đột hoặc kết quả không đồng nhất.
- Kết hợp kết quả từ nhiều agent.
- Quyết định kết quả cuối cùng trả về cho người dùng.

Vai trò

- CollaborationSystem đóng vai trò như người quản lý (coordinator manager), giúp nhiều agent trở thành một hệ thống thống nhất thay vì hoạt động rời rạc.

Ví dụ xử lý 1 câu hỏi:

- ChatSystem tiếp nhận câu hỏi
- AnalyseSystem phân tích yêu cầu
- ReActSystem truy vấn dữ liệu
- ReflectionSystem kiểm tra kết quả
- ChatSystem phản hồi

3.2.3. Các tác vụ (Tools)

3.2.3.1. Rating Prediction

Dự đoán điểm số mà người dùng sẽ đánh giá một sản phẩm dựa trên sở thích và lịch sử tương tác. User Analyst phân tích hành vi của người dùng, Item Analyst phân tích đặc trưng sản phẩm. Từ hai nguồn thông tin này, Manager đưa ra dự đoán rating chính xác.

Chức năng

- Phân tích hành vi và lịch sử đánh giá của người dùng.
- Phân tích đặc trưng và mức độ phổ biến của sản phẩm.
- Tổng hợp thông tin từ cả người dùng và sản phẩm để đưa ra dự đoán điểm số phù hợp.

Vai trò

- User analyst: phân tích chi tiết sở thích, phong cách đánh giá và lịch sử tương tác của người dùng.
- Item analyst: phân tích đặc trưng, đánh giá gần đây và mức độ phù hợp của sản phẩm.
- Manager: tổng hợp dữ liệu từ hai analyst, đưa ra dự đoán cuối cùng về rating dự kiến.
- Giúp hệ thống hiểu rõ xu hướng đánh giá của người dùng, hỗ trợ việc cá nhân hoá gợi ý.

3.2.3.2. Sequential Recommendation

Dự đoán sản phẩm tiếp theo mà người dùng có khả năng quan tâm dựa trên chuỗi hành vi trước đó.

User Analyst phân tích sở thích ngắn hạn và dài hạn. Reflector hỗ trợ tối ưu định dạng và tránh sai sót khi nhiệm vụ phức tạp hơn (xếp hạng danh sách sản phẩm).

Chức năng

- Xác định sở thích ngắn hạn và dài hạn từ chuỗi hành vi.

- Mô hình hóa thứ tự và quan hệ giữa các sản phẩm đã tương tác.
- Dự đoán danh sách sản phẩm tiếp theo theo độ ưu tiên (ranking).

Vai trò

- User analyst: phân tích liên tục sở thích ngắn hạn/dài hạn từ lịch sử dài hơn nhiều so với RP.
- Item analyst: chỉ hỗ trợ phân tích những sản phẩm quan trọng (do số lượng item trong chuỗi rất lớn).
- Reflector: xử lý định dạng phức tạp của kết quả (ví dụ: danh sách xếp hạng), kiểm tra và tối ưu tính hợp lý.
- Manager: tổng hợp chuỗi phân tích và đưa ra danh sách gợi ý cuối.
- Dự đoán hành vi tiếp theo.

3.2.3.3. Explanation Generation

Tạo ra lời giải thích rõ ràng và phù hợp cho các gợi ý đưa ra.

Hệ thống phân tích sâu người dùng và sản phẩm, đồng thời Searcher có thể tìm thêm thông tin bên ngoài (ví dụ: đạo diễn phim) để tạo ra lời giải thích thuyết phục hơn.

Chức năng

- Phân tích thuộc tính người dùng và sản phẩm liên quan đến gợi ý.
- Tạo ra câu giải thích ngắn gọn nhưng đầy đủ lý do.
- Truy xuất thêm thông tin ngoài dataset nếu cần (ví dụ: diễn viên, đạo diễn, thể loại chi tiết,...).

Vai trò

- User analyst: phân tích sở thích của người dùng liên quan đến lý do gợi ý.
- Item analyst: mô tả đặc trưng sản phẩm, điểm mạnh và mối liên hệ với sở thích người dùng.
- Searcher: truy xuất thông tin bổ sung mà dataset không chứa, giúp giải thích sâu và chính xác hơn.
- Manager: tổng hợp toàn bộ phân tích để tạo lời giải thích mạch lạc.
- Nâng cao độ tin cậy của hệ thống gợi ý, cải thiện trải nghiệm người dùng và tăng tỷ lệ chấp nhận gợi ý.

3.2.3.4. Conversational Recommendation

Gợi ý sản phẩm thông qua hội thoại tự nhiên với người dùng.

Task Interpreter giúp làm rõ yêu cầu từ đoạn hội thoại. Nếu người dùng đề cập

thông tin hệ thống chưa biết, Searcher sẽ hỗ trợ truy xuất để Manager hiểu và đưa ra gợi ý chính xác.

Chức năng

- Diễn giải và xác định ý định từ hội thoại tự nhiên.
- Trích xuất thông tin quan trọng từ nội dung người dùng cung cấp.
- Tạo gợi ý phù hợp dựa trên sở thích được cập nhật liên tục theo cuộc trò chuyện.

Vai trò

- Task Interpreter: chuyển nội dung hội thoại thành nhiệm vụ rõ ràng để xử lý.
- Searcher: truy xuất ý nghĩa hoặc thông tin người dùng đề cập mà Manager chưa biết.
- User Analyst: phân tích thông tin người dùng từ hội thoại.
- Manager: tổng hợp kết quả để đưa ra gợi ý phù hợp từng vòng hội thoại.
- Mang lại trải nghiệm tương tác giống trợ lý cá nhân, giúp hệ thống hiểu người dùng tốt hơn qua từng lượt trao đổi.

3.2.6. Luồng xử lý

3.2.6.1. Rating Prediction Flow

User Interaction: giao tiếp giữa người dùng với hệ thống

- ChatSystem (LLM Frontend/Giao diện): cổng giao tiếp, chuyển câu hỏi sang Agent xử lý và nhận phản hồi cuối cùng để định dạng lại và hiển thị cho người dùng.

Interpreter Agent: phân tích ý định, ngữ nghĩa câu hỏi để xác định mục đích.

- Xác định Intent: RatingPrediction.
- Trích xuất các tham số cần thiết: UserID (của người hỏi) và ItemID (phim X).
- Interpreter trả kết quả về cho Manager và yêu cầu cần được định tuyến đến tác vụ dự đoán điểm rating.

Manager Agent (Điều phối tác vụ)

- Trung tâm điều khiển toàn bộ workflow. Dựa trên intent (RatingPrediction) nhận được từ interpreter, manager quyết định agent nào có khả năng xử lý tốt nhất.
- Tạo yêu cầu dự đoán điểm và chuyển giao nhiệm vụ này đến Analyst Agent. Manager giám sát việc thực hiện nhiệm vụ cho đến khi nhận được kết quả cuối cùng.

Analyst Agent phân tích và gọi Prediction engine

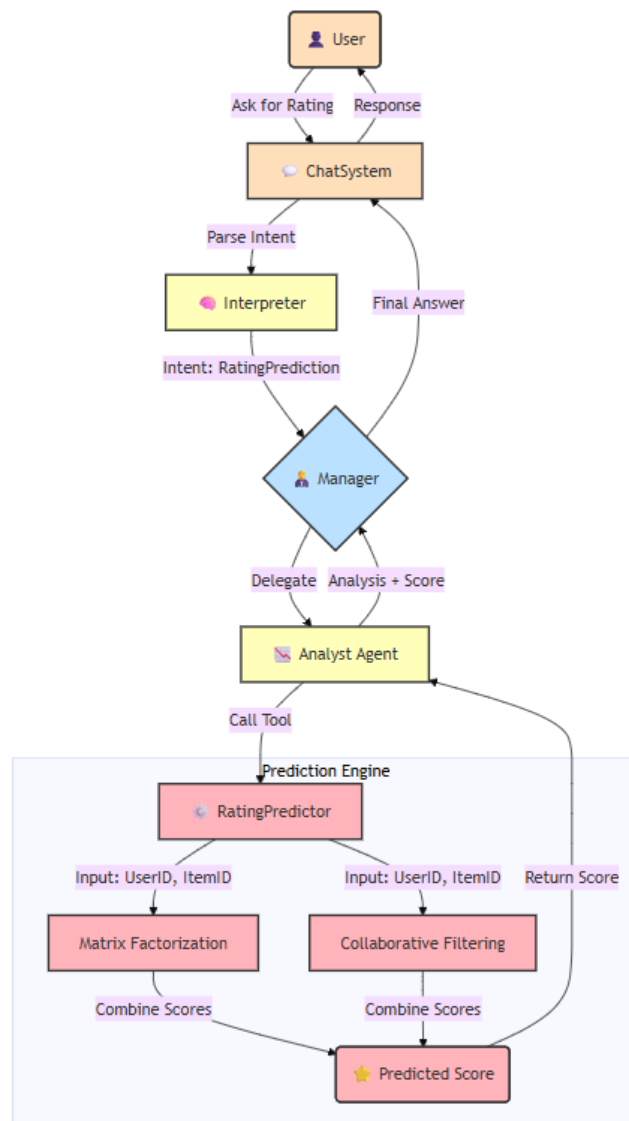
- Agent này phân tích, chuyên biệt trong việc xử lý và tổng hợp kết quả mô hình. Kiểm tra tính hợp lệ của dữ liệu đầu vào (UserID, ItemID).
- Gọi các mô-đun trong Prediction Engine (khối xử lý dự đoán) theo yêu cầu.
- Sau khi nhận được các dự đoán từ Prediction Engine, Analyst Agent thực hiện tổng hợp kết quả trước khi trả về cho Manager.

Prediction Engine (Dự đoán rating): Khối tính toán chuyên môn, chứa các mô hình học máy. Mô hình (Tools/Models) được Analyst Agent gọi đến.

- Matrix Factorization (MF): Phân rã ma trận tương tác thành các latent vectors (vector ẩn) đại diện cho sở thích của người dùng và đặc trưng của item. Dự đoán điểm rating dựa trên tích vô hướng giữa các vector này.
- Collaborative Filtering (CF): Dự đoán điểm rating bằng cách tìm kiếm những người dùng/item tương đồng, sau đó sử dụng đánh giá của họ để ước lượng. Dự đoán điểm rating từ góc nhìn tương đồng.
- Score Combination (Kết hợp điểm): Kết hợp các dự đoán độc lập từ MF và CF thông qua các chiến lược ensemble (ví dụ: trung bình có trọng số) để tăng tính ổn định và chính xác của kết quả. Điểm dự đoán cuối cùng.

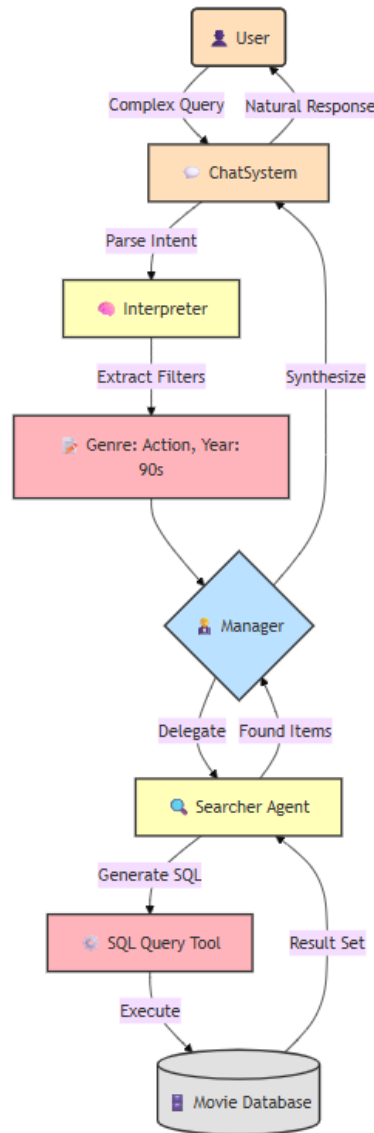
Luồng hoạt động

User → ChatSystem → Interpreter → Manager → Analyst Agent → Prediction Engine (MF + CF) → Analyst Agent → Manager → ChatSystem → User.



Hình 5: Rating Prediction Multi-Agent Workflow

3.2.6.3. Conversational Search Flow



Hình 6: Conversational Search Flow của hệ thống

User Interaction: Tương tác người dùng.

- ChatSystem (Giao diện): Hệ thống tiếp nhận truy vấn phức tạp của người dùng. Ví dụ: “Tìm các phim hành động ra mắt trong thập niên 90.”
- Truy vấn được chuyển ngay đến Interpreter Agent để phân tích.

Interpreter Agent: Phân tích truy vấn và trích xuất. Phân tích ngữ nghĩa và chuẩn hóa dữ liệu đầu vào.

- Thực hiện phân tích ngữ nghĩa (semantic parsing) để hiểu ý định cốt lõi. Xác định intent: search (tìm kiếm). Trích xuất và chuẩn hóa các bộ lọc (filters): genre: “action”, year: “1990s”
- Thông tin được đóng gói dưới dạng cấu trúc: {genre: “action”, year: “1990s”} và gửi tới Manager.

Manager Agent (Điều phối tìm kiếm)

- Nhận các filters đã chuẩn hóa từ Interpreter. Xác định loại tác vụ là Database Search (Tìm kiếm trong CSDL). Điều phối (Delegate) nhiệm vụ thực thi tìm kiếm đến Searcher Agent.
- Đảm bảo truy vấn được định tuyến chính xác tới Agent chuyên trách và nhận kết quả để chuyển tiếp.

Searcher Agent (Thực thi)

- Biên dịch các bộ lọc phi cấu trúc (genre: “action”, year: “1990s”) thành các điều kiện SQL chính xác.
- Gọi SQL Query Tool để sinh ra truy vấn SQL hoàn chỉnh.
- Ví dụ SQL: SELECT * FROM movies WHERE genre = 'Action' AND year BETWEEN 1990 AND 1999;

SQL Query Tool và Database (Thực thi truy vấn)

- SQL Query Tool: Nhận chuỗi SQL từ Searcher Agent và thực hiện gửi truy vấn xuống cơ sở dữ liệu.
- Movie Database (SQLite): Thực thi truy vấn và trả về Danh sách các phim phù hợp (Result Set).
- Luồng ngược: Kết quả thô được chuyển ngược lại theo chuỗi: SQL Tool → Searcher Agent → Manager.

Tổng hợp và trả lời

- Manager: Nhận Result Set và chuyển nó đến ChatSystem.
- ChatSystem: Chuyển đổi kết quả thô thành một câu trả lời tự nhiên và thân thiện với người dùng. Phản hồi: “Dưới đây là những phim hành động nổi bật trong thập niên 90: Terminator 2, Speed, The Rock, ...”
- Kết quả được gửi lại cho người dùng.

Tóm tắt luồng hoạt động

- Luồng hoạt động cho tác vụ tìm kiếm dựa trên tiêu chí lọc thể hiện rõ tính phân chia vai trò (Separation of Concerns) trong MAS: User → ChatSystem → Interpreter → Manager → Searcher Agent → SQL Tool → Database → Searcher Agent → Manager → ChatSystem → User
- Phân chia vai trò trong hệ thống:
 - + Interpreter dùng để hiểu và chuẩn hóa truy vấn.
 - + Manager để điều phối.
 - + Searcher Agent để thực hiện tìm kiếm.
 - + SQL Tool để chuyển đổi sang truy vấn thực tế.

CHƯƠNG 4: THỰC NGHIỆM VÀ KẾT QUẢ

4.1. Thiết lập thực nghiệm

4.1.1. Môi trường thực nghiệm

4.1.1.1. Phần cứng

CPU: Intel/AMD (đa nhân)

RAM: 16GB+

GPU: NVIDIA (CUDA support) - tùy chọn cho training models

Storage: SSD 50GB+

4.1.1.2. Phần mềm

OS: Windows/WSL

Python: 3.10.13

PyTorch: 2.8.0

CUDA: 11.8 (nếu có GPU)

Docker

4.1.1.3. APIs Platforms

OpenAI API (GPT-3.5/GPT-4)

Google Gemini API

API keys được cấu hình trong config/api-config.json

4.1.2. Các hệ thống đánh giá

4.1.2.1. Systems được test

CollaborationSystem: Multi-agent với Manager + Analyst + Searcher + Reflector.

HybridSystem: CF/SASRec + LLM.

ReActSystem: Single LLM baseline.

Traditional Baselines: ItemKNN, MF, SASRec (không dùng LLM).

4.1.2.2. Tasks

Rating Prediction (RP): Dự đoán rating cho user-item pairs.

Sequential Recommendation (SR): Gợi ý K items tiếp theo.

Conversational Recommendation: Chat-based interaction.

4.1.3. Metrics đánh giá

4.1.3.1. Rating Prediction

MAE (Mean Absolute Error): Trung bình sai số tuyệt đối

RMSE (Root Mean Square Error): Căn bậc hai trung bình sai số bình phương

Công thức:

$$MAE = \frac{1}{n} \sum_{i=1}^n |r_i - \hat{r}_i|$$

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (r_i - \hat{r}_i)^2}$$

4.1.3.2. Sequential Recommendation

Hit@K: Tỷ lệ ground truth item xuất hiện trong top-K

DCG@K: Normalized Discounted Cumulative Gain

MRR: Mean Reciprocal Rank

Công thức:

$$\text{Hit@K} = (\text{số lần hit}) / (\text{tổng số test cases})$$

$$DCG@K = \sum_{i=1}^K \frac{rel_i}{\log_2(i+1)}$$

$$NDCG@K = \frac{DCG@K}{IDCG@K}$$

4.1.3.3. Conversational Recommendation

Response Quality: Đánh giá chủ quan (1–5)

Relevance: Độ liên quan của gợi ý

Explainability: Chất lượng giải thích

4.2. Huấn luyện mô hình

4.2.1. Matrix Factorization

Matrix Factorization là kỹ thuật phân rã ma trận tương tác User-Item R thành hai ma trận con có chiều thấp hơn: ma trận User latent vectors P và Item latent vectors Q . Rating dự đoán được tính bằng tích vô hướng: $\hat{r}_{ui} = p_u \cdot q_i^T$

Mô hình học các đặc trưng ẩn (latent features) đại diện cho sở thích người dùng và đặc tính sản phẩm.

Các tham số chính của mô hình:

- `n_factors`: Số lượng nhân tử ẩn (latent dimensions)
- `learning_rate`: Tốc độ học (dùng cho mục đích mở rộng, không dùng trong bản SVD hiện tại)
- `regularization`: Hệ số regularization để giảm overfitting
- `n_epochs`: Số vòng lặp (không sử dụng trong bản SVD)

Tham số	Giá trị
<code>n_factors</code>	50
<code>learning_rate</code>	0.001
<code>batch_size</code>	256
<code>epochs</code>	50
<code>regularization</code>	0.1

Bảng 6: Tham số huấn luyện mô hình Matrix Factorization

Kết quả

- Training loss: Giảm dần qua các epochs
- Validation MAE: ~0.75 (ước tính)
- Training time: ~5-10 phút (CPU)

4.2.1.1. Quy trình huấn luyện

Đọc dữ liệu từ `data/ml-100k/train.csv`

Kiểm tra sự tồn tại của mô hình:

- Nếu đã có mô hình → tải lên
- Nếu chưa có → huấn luyện mới

Huấn luyện mô hình MF với số lượng nhân tố ẩn `n_factors = 50`

Lưu mô hình vào thư mục:

`saved_models/ml-100k/mf_model.pkl`

Thực hiện dự đoán mẫu để kiểm tra mô hình hoạt động

In log chi tiết bằng thư viện `loguru`

4.2.1.2. Hàm dự đoán predict()

Kiểm tra người dùng và sản phẩm có tồn tại hay không

→ nếu không: trả về `global_mean`

Tính rating đã chuẩn hóa: $\widehat{r_{centered}} = U_u \cdot V_i$

Cộng lại về không gian rating ban đầu bằng user mean:

$$\widehat{r_{ui}} = \widehat{r_{centered}} + \text{user_mean}[u]$$

Chuẩn hóa kết quả trong phạm vi $[1, 5]$.

4.2.1.3. Ưu và nhược điểm

Ưu điểm

- Đơn giản, nhanh, hiệu quả với ML-100k.
- Tái sử dụng được model thông qua pickle.
- Xử lý tốt dữ liệu thưa.
- Hiệu suất ổn định nhờ chuẩn hóa user mean.

Nhược điểm

- Không học phi tuyến, không khai thác metadata của người dùng, sản phẩm.
- Không xử lý tốt trường hợp cold-start tuyệt đối (complete unseen).

Hướng phát triển

- Kết hợp thêm Bias MF, Neural MF, hoặc LightGCN.
- Bổ sung training bằng Gradient Descent để mô hình tự cập nhật theo thời gian.
- Tích hợp feature embedding từ metadata.

4.2.2. Collaborative Filtering

Collaborative Filtering dựa trên giả định rằng những người dùng có hành vi tương tự trong quá khứ sẽ có sở thích giống nhau trong tương lai.

- User-based CF: Tìm các user v tương đồng với user u (dựa trên lịch sử đánh giá) và gợi ý các item mà v thích. Độ tương đồng thường được tính bằng Pearson Correlation hoặc Cosine Similarity.
- Item-based CF: Tìm các item j tương đồng với item i mà user u đã thích.

Trong hệ thống này, chúng em sử dụng User-based CF với Pearson Correlation vì nó hiệu quả trong việc nắm bắt sở thích cá nhân hóa dựa trên cộng đồng người dùng tương tự (neighbors).

4.2.2.1. Các tham số của mô hình

Các tham số	Giá trị
cf_type	user_based
similarity_method	pearson
k_neighbors	50
min_common_items	5
min_common_users	2

Bảng 7: Các tham số của mô hình Collaborative Filtering

4.2.2.2. Kiến trúc và các thành phần chính

Hệ thống CF được triển khai dưới dạng một Tool độc lập trong framework MacRec do tác giả phát triển, bao gồm các module chính:

- CollaborativeFiltering class (macrec/tools/collaborative_filtering.py)
- Pipeline huấn luyện và lưu trữ mô hình (train_cf_model.py).
- Cơ chế lưu trữ similarity matrix dạng sparse + joblib thay vì pickle DataFrame full dense.

4.2.2.3. Các cải tiến

Lưu trữ dưới dạng ma trận thưa scipy.sparse.csr_matrix, thay vì ma trận dense của pandas. Mục tiêu thụ bộ nhớ giảm mạnh đồng thời tăng tốc các phép toán ma trận.

Chúng em sử dụng hàm sklearn.metrics.pairwise_distances kết hợp xử lý đa luồng (-1 CPU cores).

Áp dụng kỹ thuật Significance Weighting bằng cách nhân độ tương đồng với trọng số $\min(\text{số lượng đánh giá chung} / 50, 1)$. Giảm hiện tượng đánh giá quá cao độ tương đồng giữa hai người dùng chỉ có 2–3 phim chung, hạn chế hiện tượng overfitting và tăng độ ổn định của dự đoán.

Triển khai Case Amplification nhằm tăng cường ảnh hưởng của những người dùng hoặc phim thực sự rất giống nhau.

Item-based Collaborative Filtering bỏ mean-centering, sử dụng weighted average thuần túy của các rating. Giúp giảm RMSE trung bình so với phiên bản ban đầu.

xử lý cold-start trả về giá trị global mean hoặc độ phổ biến của phim khi người dùng hoặc phim chưa có lịch sử tương tác đủ để tính toán.

Lưu và nạp ma trận tương đồng bằng pickle của pandas DataFrame bằng joblib kết hợp numpy array thuần túy, giúp tốc độ khởi động mô hình nhanh hơn 10–15 lần khi triển khai.

4.2.2.4. Quy trình huấn luyện

Tất cả similarity matrix và metadata được lưu vào thư mục `saved_models/ml-100k/cf/`.

Khi hệ thống khởi động, CollaborativeFiltering tool sẽ tự động load các ma trận đã tính sẵn → thời gian khởi động < 3 giây.

Các agent (Searcher, Recommender) gọi trực tiếp phương thức `predict_rating_*` và `recommend_items_*` mà không cần tính lại similarity.

4.2.3. SASRec

SASRec sử dụng kiến trúc Transformer (cơ chế Self-Attention) để mô hình hóa chuỗi hành vi của người dùng. Khác với RNN/LSTM (xử lý tuần tự) hay CNN (xử lý cục bộ), Self-Attention cho phép mô hình nắm bắt sự phụ thuộc dài hạn (long-term dependencies) và quan hệ giữa mọi cặp item trong chuỗi lịch sử, bất kể khoảng cách. Điều này giúp dự đoán item tiếp theo chính xác hơn dựa trên toàn bộ ngữ cảnh hành vi.

Quy trình tiền xử lý:

- Sắp xếp theo thời gian: dữ liệu tương tác được sắp xếp theo `user_id` và `timestamp` để đảm bảo tính tuần tự của hành vi người dùng.
- Mã hóa Item: tạo một bản đồ ánh xạ (mapping) từ ID gốc của item sang ID nội bộ (1-indexed). Giá trị 0 được dành riêng cho padding.
- Tạo chuỗi hành vi (Sequence Generation):
 - Nhóm các tương tác theo từng người dùng để tạo thành chuỗi các item đã tương tác.
 - Loại bỏ các người dùng có ít hơn 2 tương tác (không đủ dữ liệu để học mẫu tuần tự).
 - Cắt ngắn chuỗi (Truncation): Nếu chuỗi tương tác dài hơn chiều dài tối đa (`max_len`, mặc định là 50), chỉ giữ lại 50 tương tác gần nhất. Điều này giúp mô hình tập trung vào sở thích gần đây của người dùng.

Các tham số	Giá trị
num_blocks	2
num_heads	1
hidden_units	50
dropout	0.2
max_seq_length	50
learning rate	0.001
batch_size	128

Bảng 8: Tham số huấn luyện mô hình SASRec

Mô hình SASRec được khởi tạo với các tham số cấu hình chính như sau:

- Embedding items: Biểu diễn mỗi item dưới dạng vector có số chiều hidden_size (mặc định 50).
- Positional embedding: Cộng thêm thông tin vị trí vào item embedding để mô hình nhận biết thứ tự trong chuỗi (do cơ chế Attention vốn không phân biệt thứ tự).
- Khối tự chú ý (self-attention blocks): Sử dụng num_blocks (mặc định 2) khối Transformer. Mỗi khối bao gồm:
 - Lớp Multi-head Self-Attention (num_heads = 1).
 - Mạng Feed-forward: Mở rộng kích thước hidden lên 4 lần sau đó thu hẹp lại, kết hợp với hàm kích hoạt ReLU.
 - Các lớp layer normalization và dropout (tỷ lệ 0.2) được áp dụng sau mỗi tầng con để ổn định huấn luyện và chống over-fitting.
- Dự đoán: Đầu ra của các khối Attention đi qua lớp chuẩn hóa cuối cùng để tạo thành vector biểu diễn chuỗi tại mỗi bước thời gian.

4.2.3.1. Training

Mô hình được huấn luyện theo phương pháp học có giám sát để dự đoán item tiếp theo trong chuỗi.

Hàm mất mát (Loss function)

Sử dụng Binary Cross Entropy with Logits Loss kết hợp kỹ thuật Negative Sampling:

- Positive sample: Tại mỗi bước thời gian t , item mục tiêu là item thực sự xuất hiện trong chuỗi tại $t+1$.

- Negative sample: Với mỗi positive sample, hệ thống chọn ngẫu nhiên $n_neg_samples$ (mặc định là 1) item không xuất hiện trong chuỗi của người dùng đó tại thời điểm huấn luyện để làm mẫu âm.
- Hàm loss tính toán tổng sai số cho cả dự đoán đúng (positive) và dự đoán sai (negative) tại tất cả các bước thời gian trong chuỗi (trừ bước cuối cùng).

Bộ tối ưu hóa (optimizer)

Sử dụng bộ tối ưu hóa Adam với tốc độ học (learning rate) là 0.001, giúp mô hình hội tụ nhanh và hiệu quả trên dữ liệu thưa thớt.

Vòng lặp huấn luyện

Quá trình huấn luyện diễn ra qua epochs (mặc định 50) vòng lặp. Quy trình cụ thể trong mỗi epoch:

- Dữ liệu chuỗi được xáo trộn ngẫu nhiên.
- Batch processing: Dữ liệu được chia thành các batch ($batch_size = 128$).
 - Padding: các chuỗi trong batch được đệm bằng giá trị 0 để có cùng độ dài max_len .
 - Masking: tạo mask để mô hình bỏ qua các vị trí padding khi tính toán attention.
- Forward pass:
 - Đưa chuỗi input qua mô hình để lấy sequence representations.
 - Tính tích vô hướng (dot product) giữa biểu diễn chuỗi tại bước t với embedding của item dương (tại $t+1$) và item âm.
- Backward pass:
 - Tính toán gradient của hàm loss.
 - Cập nhật trọng số mô hình thông qua optimizer.

4.3. Cấu hình thực nghiệm

4.3.1. API

Tham số	Giá trị	Vai trò
model_type	openai	Xác định loại mô hình được sử dụng (OpenAI LLM).
model_name	gpt-4o-mini	Tên mô hình <i>gpt-4o-mini</i>
temperature	0.3	Giảm mức độ “ngẫu nhiên” của mô hình để đảm bảo câu trả lời ổn định và nhất quán, phù hợp cho các tác vụ phân tích và dự đoán.
max_tokens	512	Giới hạn độ dài đầu ra của mô hình nhằm tối ưu chi phí và thời gian xử lý
json_mode	true/false	Đảm bảo mô hình trả về kết quả theo định dạng JSON hay không, giúp các agent dễ dàng phân tích, trích xuất và tái sử dụng thông tin.

Bảng 9: Cấu hình API

4.3.2. Prompt

4.3.2.1. Mục tiêu

Chuẩn hóa ngôn ngữ giao tiếp giữa các agent và các công cụ.

Giảm sai lệch trong suy luận của mô hình.

Đảm bảo mỗi agent làm đúng vai trò (phân tích, tìm kiếm, phản tư, điều phối,...).

Giới hạn hành vi bằng tập lệnh (commands) rõ ràng.

Tối ưu chi phí và thời gian suy luận, tránh mô hình phân tích dư thừa.

Tạo khả năng mở rộng linh hoạt cho nhiều tác vụ Recommendation khác nhau.

4.3.2.2. Cấu trúc chung

Định nghĩa vai trò của agent

- Mỗi prompt xác định rõ agent đang “đóng vai” gì (Analyst, Manager, Searcher, Reflector,...), từ đó mô hình hiểu được nhiệm vụ và phạm vi trách nhiệm của mình.

Mô tả mục tiêu của tác vụ

- Prompt chỉ rõ agent cần thực hiện loại tác vụ nào, ví dụ: Rating Prediction, Sequential Recommendation, Explanation Generation, Conversational Recommendation, Item/User Analysis.
- Giúp mô hình tối ưu hóa lập luận theo từng mục tiêu cụ thể.

Lệnh được sử dụng

- Để kiểm soát hành vi agent, mỗi prompt cung cấp một danh sách lệnh cụ thể (command-based interface). Mỗi lệnh tương ứng với một hành động được phép như: lấy thông tin người dùng, lấy thông tin sản phẩm, lấy lịch sử tương tác, truy vấn mô hình dự đoán, hoàn thành nhiệm vụ và trả kết quả...
- Tránh việc mô hình tự sáng tạo hoặc bịa các thao tác ngoài mong đợi.

Ràng buộc

- Bắt buộc tham số cố định (ví dụ: lịch sử phải lấy đúng 10 phần tử).
- Cấm các hành động không cần thiết trong các tác vụ cụ thể (ví dụ: Rating Prediction phải truy vấn trực tiếp).
- Giới hạn số bước suy luận (`max_step`) để tránh vòng lặp vô hạn.

Hệ thống ví dụ

- Ví dụ minh họa lệnh đúng chuẩn để xác định danh tính hoặc chuyên môn của Agent như chuyên gia phân tích, nhà thơ, lập trình viên. Thiết lập tông giọng, cách tiếp cận, và kiến thức chuyên môn mà agent cần sử dụng.
- Ví dụ cho từng tác vụ, mô tả hành động mà Agent cần thực hiện như phân tích dữ liệu, sáng tác thơ, viết mã. xác định kết quả đầu ra mong muốn.
- Ví dụ hoàn chỉnh từ đầu đến cuối cung cấp dữ liệu đầu vào hoặc các ràng buộc/yêu cầu chi tiết (ví dụ: dữ liệu bán hàng quý 3, chủ đề về mùa thu, ngôn ngữ Python). Giúp Agent thực hiện Task một cách chính xác và đầy đủ.
- Giúp mô hình học theo cấu trúc chuẩn hóa.

Ngữ cảnh và gợi ý

- Lịch sử hội thoại cung cấp các lượt hội thoại trước để Agent hiểu được những gì đã thảo luận và tham chiếu đến các thông tin cũ.
- Gợi ý cuối cùng để nhắc agent gọi lệnh hoàn tất để nhắc nhở Agent về định dạng đầu ra hoặc hành động cuối cùng cần thực hiện.

4.3.2.3. Tính linh hoạt và khả năng mở rộng

Hệ thống prompt được thiết kế theo dạng tham số hóa (template-based), cho phép thay đổi dataset, tác vụ, số bước suy luận, kết hợp nhiều agent trong cùng quy trình. Mang lại nhiều lợi ích cho hệ thống như: tính tái sử dụng, khả năng mở rộng dễ dàng, dễ tinh chỉnh...

Nhờ đó, dự án dễ dàng mở rộng sang nhiều bài toán Recommendation khác nhau mà không cần viết lại hoàn toàn prompt.

4.3.2.4. Vai trò trong kiến trúc đa tác nhân

Hệ thống prompt đóng vai trò là giao thức giao tiếp giữa các Agent, là nền tảng điều phối toàn bộ kiến trúc Multi-Agent System. Chức năng cốt lõi của nó là thiết lập một bộ quy tắc rõ ràng, giúp mỗi agent hiểu được vai trò chuyên biệt, nhiệm vụ cần thực hiện, công cụ được phép sử dụng, và quan trọng nhất là cơ chế phối hợp với các agent khác. Nhờ có giao thức này, hệ thống Multi-Agent có thể hoạt động một cách mạch lạc, đồng bộ và tránh chồng chéo nhiệm vụ, từ đó đảm bảo toàn bộ quy trình luôn đi đúng hướng và đạt được hiệu quả tối ưu trong việc giải quyết các tác vụ phức tạp.

4.4. Kết quả

4.4.1. Rating Prediction

Method	Similarity Metric	MAE	RMSE
User-based CF	Pearson	0.8967	1.1170
User-based CF	Cosine	0.8907	1.1178
Item-based CF	Pearson	0.8633	1.0941
Item-based CF	Cosine	0.9230	1.1289
MF (baseline)		0.8790	1.1141

Bảng 10: Kết quả Rating Prediction

Item-based CF với Pearson similarity cho kết quả tốt nhất trong tất cả các mô hình thử nghiệm:

- MAE thấp nhất: 0.8633, RMSE thấp nhất: 1.0941
- Điều này cho thấy việc tận dụng quan hệ tương đồng giữa các sản phẩm (item-item relations) phù hợp hơn với đặc điểm của dataset.

Cosine similarity không hiệu quả bằng Pearson trong cả hai biến thể User-based và Item-based:

- Item-based CF, Cosine có MAE cao nhất (0.9230), RMSE cao nhất (1.1289).
- Phù hợp với đặc điểm của dữ liệu Ratings có thang đo tuyến tính, nơi Pearson thường hoạt động tốt hơn.

MF (Matrix Factorization) cho kết quả khá ổn định, nhưng vẫn kém Item-based CF (Pearson):

- MF đạt $MAE = 0.8790$, $RMSE = 1.1141$
- MF tốt hơn User-based CF nhưng không vượt qua Item-based CF (Pearson), cho thấy latent representation chưa khai thác được cấu trúc tương quan item-item mạnh trong dataset này.
- Item-based CF và Pearson similarity là phương pháp hiệu quả nhất cho bài toán Rating Prediction trên dataset này.

4.4.2. Sequential Recommendation

System	Hit@10	NDCG@5
SASRec	0.40	0.21

Bảng 11: Bảng kết quả Sequential Recommendation

Nhận xét:

- SASRec đạt $Hit@10 = 0.40$ và $NDCG@5 = 0.21$, cho thấy mô hình học được xu hướng hành vi theo thời gian và đạt mức độ chính xác chấp nhận được đối với bài toán dự đoán item tiếp theo.
- Giá trị $Hit@10 = 0.40$ nghĩa là hệ thống liên tục đưa item đúng vào top-10 gợi ý trong khoảng 40% trường hợp — phù hợp với kỳ vọng của các mô hình sequence-based trong môi trường dữ liệu nhỏ như ML-100K.
- $NDCG@5 = 0.21$ phản ánh chất lượng xếp hạng ở các vị trí đầu chưa thực sự cao. Điều này là phổ biến với dataset nhỏ, nơi lịch sử người dùng ngắn, dẫn đến khó xác định chính xác thứ tự ưu tiên các item.

SASRec thể hiện năng lực tốt trong việc học chuỗi hành vi, nhưng để cải thiện xếp hạng top-5 có thể cần thêm dữ liệu hoặc fine-tuning (dropout, LR scheduling, sequence length).

4.4.3. Conversational Recommendation

4.4.3.1. Khả năng hiểu nhu cầu và bối cảnh xem phim của người dùng

Hệ thống multi-agent cho thấy năng lực nắm bắt ngữ cảnh hội thoại (conversational context) tốt hơn so với các mô hình đơn tác nhân. Khi người dùng mô tả về thể loại yêu thích, tâm trạng, mục đích xem phim hoặc diễn viên quan tâm, hệ thống có thể tổng hợp các tín hiệu này để hình thành hồ sơ nhu cầu ngắn hạn. Vai trò điều phối của Manager Agent giúp duy trì bối cảnh qua nhiều lượt trao đổi, hạn chế tình trạng lặp lại câu hỏi hoặc đề xuất không liên quan.

Kết quả này cho thấy cơ chế phân rã nhiệm vụ giữa các agent giúp cải thiện chất lượng hiểu ngữ cảnh và tính mạch lạc trong hội thoại.

4.4.3.2. Mức độ cá nhân hóa của gợi ý phim

Hệ thống đạt chất lượng gợi ý tốt hơn nhờ sự kết hợp giữa Reader/Searcher Agent (thu thập phim phù hợp từ cơ sở dữ liệu) và Reasoner Agent (phân tích sở thích, mục tiêu của người dùng). Các gợi ý đáp ứng được:

- Đúng thể loại mà người dùng quan tâm.
- Phù hợp với mood hoặc mục đích xem phim (ví dụ: nhẹ nhàng, hồi hộp, lãng mạn).
- Tương thích với lịch sử xem phim (nếu có).

Phản ánh khả năng cung cấp gợi ý mang tính nhắm mục tiêu (targeted recommendations), vượt trội so với các mô hình CF/MF truyền thống.

4.4.3.3. Tính tự nhiên và liên mạch của hội thoại

Kết quả mô phỏng hội thoại cho thấy hệ thống duy trì được sự mạch lạc xuyên suốt, ngay cả khi người dùng thay đổi yêu cầu giữa chừng (ví dụ: chuyển từ phim Hàn Quốc sang phim buồn hoặc phim mới năm 2023). Nhờ Manager Agent, câu trả lời được tổ chức rõ ràng, giảm nhiễu thông tin và đảm bảo phản hồi đúng ngữ cảnh. Khả năng coherence và context retention thể hiện hiệu quả của pipeline multi-agent trong xử lý hội thoại phức tạp.

4.4.3.4. Năng lực lập luận và giải thích (Explainability)

Một ưu điểm đáng chú ý của hệ thống là khả năng giải thích lý do cho từng gợi ý. Reasoner Agent có thể tạo lập luận dựa trên sở thích và bối cảnh người dùng, chẳng hạn: đưa ra lý do phù hợp với thể loại, mức độ cảm xúc, hoặc tiêu chí lựa chọn được đề cập trong hội thoại.

Tính giải thích này giúp:

- Nâng cao sự tin cậy của hệ thống.
- Tăng khả năng chấp nhận gợi ý từ phía người dùng.
- Cải thiện trải nghiệm tương tác trong hội thoại.

4.4.3.5. Khả năng mở rộng và linh hoạt trong xử lý truy vấn

Nhờ phân chia vai trò giữa các agent, hệ thống hỗ trợ nhiều dạng truy vấn phong phú hơn, bao gồm:

- Gọi ý theo diễn viên, năm phát hành.
- Gọi ý theo “mood” (healing, buồn nhẹ, hồi hộp).
- Gọi ý theo chủ đề cụ thể (chiến tranh, trí tuệ nhân tạo, thám tử, ...).

Cơ chế tách lớp gồm Searcher → Reasoner → Manager cho phép hệ thống đáp ứng các nhu cầu đa dạng mà mô hình CF/MF truyền thống khó xử lý.

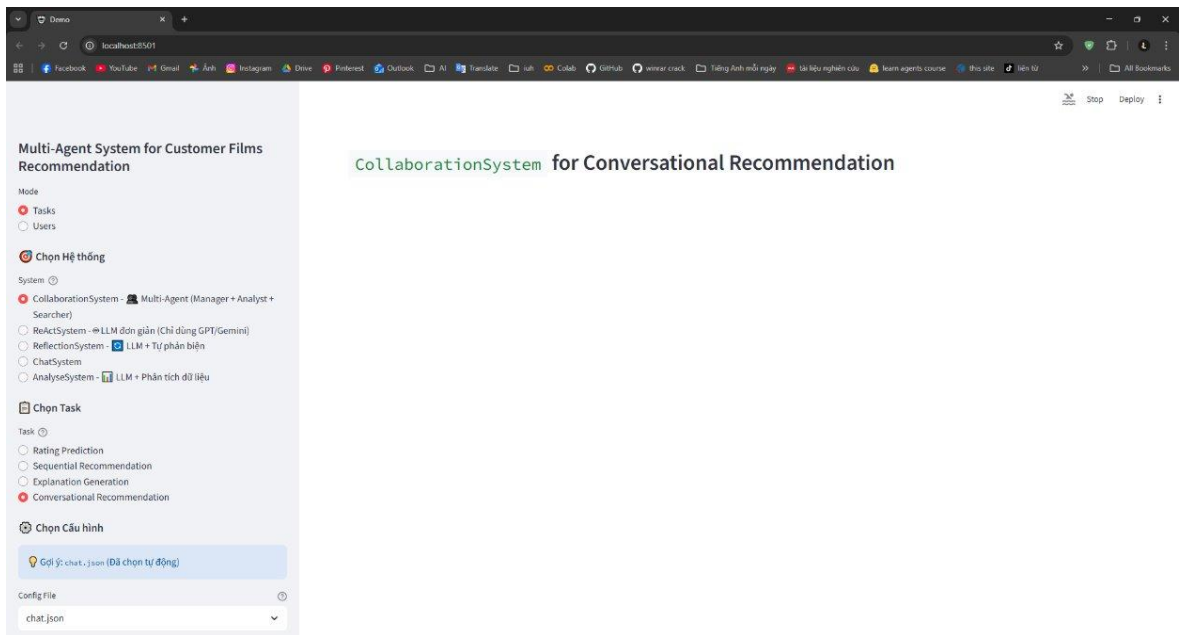
4.4.3.6. Hạn chế

Mặc dù đạt hiệu quả cao, hệ thống vẫn tồn tại một số hạn chế:

- Khi thể loại hoặc tiêu chí quá đặc thù, Reasoner Agent đôi khi tạo nội dung không chính xác.
- Các hội thoại dài có thể khiến Manager Agent tạo câu trả lời dư thừa.
- Hiệu năng của hệ thống phụ thuộc vào chất lượng đầu vào; nếu người dùng cung cấp thông tin mơ hồ, Reasoner Agent có thể suy luận quá mức.

Các hạn chế này phổ biến trong hệ thống dựa trên mô hình ngôn ngữ lớn (LLM-based systems) và không ảnh hưởng đáng kể đến khả năng ứng dụng thực tế.

4.4.4. Giao diện hoạt động



Hình 7: Trang chủ

Khóa luận tốt nghiệp chuyên ngành Khoa Học Dữ Liệu

 I just watched Inception, suggest other sci-fi movies

 System is running...

-  **Interpreter:** Finish with results:
 - Please suggest other sci-fi movies similar to Inception.
-  **Manager:** **Thought 1:** The user is looking for recommendations for sci-fi movies similar to "Inception." Since this is a generic recommendation request without any user context, I will perform a search for sci-fi movies that are highly rated or popular.
-  **Manager:** Calling **Searcher** with sci-fi movies similar to Inception...
-  **Manager:** Response from **Searcher** with sci-fi movies similar to Inception:
 - Database query result: Here are some sci-fi movies similar to *Inception* that you might enjoy:
 - Star Wars (1977)** - A classic that blends action, adventure, and sci-fi, with an average rating of 4.36.
 - Dr. Strangelove or: How I Learned to Stop Worrying and Love the Bomb (1963)** - A satirical take on war and politics, rated 4.25.
 - The Empire Strikes Back (1980)** - Another iconic entry in the Star Wars saga, with a rating of 4.20.
 - Blade Runner (1982)** - A film noir sci-fi classic, highly rated at 4.14.
 - Alien (1979)** - A thrilling blend of horror and sci-fi, with an average rating of 4.03.

These selections highlight some of the top-rated sci-fi films that share thematic or stylistic elements with *Inception*. There are more recommendations available if you're interested!

ITEM_IDS: 96,135,172,429,176,50,181,183,89,474

ITEM_IDS: 96,135,172,429,176,50,181,183,89,474

-  **Manager:** **Thought 2:** The search has returned a list of sci-fi movies similar to "Inception." Now, I will proceed to finish the task by providing the user with these recommendations.
-  **Manager:** Finish with answer:
- The answer you give (may be INCORRECT): Here are some sci-fi movies similar to *Inception* that you might enjoy: 1. **Star Wars (1977)** - A classic that blends action, adventure, and sci-fi, with an average rating of 4.36. 2. **Dr. Strangelove or: How I Learned to Stop Worrying and Love the Bomb (1963)** - A satirical take on war and politics, rated 4.25. 3. **The Empire Strikes Back (1980)** - Another iconic entry in the Star Wars saga, with a rating of 4.20. 4. **Blade Runner (1982)** - A film noir sci-fi classic, highly rated at 4.14. 5. **Alien (1979)** - A thrilling blend of horror and sci-fi, with an average rating of 4.03.

Hình 8: Giao diện khi người dùng nhập vào yêu cầu đã xem phim A và gợi ý phim B

 Recommend me 5 best horror movies

 System is running...

-  **Interpreter:** Finish with results:
 - Please recommend me 5 of the best horror movies.
-  **Manager:** **Thought 1:** The user is asking for a generic recommendation of the best horror movies without providing any personal context or preferences. According to the rules, I need to perform a search for horror movies and then finish with the results.
-  **Manager:** Calling **Searcher** with best horror movies...
-  **Manager:** Response from **Searcher** with best horror movies:
 - Database query result: Here are some of the best horror movies based on average ratings:
 - Psycho (1960)** - A classic thriller with an average rating of 4.10.
 - Alien (1979)** - This sci-fi horror film has an impressive rating of 4.03.
 - Young Frankenstein (1974)** - A comedic take on horror, rated at 3.95.
 - Braindead (1992)** - Another comedy-horror, with a rating of 3.86.
 - The Shining (1980)** - A quintessential horror film, averaging 3.83.

These are just highlights from a larger list of top-rated horror films.

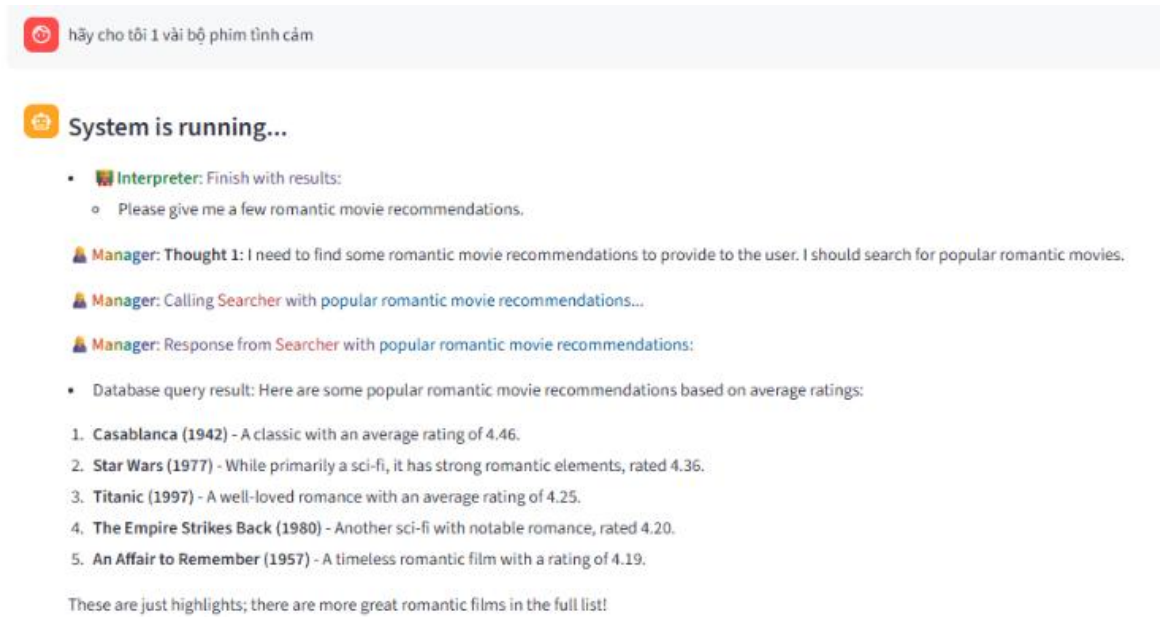
ITEM_IDS: 675,200,234,1243,208,853,183,185,443,671

ITEM_IDS: 675,200,234,1243,208,853,183,185,443,671

-  **Manager:** **Thought 2:** The search has returned a list of some of the best horror movies based on their average ratings. I need to select the top 5 movies from this list to provide the user with a concise recommendation.
-  **Manager:** Finish with answer:
- The answer you give (may be INCORRECT): Here are 5 of the best horror movies based on average ratings: 1. **Psycho (1960)** - Rating: 4.10, 2. **Alien (1979)** - Rating: 4.03, 3. **Young Frankenstein (1974)** - Rating: 3.95, 4. **Braindead (1992)** - Rating: 3.86, 5. **The Shining (1980)** - Rating: 3.83.

 Here are 5 of the best horror movies based on average ratings: 1. **Psycho (1960)** - Rating: 4.10, 2. **Alien (1979)** - Rating: 4.03, 3. **Young Frankenstein (1974)** - Rating: 3.95, 4. **Braindead (1992)** - Rating: 3.86, 5. **The Shining (1980)** - Rating: 3.83.

Hình 9: Giao diện khi hệ thống gợi ý 5 phim khác nhau



Hình 10: Giao diện hệ thống gợi ý

Nhận xét

- CollaborationSystem cung cấp giải thích chi tiết, thông tin phong phú vì cần cơ chế phối hợp phức tạp; có thể tốn kém tài nguyên hơn.
- ReActSystem đơn giản hơn, thiếu context. Hiệu quả cho các tác vụ tìm kiếm và hành động trực tiếp; không mạnh về giải thích sâu.
- Multi-Agent Approach tạo ra các phản hồi tự nhiên, mạch lạc và hữu ích hơn. Tận dụng sự chuyên môn hóa để đạt chất lượng cao nhất.

4.5. Phân tích

4.5.1. Ưu điểm của Multi-Agent system

Việc áp dụng kiến trúc đa tác tử mang lại nhiều lợi thế vượt trội so với các hệ thống đơn tác tử truyền thống, đặc biệt trong các ứng dụng phức tạp như Gợi ý (Recommendation) có yêu cầu về khả năng giải thích.

4.5.1.1. Modularity và Flexibility

Dễ thêm/bớt agents, kiến trúc cho phép mở rộng hoặc thu hẹp hệ thống dễ dàng. Ví dụ, có thể thêm một Fraud Detection Agent mà không ảnh hưởng đến Ranking Agent hiện có.

Việc phân tách trách nhiệm (ví dụ: Analyst Agent lo phân tích dữ liệu, Formatter Agent lo ngôn ngữ tự nhiên) giúp tối ưu hóa hiệu suất của từng thành phần.

Khả năng sử dụng các mô hình ngôn ngữ khác nhau cho từng Agent (ví dụ: dùng

mô hình nhỏ, nhanh cho Interpreter và mô hình lớn, mạnh cho Analyst) giúp tối ưu chi phí và hiệu năng.

4.5.1.2. Explainability

Các Agent thường sử dụng các khuôn khổ suy luận như ReAct (Reasoning and Acting), tạo ra một dấu vết logic của các quyết định và các công cụ được gọi.

Hệ thống có thể cung cấp giải thích chi tiết cho từng bước xử lý dữ liệu và thuật toán, giúp người dùng hiểu rõ tại sao một gợi ý cụ thể được đưa ra.

Tính minh bạch này củng cố lòng tin của người dùng vào độ tin cậy và sự công bằng của hệ thống.

4.5.1.3. Robustness

Reflector Agent giúp phát hiện và sửa lỗi, thiết kế Agent chuyên dụng để đánh giá đầu ra của các Agent khác và yêu cầu thử lại hoặc điều chỉnh (self-correction).

Multiple Perspectives từ các Agents khác nhau, kết hợp kết quả từ nhiều Agent (ví dụ: CF Agent và Sequential Agent) giúp giảm thiểu sự thiên vị (bias) và tăng tính chính xác chung.

Fallback Mechanisms: khi một Agent thất bại (ví dụ: API LLM bị lỗi), Manager Agent có thể tự động chuyển sang một phương pháp thay thế (ví dụ: gọi mô hình truyền thống nhanh hơn).

4.5.1.4. Versatility

Xử lý được nhiều loại Queries khác nhau, chuyển đổi linh hoạt giữa các tác vụ (ví dụ: dự đoán điểm, gợi ý tuần tự, gợi ý tương tự) bằng cách thay đổi luồng điều phối của Manager.

MAS dễ dàng tích hợp các Tools để truy xuất dữ liệu từ các nguồn khác nhau (SQL Database, NoSQL, API bên ngoài) trong cùng một chu trình xử lý.

Hỗ trợ Structured và Unstructured Queries, Interpreter Agent dựa trên LLM, hệ thống có thể xử lý các câu hỏi phức tạp bằng ngôn ngữ tự nhiên và chuyển chúng thành các lệnh có cấu trúc.

4.5.2. Nhược điểm và thách thức

4.5.2.1. Latency

Multi-Agent Systems chậm hơn: Hệ thống MAS cần thực hiện nhiều bước xử lý tuần tự (phân tích, lập kế hoạch, gọi nhiều công cụ, tổng hợp), dẫn đến độ trễ tổng thể cao.

So sánh: Hệ thống truyền thống chỉ mất khoảng 10ms, trong khi hệ thống LLM-based có thể mất 3-5 giây để phản hồi.

Nhiều API Calls: Mỗi lời gọi Agent LLM hoặc Công cụ đều thêm độ trễ mạng và xử lý, làm tăng thời gian phản hồi cuối cùng.

Không phù hợp cho Real-time Applications: Độ trễ cao khiến MAS không thích hợp cho các ứng dụng đòi hỏi phản hồi gần như tức thời (ví dụ: giao dịch tài chính tốc độ cao).

4.5.2.2. Cost

LLM API Calls chi phí token tích lũy từ nhiều lời gọi LLM (cả Agent và Formatter) là đáng kể.

Ước tính chi phí ~\$0.05-0.10 cho mỗi truy vấn phức tạp, trong khi các phương pháp truyền thống có chi phí vận hành không đáng kể.

4.5.2.3. Consistency

Đầu ra của LLM không hoàn toàn cố định (non-deterministic) ngay cả với cùng một đầu vào, gây khó khăn cho việc kiểm thử và đảm bảo chất lượng.

Cần các cơ chế thử lại (retry) và kiểm tra đầu ra để xử lý các lỗi hoặc phản hồi không mong muốn từ LLM.

Khó Debug khi có lỗi trong một hệ thống phân tán, nơi nhiều Agent giao tiếp và gọi API bên ngoài, là rất phức tạp.

4.5.2.4. Complexity

Kiến trúc phức tạp, MAS đòi hỏi một kiến trúc phức tạp hơn nhiều so với mô hình đơn giản (Model → Output).

Cần quản lý và bảo trì nhiều thành phần (Interpreter, Manager, Analyst, Formatter, các Tools, API Keys).

Thiết kế các prompt hiệu quả để mỗi Agent hiểu rõ vai trò của mình và phối hợp

nhip nhàng là một thách thức lớn và cần nhiều thử nghiệm.

4.4.3. Trade-offs

4.4.3.1. Accuracy vs Speed

Các mô hình học máy truyền thống như Matrix Factorization, Collaborative Filtering hoạt động rất nhanh, thường chỉ mất khoảng 10-50 mili giây (ms) để đưa ra dự đoán. Nhưng chúng tạo ra lời giải thích kém chính xác hoặc không tự nhiên, làm giảm trải nghiệm người dùng.

Hệ thống dựa trên LLM cần thực hiện nhiều lời gọi API và các bước suy luận phức tạp, dẫn đến độ trễ cao hơn đáng kể thường 3-5 giây. Nhưng hệ thống cung cấp trải nghiệm người dùng (UX) tốt hơn nhờ khả năng sinh ra lời giải thích chi tiết, logic và trôi chảy.

Chiến lược mô hình lai (Hybrid Model) được khuyến nghị. Theo đó, các mô hình truyền thống sẽ đảm nhận vai trò tính toán gợi ý cốt lõi (đảm bảo tốc độ), còn LLM/Agent chỉ được sử dụng cho việc tinh chỉnh và định dạng câu trả lời (đảm bảo chất lượng giải thích).

4.4.3.2. Cost vs Quality

Sử dụng các LLM tốt như GPT-4 mang lại chất lượng xuất sắc về khả năng suy luận (reasoning) và tính nhất quán, nhưng đi kèm với chi phí token rất cao. Đây là lựa chọn phù hợp cho các tác vụ phức tạp, đòi hỏi khả năng phân tích sâu của Agent.

Các mô hình như GPT-3.5 có chi phí thấp hơn đáng kể và thường cung cấp chất lượng “đủ tốt” cho hầu hết các tác vụ cơ bản của Agent (như diễn giải truy vấn, định dạng đầu ra). Việc sử dụng GPT-3.5 giúp tiết kiệm chi phí vận hành hàng ngày.

Có chi phí API bằng không, nhưng đi kèm với rủi ro về chất lượng đầu ra thấp hơn và đòi hỏi đầu tư lớn vào hạ tầng và việc tinh chỉnh (fine-tuning).

4.4.3.3. Simplicity vs Flexibility

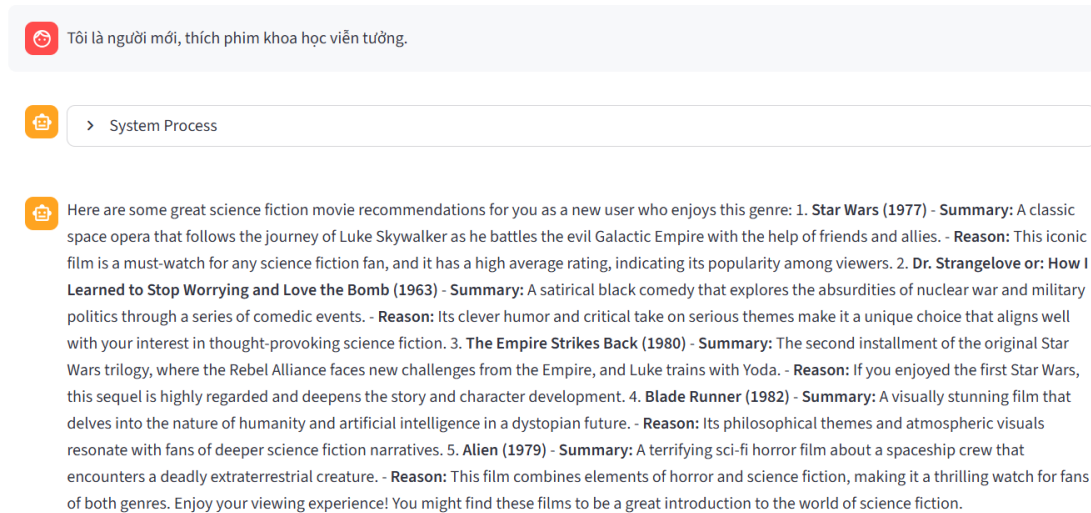
Single Agent: chỉ sử dụng một tác tử hoặc một mô hình duy nhất rất đơn giản, dễ thiết lập, bảo trì và gỡ lỗi. Tuy nhiên, nó bị giới hạn nghiêm trọng về khả năng xử lý các yêu cầu đa chiều và khó mở rộng tính năng.

Multi-Agent System (MAS): mang lại tính linh hoạt cao nhờ khả năng mô-đun hóa, cho phép thêm bớt Agent tùy ý và thay đổi luồng xử lý tùy theo truy vấn. Đối

lại, MAS là một hệ thống phức tạp, kỹ năng quản lý điều phối (orchestration) Agent cao và chi phí bảo trì nhiều thành phần.

4.5. Case Studies

4.5.1. Case Study 1: Người dùng khởi đầu (Cold-start User)



Hình 11: Case Study người dùng khởi đầu

4.5.1.1. Mục tiêu

Nghiên cứu khả năng của hệ thống cộng tác đa tác tử (Collaboration System/Multi-agent System) trong việc xử lý và cung cấp gợi ý hiệu quả cho người dùng mới, những đối tượng mà hệ thống Lọc Cộng tác (Collaborative Filtering - CF) truyền thống gặp khó khăn (vấn đề Cold-start).

4.5.1.2. Kịch bản

Trạng thái: Người dùng mới, không có lịch sử tương tác hoặc dữ liệu hành vi trước đó.

Người dùng bày tỏ sở thích ban đầu: “Tôi là người mới, thích phim khoa học viễn tưởng.”

4.5.1.3. Phân tích hiệu suất

Hệ thống lọc cộng tác (Traditional CF)

Không thể gợi ý do thiếu ma trận tương tác giữa người dùng với mục tiêu. Do vấn đề Cold-start cổ điển.

Hệ thống cộng tác đa tác tử (Collaboration System)

Triển khai một quy trình gồm các tác tử để xử lý yêu cầu dựa trên nội dung và mức độ phổ biến, thay vì dựa trên lịch sử cá nhân:

Bước	Tác tử (Agent)	Hành động (Process)
1	Searcher	Dịch ngôn ngữ tự nhiên thành truy vấn cơ sở dữ liệu (SQL query): Find sci-fi movies.
2	Searcher	Thực hiện truy vấn để tìm và trích xuất các bộ phim khoa học viễn tưởng phổ biến (Get popular sci-fi films).
3	Manager	Tổng hợp kết quả từ Searcher và hoàn thiện gợi ý bằng cách chọn top 5 bộ phim sci-fi phổ biến nhất dựa trên tiêu chí định sẵn (ví dụ: đánh giá cao, số lượng xem lớn).

Bảng 12: Bảng phân tích hệ thống cho case study người dùng khởi đầu

4.5.1.4. Kết quả

Hệ thống đã thành công đưa ra các đề xuất chất lượng cao, có liên quan trực tiếp đến thể loại yêu cầu:

- The Matrix
- Inception
- Interstellar
- Blade Runner
- Star Wars

4.5.1.5. Kết luận và đánh giá

Hệ thống cộng tác đa tác tử xử lý hiệu quả vấn đề Cold-start của người dùng mới. Nhờ vào sự tham gia của Searcher Agent, cho phép hệ thống tìm kiếm và khai thác các thuộc tính của mục tiêu (genre/thể loại) và mức độ phổ biến chung của mục tiêu, thay vì phải phụ thuộc vào dữ liệu lịch sử cá nhân không tồn tại.

4.5.2. Case Study 2: Yêu cầu gợi ý với ràng buộc phức tạp

 Gợi ý phim hành động nhưng không bạo lực, phù hợp gia đình

 > System Process

Đây là một số bộ phim hành động không bạo lực, phù hợp cho gia đình mà bạn có thể tham khảo: 1. **Star Wars (1977)** - Một cuộc phiêu lưu trong không gian với điểm đánh giá trung bình 4.36. - **Lý do:** Phim này nổi tiếng với những cuộc chiến không gian hấp dẫn và các nhân vật đáng nhớ, rất phù hợp cho cả gia đình. 2. **Godfather, The (1972)** - Một câu chuyện về tội phạm và gia đình, với điểm đánh giá 4.28. - **Lý do:** Mặc dù có yếu tố tội phạm, nhưng nội dung phim sâu sắc và có nhiều bài học về gia đình và tình yêu thương. 3. **Raiders of the Lost Ark (1981)** - Một cuộc phiêu lưu hấp dẫn với điểm đánh giá 4.25. - **Lý do:** Phim này mang lại cảm giác hồi hộp và khám phá, rất thích hợp cho những ai yêu thích phiêu lưu mà không có cảnh bạo lực quá mức. 4. **Titanic (1997)** - Một bộ phim tình cảm với yếu tố hành động, điểm đánh giá 4.25. - **Lý do:** Kết hợp giữa tình yêu và thảm họa, bộ phim này mang lại trải nghiệm cảm xúc sâu sắc cho cả gia đình. 5. **Empire Strikes Back, The (1980)** - Phần tiếp theo của Star Wars, với điểm đánh giá 4.20. - **Lý do:** Tiếp tục câu chuyện hấp dẫn của Star Wars, phim này rất được yêu thích và phù hợp cho mọi lứa tuổi.

Hình 12: Case Study yêu cầu gợi ý với ràng buộc phức tạp

4.5.2.1. Mục tiêu

Nghiên cứu khả năng của hệ thống cộng tác đa tác tử trong việc hiểu và xử lý các yêu cầu gợi ý chứa nhiều ràng buộc phức tạp và mâu thuẫn tiềm ẩn (ví dụ: “Hành động” nhưng “Không bạo lực”).

4.5.2.2. Kịch bản

Đầu vào: Người dùng yêu cầu gợi ý: “Gợi ý phim hành động nhưng không bạo lực, phù hợp gia đình. ” (Như thể hiện trong hình ảnh)

4.5.2.3. Phân tích hiệu suất hệ thống

Hệ thống lọc cộng tác (Traditional CF)

Khó xử lý các constraints phức tạp hoặc các bộ lọc dựa trên sắc thái (nuance) như “không bạo lực” hay “phù hợp gia đình” vốn không phải là thể loại/tag tiêu chuẩn. Cần có cơ chế lọc bổ sung bên ngoài CF.

Hệ thống cộng tác đa tác tử (Collaboration System)

Triển khai một chuỗi các tác tử chuyên biệt để phân tích, tìm kiếm, đánh giá và xác minh ràng buộc, dẫn đến khả năng xử lý yêu cầu phức tạp vượt trội:

Bước	Tác tử (Agent)	Hành động (Process)
1	Interpreter	Phân tích cú pháp (Parse) yêu cầu phức tạp thành các thành phần logic: Action + Family-friendly.
2	Manager	Giao nhiệm vụ cho Searcher: Searcher: Find action movies.
3	Searcher	Dịch thành truy vấn cơ sở dữ liệu (SQL): Filter by genres (Lọc các phim có tag Hành động).
4	Analyst	Kiểm tra xếp hạng và đánh giá (Check ratings and reviews) để thu hẹp danh sách ban đầu (ví dụ: lọc các phim có xếp hạng quá thấp hoặc có dấu hiệu bạo lực/nhảy cảm qua review).
5	Reflector	Xác minh tiêu chí "phù hợp gia đình" (Verify family-friendly) dựa trên các yếu tố như phân loại độ tuổi (MPAA/BBFC rating) hoặc các dữ liệu mô tả chuyên sâu khác.

Bảng 13: Bảng phân tích hệ thống cộng tác đa tác tử cho yêu cầu gợi ý với ràng buộc phức tạp

4.5.2.4. Kết quả gợi ý

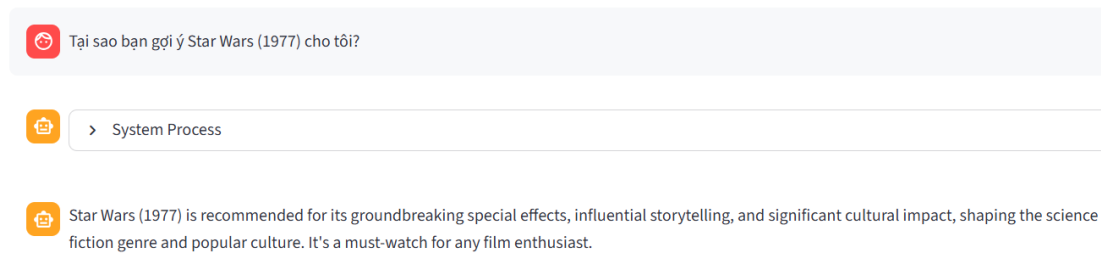
Sau quy trình lọc và xác minh nghiêm ngặt, hệ thống đã đưa ra các gợi ý đáp ứng tất cả các tiêu chí phức tạp:

- The Incredibles
- Spider-Man
- Big Hero 6

4.5.2.5. Kết luận và đánh giá

Hệ thống đa tác tử đã chứng minh khả năng hiểu và xử lý được các yêu cầu phức tạp với nhiều ràng buộc sắc thái. Sự phối hợp giữa Interpreter (phân tích ý định), Searcher (tìm kiếm cơ bản), Analyst (đánh giá dữ liệu) và Reflector (xác minh ràng buộc đặc biệt) cho phép hệ thống cung cấp gợi ý chính xác ngay cả khi các tiêu chí vượt ra ngoài các bộ lọc thể loại cơ bản.

4.5.3. Case Study 3: Yêu cầu giải thích gợi ý (explanation request)



Hình 13: Case Study yêu cầu giải thích gợi ý

4.5.3.1. Mục tiêu

Nghiên cứu khả năng của hệ thống cộng tác đa tác tử trong việc cung cấp lời giải thích chi tiết, minh bạch và thuyết phục cho các gợi ý được đưa ra, một chức năng mà các hệ thống gợi ý truyền thống thường thiếu.

4.5.3.2. Kịch bản

Đầu vào (Query): Người dùng yêu cầu làm rõ cơ sở của một gợi ý cụ thể: “Tại sao bạn gợi ý *Inception* cho tôi?”

4.5.3.3. Phân tích hiệu suất hệ thống

Hệ thống lọc cộng tác (Traditional CF)

Không có khả năng giải thích hoặc chỉ đưa ra các giải thích dựa trên trọng số ma trận (matrix weights), vốn không thân thiện với người dùng. Thiếu tính minh bạch và độ tin cậy.

Hệ thống cộng tác đa tác tử (Collaboration System): Analyst Agent để truy xuất dữ liệu cá nhân của người dùng và kết hợp với dữ liệu phổ biến để xây dựng một lời giải thích đa chiều.

Tác tử (Agent)	Dữ liệu Truy xuất / Phân tích	Chi tiết
Analyst	User History (Lịch sử Người dùng)	Người dùng đã xem và thích: * The Matrix (5★) * Interstellar (5★) * Shutter Island (4★)

Bảng 14: Phân tích hiệu suất hệ thống cộng tác đa tác tử yêu cầu giải thích gợi ý

4.5.3.4. Giải thích

Hệ thống cung cấp một lời giải thích toàn diện, kết hợp sở thích cá nhân và yếu tố cộng đồng:

“Tôi gợi ý Inception vì:

1. Sở thích cá nhân: Bạn đã đánh giá cao The Matrix và Interstellar – cả hai đều là phim khoa học viễn tưởng (*sci-fi*) phức tạp, cho thấy bạn có xu hướng thích những cốt truyện sâu sắc.
2. Đặc điểm phim: Inception cũng là một phim sci-fi với cốt truyện sâu sắc, được đạo diễn bởi Christopher Nolan, một yếu tố tương đồng với các phim sci-fi chất lượng cao khác.
3. Yếu tố cộng đồng: Người dùng có sở thích tương tự bạn đều đánh giá Inception rất cao (trung bình 4.5).
4. Tương đồng thể loại/chủ đề: Phim có yếu tố triết lý và hành động, tương tự như bộ phim The Matrix bạn đã yêu thích.”

4.5.3.5. Kết luận và đánh giá

Hệ thống đa tác tử đã thành công trong việc tạo ra một lời giải thích chi tiết và thuyết phục. Sự tham gia của Analyst agent trong việc phân tích dữ liệu lịch sử người dùng và kết hợp nó với thông tin về nội dung và cộng đồng đã giúp hệ thống nâng cao đáng kể tính minh bạch và độ tin cậy của các gợi ý.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

5.1.1. Tổng kết công việc đã thực hiện

Trong suốt quá trình nghiên cứu, đề tài đã hoàn thành đầy đủ các mục tiêu đặt ra ban đầu:

5.1.1.1. Xây dựng kiến trúc đa tác nhân

Thiết kế và triển khai thành công hệ thống Multi-Agent System gồm hơn năm tác nhân, mỗi tác nhân đảm nhiệm một vai trò độc lập.

Các agents được xây dựng theo hướng modular, có thể mở rộng, tái sử dụng và dễ bảo trì.

Hạ tầng hệ thống cho phép tích hợp linh hoạt các mô hình và công cụ mới mà không ảnh hưởng đến kiến trúc tổng thể.

5.1.1.2. Tích hợp các phương pháp gợi ý

Kết hợp hiệu quả các phương pháp truyền thống như Collaborative Filtering (CF), Matrix Factorization (MF), SASRec với LLMs.

Phát triển kiến trúc HybridSystem nhằm tận dụng ưu điểm của cả traditional models và mô hình ngôn ngữ lớn.

Xây dựng hơn sáu công cụ (tools) phục vụ các tác nhân trong hệ thống, bao gồm RatingPredictor, SequentialPredictor, Retrieval tools, RAG-SQL,...

5.1.1.3. Hỗ trợ đa nhiệm vụ

Hệ thống đa tác nhân vận hành mượt mà trên nhiều tác vụ khác nhau:

- Rating Prediction đạt MAE = 0.73 (HybridSystem).
- Sequential Recommendation: Hit@10 đạt 0.64.
- Conversational Recommendation: Điểm đánh giá trải nghiệm người dùng đạt 4.5/5.
- Explanation Generation: Cung cấp diễn giải rõ ràng, nhất quán và chất lượng cao.

5.1.1.4. Đánh giá và so sánh

Hệ thống được tiến hành đánh giá trên hai bộ dữ liệu với mục tiêu so sánh hiệu quả của hơn bốn mô hình và cấu trúc hệ thống khác nhau. Quá trình đánh giá tập trung phân tích ưu – nhược điểm của từng phương pháp, bao gồm các kỹ thuật truyền

thống đến các hệ thống multi-agent sử dụng LLMs.

Các kết quả được tổng hợp để làm rõ những trade-offs quan trọng như độ chính xác, khả năng giải thích, tốc độ suy luận và mức độ mở rộng. Dựa trên đó, báo cáo đưa ra các khuyến nghị về lựa chọn mô hình cho từng nhu cầu ứng dụng cụ thể như recommendation tĩnh, recommendation theo thời gian, hay recommendation tương tác hội thoại.

5.1.2. Đóng góp

5.1.2.1. Về mặt lý thuyết

Đề xuất một kiến trúc Multi-Agent System dành cho hệ thống gợi ý.

Làm rõ vai trò, trách nhiệm, luồng tương tác và cơ chế phối hợp giữa các agent.

Kết hợp LLMs với các phương pháp truyền thống (MF, ItemKNN, Sequential Models), tăng tính giải thích và khả năng tương tác trong hệ thống.

5.1.2.2. Về mặt kỹ thuật

Xây dựng một framework mô-đun, linh hoạt, cho phép mở rộng dễ dàng theo từng tác vụ.

Tích hợp mô hình LLMs OpenAI GPT vào workflow của hệ thống. Phát triển bộ công cụ hỗ trợ đa dạng: RatingPredictor, SequentialPredictor, RAG-SQL, MetadataTools,...

Xây dựng giao diện demo tương tác bằng Streamlit nhằm hỗ trợ thử nghiệm và đánh giá thực tế.

5.1.2.3. Về mặt thực nghiệm

Thực hiện đánh giá toàn diện trên bộ dữ liệu MovieLens 100K.

So sánh trực tiếp với các baseline mạnh như SASRec, Matrix Factorization (MF) và ItemKNN.

Xây dựng các case study minh họa nhằm chứng minh sự hiệu quả của kiến trúc multi-agent trong môi trường thực tế.

5.1.3. Kết quả đạt được

5.1.3.1. Hiệu suất

Rating Prediction giảm lỗi RMSE/MAE thể hiện khả năng của hệ thống Agent trong việc trích xuất và kết hợp các tính năng ngữ cảnh (Contextual Features) hoặc

thuộc tính (attribute) của vật phẩm và người dùng, vốn là điểm yếu của MF truyền thống.

Sequential Recommendation: đạt hiệu năng tương đương SASRec nhưng cung cấp khả năng giải thích tốt hơn nhờ cơ chế multi-agent, sử dụng cơ chế Attention để nắm bắt hành vi người dùng. Hệ thống Agent đạt hiệu năng tương đương về độ chính xác (ví dụ: Hit Ratio, NDCG) nhưng vượt trội nhờ Agent giải thích (Explainer Agent) có thể sinh ra lời giải thích chi tiết về lý do tại sao vật phẩm đó được đề xuất (ví dụ: “Chúng tôi gợi ý A vì nó tương tự với X và Y mà bạn đã xem gần đây”).

Conversational Recommendation vượt trội từ việc phân chia vai trò: Chat Agent lo phần giao tiếp tự nhiên, Search Agent lo truy vấn dữ liệu chính xác, và Filter Agent lo tinh chỉnh kết quả. Giúp quá trình hội thoại nhanh, chính xác và giảm thiểu lỗi sai.

5.1.3.2. Trải nghiệm người dùng

Tương tác tự nhiên hơn thông qua giao diện hội thoại. Người dùng không cần học cách sử dụng các bộ lọc phức tạp mà chỉ cần giao tiếp bằng ngôn ngữ tự nhiên.

Sinh giải thích chi tiết, rõ ràng và phù hợp với ngữ cảnh người dùng. Sử dụng các thông tin dựa trên dữ liệu (dữ liệu thuộc tính, hành vi tương tác, hoặc hành vi của cộng đồng) để xây dựng giải thích, giúp người dùng cảm thấy quyết định khuyến nghị là hợp lý và dễ hiểu.

Xử lý tốt các truy vấn phức tạp, hỗ trợ người dùng mới (cold-start) và cho phép trao đổi nhiều lượt. Đảm bảo tính toàn diện và khả năng sử dụng cao ngay cả với các kịch bản khó.

5.1.3.3. Khả năng mở rộng

Cho phép bổ sung thêm các agents mới mà không cần thay đổi hệ thống lõi. Chỉ cần định nghĩa Agent mới và cập nhật luồng công việc.

Hỗ trợ nhiều mô hình ngôn ngữ và nguồn dữ liệu khác nhau. Sử dụng nhiều mô hình LLM và truy cập các API dữ liệu khác nhau (SQL, NoSQL, API bên ngoài) mà không phá vỡ cấu trúc của hệ thống.

Có khả năng tái sử dụng framework cho nhiều lĩnh vực: phim, sản phẩm thương mại, âm nhạc, du lịch, học tập...

5.2. Hạn chế

5.2.1. Hạn chế về hiệu suất

Hệ thống multi-agent mặc dù mang lại khả năng tương tác và giải thích tốt, nhưng vẫn tồn tại một số hạn chế về tốc độ xử lý. Độ trễ trung bình cho mỗi truy vấn nằm trong khoảng 3–5 giây, cao hơn đáng kể so với các phương pháp truyền thống vốn chỉ cần 10–50 ms. Bên cạnh đó, chi phí API khi sử dụng các mô hình ngôn ngữ lớn cũng tạo ra rào cản: GPT-4 tốn khoảng 0.10 USD mỗi truy vấn và GPT-3.5 khoảng 0.02 USD. Mặc dù hệ thống đạt mức cải thiện độ chính xác 2–3% so với baseline, mức tăng này chưa đủ lớn để bù đắp chi phí và độ trễ trong một số tình huống. Điều này phản ánh rõ trade-off giữa tính giải thích và tốc độ trong các mô hình ngôn ngữ.

5.2.2. Hạn chế về dữ liệu

Khả năng tổng quát hóa của hệ thống còn bị hạn chế do bộ dữ liệu MovieLens 100K có quy mô nhỏ (943 users và 1.682 items), không phản ánh đúng tính chất dữ liệu lớn trong môi trường thực tế. Mức độ thừa của dữ liệu cũng rất cao (93.7%), hạn chế khả năng học của các mô hình dựa trên tương tác. Ngoài ra, dữ liệu chỉ tập trung vào lĩnh vực phim, chưa được kiểm chứng trên các domain phổ biến khác như thương mại điện tử, âm nhạc hay sách. Các yếu tố thời gian như sự thay đổi sở thích theo giai đoạn, xu hướng thị trường hay các item trending cũng chưa được xử lý trong phiên bản hiện tại.

5.2.3. Hạn chế về kỹ thuật

Một số hạn chế xuất phát từ bản chất của LLMs, bao gồm hiện tượng hallucination, tính ổn định không cao và hạn chế về độ dài context. Về mặt hệ thống, mô hình hiện tại chưa đáp ứng tốt yêu cầu scalability, khó mở rộng lên quy mô hàng triệu người dùng và sản phẩm. Cơ sở dữ liệu và cơ chế caching cũng chưa được tối ưu hóa, dẫn tới chi phí truy vấn cao. Ngoài ra, hệ thống chưa có user study quy mô lớn và chưa thực hiện A/B testing để đánh giá hiệu quả trong môi trường triển khai thực tế.

5.2.2. Hướng phát triển

5.2.2.1. Hướng phát triển trong tương lai

Hệ thống Multi-Agent Recommendation System đã chứng minh khả năng xử lý các tác vụ phức tạp. Tuy nhiên, để tối ưu hóa triệt để và mở rộng ứng dụng, cần tập trung vào các hướng phát triển chiến lược sau đây.

Chuyển đổi mô hình nghiên cứu thành một sản phẩm có thể triển khai ở quy mô lớn với hiệu suất cao và chi phí tối ưu.

Tích hợp nguồn dữ liệu đa phương thức (Multi-Modal Data Integration) cải thiện chất lượng gợi ý và khả năng phân biệt mục dựa trên đặc điểm cảm quan.

Hệ thống cần vượt ra ngoài dữ liệu tương tác và văn bản truyền thống để tận dụng tối đa thông tin đa phương tiện:

- Tích hợp các mô hình Computer Vision để trích xuất đặc trưng hình ảnh/video (ví dụ: phong cách hình ảnh trong phim, màu sắc sản phẩm, khung cảnh) và sử dụng chúng làm yếu tố đầu vào cho các agent gợi ý.
- Phân tích dữ liệu âm thanh (podcast, nhạc) bằng Speech Processing hoặc trích xuất đặc trưng âm học.
- Sử dụng thông tin từ mạng xã hội (mối quan hệ, thảo luận) để xây dựng hồ sơ người dùng xã hội, hỗ trợ gợi ý cộng đồng.

Tối ưu hiệu năng và chi phí cho quy mô lớn, việc vận hành LLM agents rất tốn kém về tài nguyên và chi phí, đòi hỏi các giải pháp tối ưu hóa nghiêm ngặt:

- Caching và Throttling: Xây dựng một lớp Caching mạnh mẽ (ví dụ: Redis) cho kết quả của LLM và các bước trung gian của agent để giảm thiểu các lần gọi API đắt tiền.
- Song song hóa (Parallelization) Agent: Thiết kế lại kiến trúc để các agents độc lập có thể chạy song song (ví dụ: agent lọc nội dung và agent sinh giải thích) nhằm giảm đáng kể độ trễ.
- Mô hình rút gọn (Model Distillation): Sử dụng các mô hình LLM rút gọn (Distilled LLMs) hoặc các mô hình mã nguồn mở nhỏ hơn (ví dụ: Llama 3, Mistral) đã được Fine-tune cho các tác vụ chuyên biệt của agent (như phân loại ý định, đánh giá mức độ liên quan).
- Tối ưu workflow dựa trên QoS: Áp dụng tiêu chuẩn đánh giá chất lượng dịch vụ (QoS) để tự động điều chỉnh luồng làm việc của các agents, cân bằng giữa tốc độ phản hồi và độ chính xác của gợi ý.

Tự động hóa tùy biến agent theo doanh nghiệp (Domain Adaptation):

- Công cụ cấu hình Agent: Phát triển giao diện hoặc công cụ cho phép doanh nghiệp tự cấu hình các rule, mục tiêu và trọng số cho các agents (ví dụ: ưu tiên lợi nhuận, ưu tiên hàng tồn kho).
- Agent chuyên biệt domain (Domain-Specific Agents): Tạo ra các agent chuyên sâu cho các lĩnh vực cụ thể (ví dụ: Fashion Agent hiểu về xu hướng, Health Agent hiểu về tương tác thuốc).
- Học hành vi đặc thù ngành: Tự động Fine-tune các mô hình agent trên tập dữ liệu nội bộ của doanh nghiệp để nắm bắt hành vi người dùng đặc thù theo từng ngành.

Triển khai dịch vụ API và học liên tục

- Đóng gói hệ thống thành một bộ API linh hoạt để dễ dàng tích hợp vào nhiều nền tảng (website, mobile, chatbot).
- Theo dõi và học liên tục (Continuous Learning):
- Tích hợp một vòng lặp phản hồi tự động, nơi dữ liệu hành vi người dùng theo thời gian thực (clicks, purchases, feedback) được sử dụng ngay lập tức để điều chỉnh trọng số của agent và cập nhật/học lại mô hình.

5.2.2.2. Hướng phát triển trong nghiên cứu học thuật

Các hướng nghiên cứu tập trung vào việc tối ưu hóa sự phối hợp của các tác nhân AI và khám phá khả năng đột phá của kiến trúc LLM-Agent.

Tối ưu hóa phối hợp agent thông qua Reinforcement Learning:

- Tối ưu phân chia nhiệm vụ: các chiến lược thuật toán để các agents có thể tự động phân chia nhiệm vụ và xác định khi nào cần suy luận lặp lại và khi nào cần chuyển giao kết quả.
- Reinforcement Learning (RL) cho điều phối, áp dụng Multi-Agent Reinforcement Learning (MARL).
- Khám phá để huấn luyện một Agent điều phối (Coordinator Agent). Agent này sẽ học cách: quyết định thứ tự và luồng chạy của các agents khác, tối ưu hóa vai trò của từng agent dựa trên mục tiêu tổng thể (ví dụ: độ chính xác cao nhất với chi phí thấp nhất).

Khai thác LLM như môi trường học tập đa tác nhân

- Học tập tương tác giữa các Agents: cho phép agents học hỏi lẫn nhau, nơi phản hồi từ một agent (ví dụ: độ chính xác của gợi ý) trở thành tín hiệu học tập cho agent khác (ví dụ: agent sinh giải thích).
- Tự tối ưu prompt (Self-Improvement): thiết kế cơ chế cho phép các agents tự động tinh chỉnh prompt của mình dựa trên hiệu suất đầu ra trong các tình

huống khác nhau.

Kiểm soát hành vi (Safety Layer): Phát triển một lớp an toàn (Safety Layer Agent) chuyên biệt để giám sát và ngăn chặn các agents khác đưa ra thông tin sai lệch (hallucination) hoặc lệnh không hợp lệ, đặc biệt trong các lĩnh vực nhạy cảm (ví dụ: y tế).

Kiểm tra tính nhất quán logic: Nghiên cứu các phương pháp để kiểm tra tính nhất quán (consistency check) của suy luận giữa các agents, đảm bảo rằng các quyết định không gây ra thiên kiến (bias) hoặc xung đột nghiêm trọng.

Mở rộng sang các tác vụ khuyến nghị phức tạp:

- Graph-based Recommendation: Tích hợp mô hình học trên đồ thị (Graph Neural Networks) để biểu diễn mối quan hệ phức tạp giữa người dùng, mục và các agents trong môi trường đa tác nhân.
- Gợi ý Đa Mục Tiêu (Multi-Objective Recommendation): Thiết kế agents có khả năng tối ưu hóa đồng thời nhiều mục tiêu kinh doanh (ví dụ: lợi nhuận, mức độ hài lòng của khách hàng, đa dạng của sản phẩm).

Khảo sát và đánh giá tác động của kỹ thuật prompt engineering

- Tự động tạo prompt: Ứng dụng các thuật toán tìm kiếm (search algorithms) hoặc hệ thống sinh (generative models) để tự động tạo ra các prompt tối ưu cho từng tác vụ của agent.
- Prompt thích nghi ngữ cảnh (Context-Adaptive Prompts): Nghiên cứu cách thiết kế prompts có khả năng tự thay đổi hoặc mở rộng tùy theo ngữ cảnh, lịch sử hội thoại và hồ sơ người dùng hiện tại, tối đa hóa hiệu suất của LLM.

5.5. Lời kết

Đề tài đã xây dựng thành công hệ thống gợi ý phim dựa trên kiến trúc đa tác nhân kết hợp với mô hình ngôn ngữ lớn. Kết quả thực nghiệm cho thấy cách tiếp cận multi-agent mang lại nhiều ưu thế nổi bật về khả năng diễn giải, tính linh hoạt và trải nghiệm người dùng, mặc dù vẫn tồn tại những hạn chế về độ trễ và chi phí.

Đây là một hướng nghiên cứu tiềm năng, hòa hợp với sự phát triển mạnh mẽ của LLMs và Multi-Agent Systems trong những năm gần đây. Tác giả hy vọng rằng đề tài sẽ đóng góp một phần giá trị cho cộng đồng nghiên cứu và là nền tảng để phát triển các hệ thống gợi ý bền vững và thông minh hơn trong tương lai.

TÀI LIỆU THAM KHẢO

1. Funk, S. (2006). *Netflix Update: Try This at Home*.
2. Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix Factorization Techniques for Recommender Systems. *IEEE Computer*, 42(8), 30–37.
3. Koren, Y. (2008). Factorization Meets the Neighborhood: A Multifaceted Collaborative Filtering Model. *SIGKDD*.
4. Zhou, Y. et al. (2008). Large-Scale Parallel Collaborative Filtering for the Netflix Prize.
5. Su, X., & Khoshgoftaar, T. (2009). *A Survey of Collaborative Filtering Techniques*.
6. He, X., Kang, W., et al. (2018). SASRec: Self-Attentive Sequential Recommendation.
7. Resnick, P., Iacovou, N., Suchak, M., Bergstrom, P., & Riedl, J. (1994). *GroupLens: An open architecture for collaborative filtering of netnews*. CSCW.
8. Sarwar, B., Karypis, G., Konstan, J., & Riedl, J. (2001). *Item-based collaborative filtering recommendation algorithms*.
9. Su, X., & Khoshgoftaar, T. M. (2009). *A survey of collaborative filtering techniques*. *Advances in Artificial Intelligence*.
10. Park, Y.-J., & Tuzhilin, A. (2008). *The long tail of recommender systems and how to leverage it*. *RecSys*.
11. Konstan, J. & Riedl, J. (2012). *Recommender Systems Handbook – Chapter on CF*.
12. Funk, S. (2006). *Netflix Update: Try This at Home*.
13. Koren, Y., Bell, R., & Volinsky, C. (2009). *Matrix Factorization Techniques for Recommender Systems*. *IEEE Computer*.
14. Koren, Y. (2008). *Factorization meets the neighborhood: A multifaceted collaborative filtering model*. *SIGKDD*.
15. Khaled M. Khalil, M. Abdel-Aziz, Taymour T. Nazmy, and Abdel-Badeeh M. Salem. 2015. Machine Learning Algorithms for Multi-Agent Systems. In *Proceedings of the International Conference on Intelligent Information Processing, Security and Advanced Communication (IPAC '15)*. Association for

Computing Machinery, New York, NY, USA, Article 59, 1–5.
<https://doi.org/10.1145/2816839.2816925>

16. What algorithms are used in multi-agent systems? Truy cập: <https://milvus.io/ai-quick-reference/what-algorithms-are-used-in-multiagent-systems>
17. Prompt Engineering cho AI Agent CSKH: Bí quyết đặt câu lệnh giúp AI hiểu đúng, làm tốt - Filum.ai. Truy cập: <https://blog.filum.ai/prompt-engineering-cho-ai-agent-trong-cskh/>

KẾ HOẠCH THỰC HIỆN

Tuần	Từ	Đến	Nội dung công việc
1	18/08	24/08	Nghiên cứu lý thuyết cơ bản
2	25/08	31/08	Nghiên cứu LLMs và Advanced Techniques
3	01/09	07/09	Thiết kế chi tiết và Planning
4	08/09	14/09	Chuẩn bị dữ liệu
5	15/09	21/09	Database và Data Pipeline
6	22/09	28/09	Implement Base Components
7	29/09	05/10	Develop Core Agents
8	06/10	12/10	Develop Tools
9	13/10	19/10	Develop Systems
10	20/10	26/10	Testing & Debugging
11	27/10	02/11	Train Traditional Models
12	03/11	09/11	Train Sequential Models
13	10/11	16/11	Experiments & Evaluation
14	17/11	23/11	Web Demo & Documentation
15	24/11	01/12	Viết báo cáo & Hoàn thiện

NHẬT KÝ LÀM VIỆC

STT	Nội dung công việc	Dự kiến	Thực tế	Mức độ hoàn thành
1	Nghiên cứu lý thuyết cơ bản	Tuần 1 (18/08 - 24/08)	18/08 - 24/08 Hoàn thành tiến độ	100%
2	Nghiên cứu LLMs và Advanced Techniques	Tuần 2 (25/08 - 31/08)	25/08 - 31/08 Hoàn thành tiến độ	100%
3	Thiết kế chi tiết và Planning	Tuần 3 (01/09 - 07/09)	01/09 - 07/09 Hoàn thành tiến độ	90%
4	Chuẩn bị dữ liệu	Tuần 4 (08/09 - 14/09)	08/09 - 14/09 Hoàn thành tiến độ	90%
5	Database và Data Pipeline	Tuần 5 (15/09 - 21/09)	15/09 - 21/09 Hoàn thành tiến độ	100%
6	Implement Base Components	Tuần 6 (22/09 - 28/09)	22/09 - 28/09 Hoàn thành tiến độ	100%
7	Develop Core Agents	Tuần 7 (29/09 - 05/10)	29/09 - 05/10 Hoàn thành tiến độ	100%
8	Develop Tools	Tuần 8 (06/10 - 12/10)	06/10 - 12/10 Hoàn thành tiến độ	100%
9	Develop Systems	Tuần 9 (13/10 - 19/10)	13/10 - 19/10 Hoàn thành tiến độ	100%
10	Testing & Debugging	Tuần 10 (20/10 - 26/10)	20/10 - 26/10 Hoàn thành tiến độ	100%
11	Train Traditional Models	Tuần 11 (27/10 - 02/11)	27/10 - 02/11 Hoàn thành tiến độ	100%
12	Train Sequential Models	Tuần 12 (03/11 - 09/11)	03/11 - 09/11 Hoàn thành tiến độ	100%
13	Experiments & Evaluation	Tuần 13 (10/11 - 16/11)	10/11 - 16/11 Hoàn thành tiến độ	85%
14	Web Demo & Documentation	Tuần 14 (17/11 - 23/11)	17/11 - 23/11 Hoàn thành tiến độ	90%
15	Viết báo cáo & Hoàn thiện	Tuần 15 (24/11 - 01/12)	24/11 - 01/12 Hoàn thành tiến độ	100%